

SpringBoot y Angular(BS)

Tabla de contenidos

- SpringBoot y Angular(BS)
- Tabla de contenidos
 - Introducción
 - Instalación
 - FASE 1 - Crear Proyecto y configurar conexión BBDD
 - Paso1. Crear proyecto con Spring Initializr en VS Code
 - Paso2. Configurar aplicación y conectar con BBDD
 - FASE 2 - Entidades, repositorios y datos iniciales
 - Paso 3. Crear entidades (modelo)
 - Paso 4. Ejecutar la aplicación y crear las tablas
 - Paso 5. Creamos los repositorios para todas las entidades.
 - Paso 6. Ahora vamos a introducir unos datos iniciales de prueba en la BBDD.
 - FASE 3: API REST Básico - Instalar Swagger/Postman
 - 3.1. Crear Controlador API REST básico para Alumnos
 - 3.2. Instalación de Postman
 - 3.3. Instalación de Swagger (Springdoc OpenAPI v3)
 - 3.4. Ampliando API REST
 - 3.5. JOINS
 - FASE 4: Bootstrap 5 y Fetch API
 - 4.1. ¿Qué es Bootstrap? CDN, descarga local y NPM
 - 4.2. Sistema de rejilla (Grid) y responsive
 - 4.3. Tipografía, colores y utilidades
 - 4.4. Componentes: navbar, cards, badges y alerts
 - 4.5. Tablas Bootstrap
 - 4.6. Modales
 - 4.7. Formularios y validación visual
 - 4.8. Fetch API: GET — listar datos reales
 - 4.9. Fetch API: POST — insertar alumno
 - 4.10. Fetch API: PUT y DELETE — editar y borrar (CRUD completo)
 - 4.11. Personalizar los colores de Bootstrap
 - 4.12. SASS: instalación y compilación básica
 - 4.13. De Fetch a Angular: por qué dar el salto
 - FASE 5: FRONT END con Angular y Bootstrap
 - Introducción a TypeScript
 - Tipado estático
 - Interfaces
 - Imports y exports
 - 5.1. Instalar Angular
 - 5.2. Crear proyecto FrontEnd
 - 5.3. El MVC para Alumnos y Asignaturas
 - 5.3.1 Modelos en Angular
 - 5.3.2 Servicios en Angular
 - 5.3.3 Vista de Consulta en Angular
 - 5.4 CORS
 - 5.5 Resto de Endpoints en Angular
 - 5.5.1. Ampliación del Servicio
 - 5.5.2. Ampliación de la vista
 - FASE 6: AMPLIAR PROYECTO: Asignaturas

- 6.1: Backend para asignaturas
- 6.2: Frontend para asignaturas
- FASE 7: SPA completa – las pantallas de los mockups
 - 7.1. Backend: migración de campos y endpoints de estadística
 - 7.2. Angular: modelos y servicios actualizados
 - 7.3. El layout general: topbar + sidebar
 - 7.4. Pantalla Dashboard
 - 7.5. Pantalla Alumnos
 - 7.6. Pantalla Asignaturas
 - 7.7. Pantalla Matrículas
 - 7.8. Prueba final
- FASE 8: El Documento de Análisis (minicurso 25h)
 - Introducción al minicurso
 - El Documento de Análisis: estructura final
 - BLOQUE 1 – ¿Qué es un Documento de Análisis?
 - BLOQUE 2 – Definición del proyecto y requisitos
 - BLOQUE 3 – Modelo de datos y API REST
 - BLOQUE 4 – Diseño del Frontend
 - BLOQUE 5 – Gestión del proyecto y trabajo en equipo
 - Sesión final – Defensa del Documento de Análisis
 - Plantilla del Documento de Análisis
- Anexo A: Los Ciclos de Vida en un trámite administrativo
 - A.1. ¿Qué es un ciclo de vida?
 - A.2. El trámite de la UPO paso a paso
 - A.3. Representación 1: la máquina de estados
 - A.4. Representación 2: el diagrama de secuencia (actores ↔ sistema)
 - A.5. Representación 3: el flujo con bifurcaciones (subsanción)
 - A.6. Representación 4: los mockups de las pantallas
 - A.7. Lo que los alumnos deben llevarse
 - Citas Coloreadas

Introducción

Tabla de contenidos

En estos apuntes vamos a ver un ejemplo completo y práctico en fases de un CRUD completo con SpringBoot. Vamos a usar las siguientes tecnologías:

- Stack
- Java25 (Temurin)
 - SpringBoot 3.5.11 con Maven
 - Spring Data JPA (con Hibernate como proveedor JPA por defecto)
 - Lombok (biblioteca que reduce boilerplate -código repetitivo- en Java; generando: setters/getters, constructores, etc)
 - Angular 17 (la actual es el 21)
 - Bootstrap 5.3.8
 - Swagger 2.8.16
 - Visual Studio Code (IDE)
 - MYSQL8.4 (SGBD)

- [Documentación oficial Spring Boot 3.5](#)
- [Documentación oficial Angular 21](#)

Diagrama Entidad-Relación del modelo de datos (inicial)

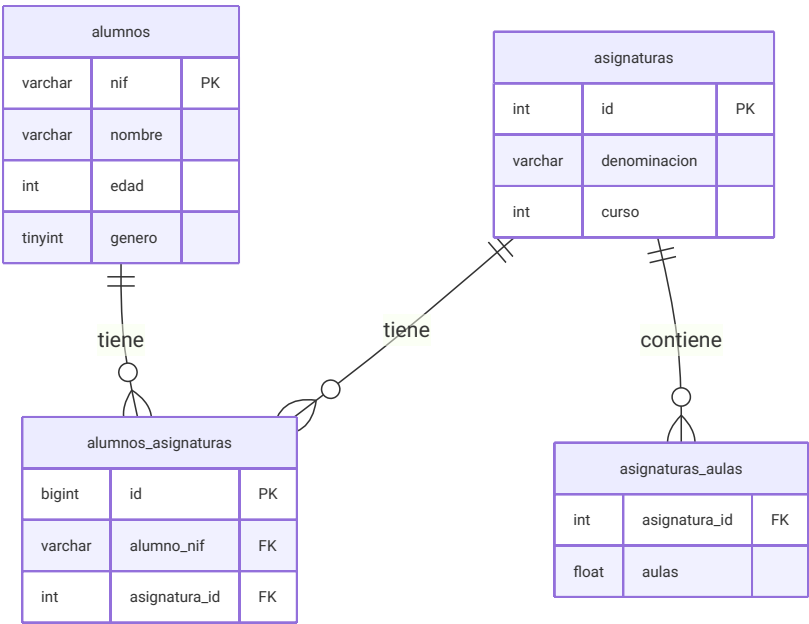
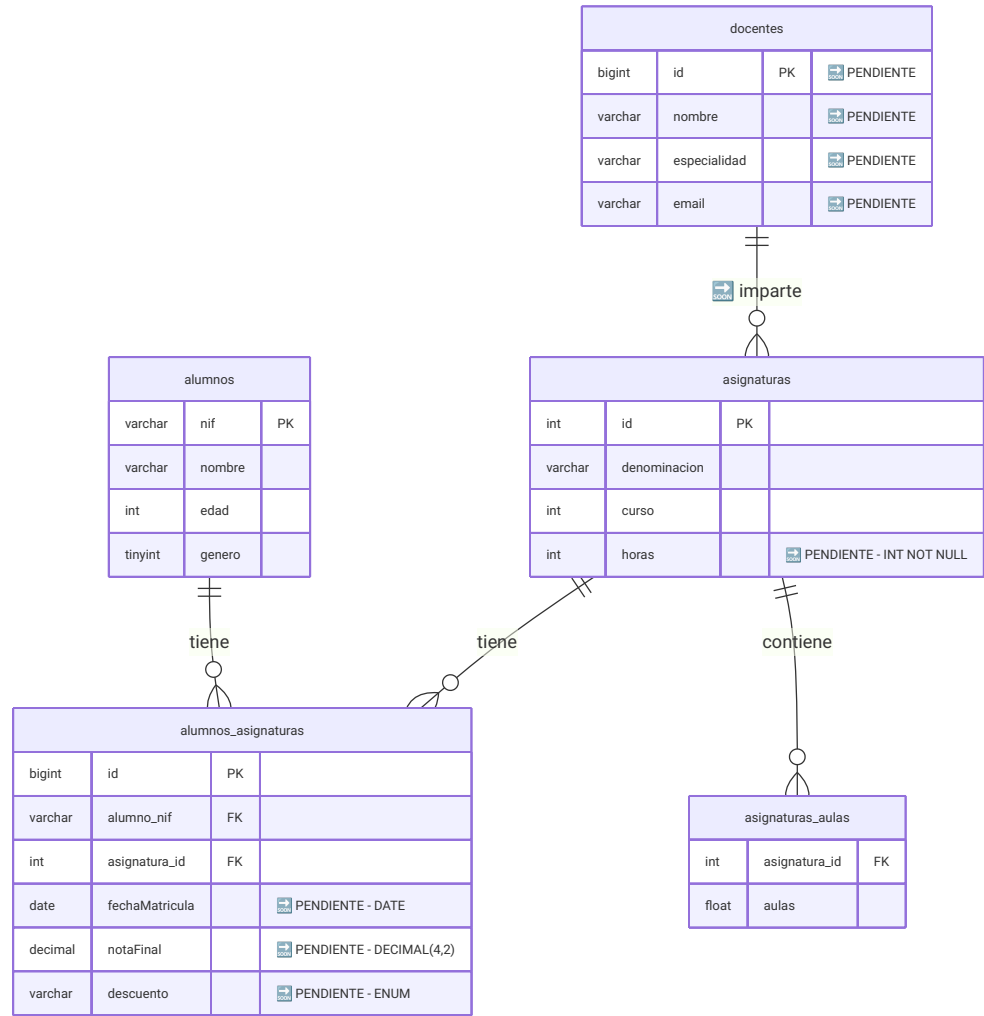


Diagrama Entidad-Relación del modelo de datos (terminado)



Academia Arelance · Gestión de Alumnos

IR

PRINCIPAL

Dashboard

Alumnos

Asignaturas

Matrículas

ANALÍTICA

Estadísticas

Carga CSV

CONFIG

Cursos

Aulas

Dashboard

Resumen académico · curso 2025-2026

+ Nueva matrícula

Alumnos activos

124

+8 este mes

Asignaturas

18

6 cursos distintos

Matrículas activas

312

media 2.5 / alumno

Nota media global

7.34

sobre 10 · sem. actual

Matrículas recientes

ALUMNO	ASIGNATURA	FECHA MAT.	NOTA
García López, Ana 12345678A	Spring Boot	15/09/2025	8.50
Martínez Ruiz, Pedro 87654321B	Angular Avanzado	15/09/2025	6.75
Sánchez Pérez, Laura 11223344C	Bases de Datos	01/10/2025	4.20
López Fernández, Iván 99887766D	Java Fundamentos	01/10/2025	9.10
Romero Gil, Carmen 55443322E	Bootstrap 5	10/10/2025	5.90

Mostrando 1 – 5 de 312

Alumnos por asignatura

Spring Boot 34

Angular 26

Java Fund. 22

Bases Datos 17

Bootstrap 11

Notas medias / asig.

GET /graficas/notas-por-asignatura

matplotlib PNG → base64

Alertas recientes

5 alumnos sin nota en "Spring Boot"

Matrículas con fechaMatricula > 30 días y notaFinal = null

Asignatura "React" sin alumnos

Ninguna matrícula activa este curso

Academia Arelance · Gestión de Alumnos

IR

PRINCIPAL

Dashboard

Alumnos

Asignaturas

Matrículas

ANALÍTICA

Estadísticas

Carga CSV

CONFIG

Cursos

Aulas

Alumnos

124 alumnos registrados

+ Nuevo alumno

Buscar por nombre o NIF...

Género

Ordenar por

Todos (124)

Hombres (67)

Mujeres (57)

NIF	NOMBRE	EDAD	GÉNERO	MATRÍCULAS	NOTA MEDIA	ACCIONES
12345678A	García López, Ana	22	♀	3	8.20	
87654321B	Martínez Ruiz, Pedro	25	♂	2	6.50	
11223344C	Sánchez Pérez, Laura	21	♀	4	4.80	
99887766D	López Fernández, Iván	28	♂	3	9.10	
55443322E	Romero Gil, Carmen	23	♀	2	5.90	
66554433F	Torres Vega, Miguel	19	♂	5	7.75	
77665544G	Navarro Díaz, Sofia	24	♀	3	8.90	

Mostrando 1 – 7 de 124

Academia Arelance · Gestión de Alumnos

IR

PRINCIPAL

Dashboard

Alumnos

Asignaturas

Matrículas

ANALÍTICA

Estadísticas

Carga CSV

CONFIG

Cursos

Aulas

Asignaturas

18 asignaturas · 3 cursos

Buscar asignatura...

Curso

Aula

Todos (18)

Primero (7)

Segundo (8)

Especialización (3)

ID	DENOMINACIÓN	CURSO	AULAS	ALUMNOS	NOTA MEDIA	ACCIONES
1	Java Fundamentos	Primero	P1-A1 P1-A2	22	7.80	
2	Bases de Datos	Primero	P1-A1 P1-A4 P2-A2	17	6.40	
3	Spring Boot	Segundo	P2-A1 P2-A2	34	8.10	
4	Angular Avanzado	Segundo	P1-A3 P2-A3	26	7.60	
5	Bootstrap 5	Segundo	P1-A2 P1-A4	11	6.90	
6	React	Especialización	P2-A3	0 alum.	—	

Mostrando 1–6 de 18

+ Nueva asignatura

Academia Arelance · Gestión de Alumnos

IR

PRINCIPAL

Dashboard

Alumnos

Asignaturas

Matrículas

ANALÍTICA

Estadísticas

Carga CSV

CONFIG

Cursos

Aulas

Matrículas

312 matrículas activas · curso 2025-2026

Buscar por nombre o NIF...

García López, Ana

12345678A · 22 años

Asignatura

Spring Boot

Fecha de matrícula

15/09/2025

Nota final (opcional)

ej: 7.50

Spring Boot · Segundo curso

Aulas: Planta 2-A1 Planta 2-A2

34 alumnos matriculados · nota media 8.10

Guardar matrícula

+ Nueva matrícula

Historial · García López, Ana

3 matrículas · nota media 8.20

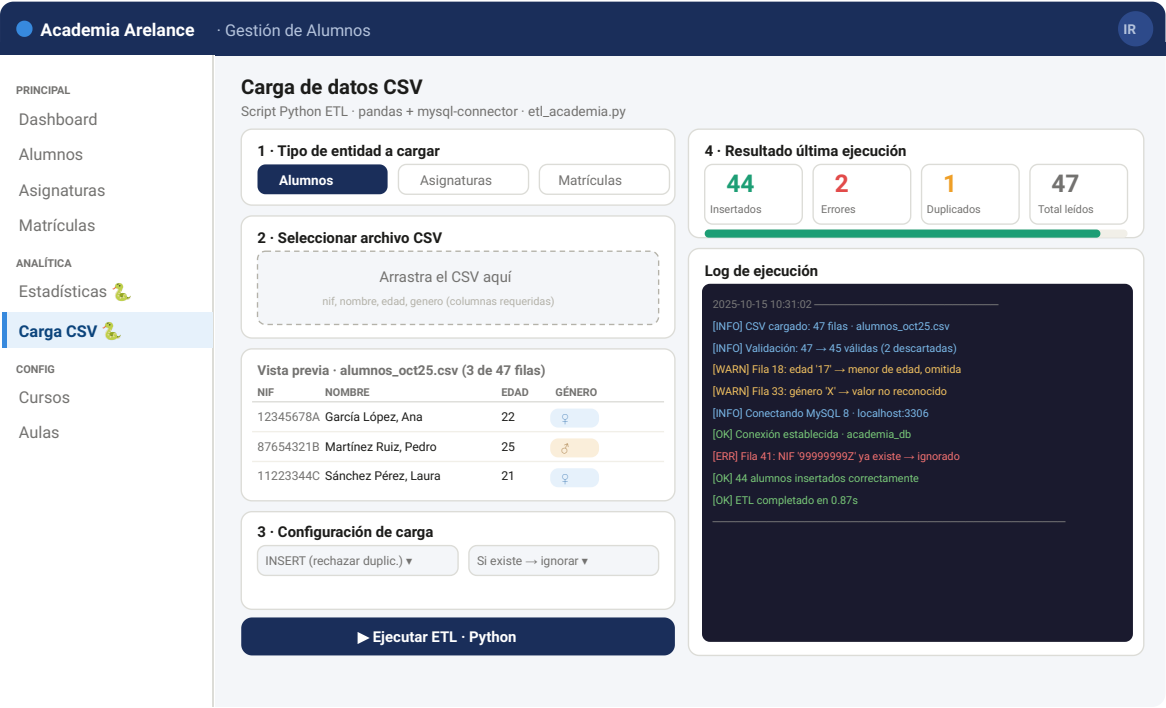
ASIGNATURA	FECHA MAT.	NOTA	ACCIÓN
Java Fundamentos	01/09/2025	8.00	
Spring Boot	15/09/2025	8.50	
Angular Avanzado	15/09/2025	8.10	

RESUMEN DEL ALUMNO

Asignaturas cursadas: 3

Nota media: 8.20

Mejor nota: 8.50 (Spring Boot)



Academia Arelance · Gestión de Alumnos

IR

PRINCIPAL

Dashboard

Alumnos

Asignaturas

Matrículas

ANALÍTICA

Estadísticas

Carga CSV

CONFIG

Cursos

Aulas

Configuración

Tablas de traducción · los valores se muestran así en la UI de Angular

Cursos

Aulas

Cursos

El campo curso en Asignatura almacena 1, 2 o 3.
Aquí defines cómo se muestra en la vista Angular.

CLAVE	TEXTO EN LA VISTA	ACCIÓN
1	Primero	Guardado ✓
2	Segundo	Guardado ✓
3	Especialización	Guardado ✓

¿Cómo funciona en Angular?

El servicio CursoService llama a
GET /config/cursos y devuelve este mapa.

En el pipe de Angular:

{{ asignatura.curso | cursoLabel }}

→ muestra "Segundo" en lugar de 2

Aulas

El campo aulas en Asignatura es List<Float>: [1.1, 1.4, 2.2]
Aquí defines el texto legible para cada valor float.

CLAVE (Float)	TEXTO EN LA VISTA	ACCIÓN
PLANTA 1		
1.1	Planta 1, Aula 1	
1.2	Planta 1, Aula 2	
1.3	Planta 1, Aula 3	
1.4	Planta 1, Aula 4	
PLANTA 2		
2.1	Planta 2, Aula 1	
2.2	Planta 2, Aula 2	
2.3	Planta 2, Aula 3	

Instalación

Tabla de contenidos

 La carpeta principal del proyecto será SpringBoot_Angular. Daremos por sentado que está en ~, es decir el home o carpeta personal del usuario. Si se está en Windows, podemos dejarla en Documentos...

 Lo primero que necesitamos es instalar la infraestructura necesaria para el proyecto:

- Visual Studio Code
 - <https://code.visualstudio.com/download>
 -  Extensiones:
 - Extension Pack for Java (Microsoft)
 - Language Support for Java™ (Red Hat)
 - Debugger for Java (Microsoft)
 - Spring Boot Extension Pack (VMWare)
 - Spring Boot Dashboard (Microsoft)
 - Spring Initializr Java Support (Microsoft)
 - Lombok Annotations Support for VS Code (Microsoft)
 - SonarQube for IDE (**lo instalaremos mas adelante**)
 - Lo que añade SonarQube for IDE por encima:
 - Detección de code smells (código que compila pero está mal escrito): variables innecesarias, métodos demasiado largos, complejidad ciclomática excesiva
 - Vulnerabilidades de seguridad (SQL injection, XSS, etc.)
 - Bugs sutiles que el compilador no detecta: null pointer potenciales, recursos no cerrados, condiciones siempre verdaderas
 - Explicación del por qué es un problema y cómo corregirlo
 - Nos creamos una carpeta en el PC (ej: SpringBoot_Angular)

 Es MUY RECOMENDABLE hacer un repositorio GitHub (Nombre SpringBoot)
En nuestro caso irá en la carpeta SpringBoot_Angular

FASE 1 - Crear Proyecto y configurar conexión BBDD

Tabla de contenidos

Paso1. Crear proyecto con Spring Initializr en VS Code

Tabla de contenidos

Nota sobre la instalación de Java en clase

En clase usamos el paquete `.deb` de Oracle JDK 25 porque funciona igual en todas las distribuciones Linux sin configuración adicional.

En tu entorno de trabajo — Eclipse Temurin

Temurin permite actualizaciones automáticas y gestión de múltiples versiones con `update-alternatives` . En cualquier caso consulta con tu Analista!.
Para instalarlo en tu sistema:

```
# Pregunta a Claude o ChatGPT exactamente esto:
"Dime los pasos para instalar Eclipse Temurin Java 25 LTS
en [tu sistema operativo, ej: Ubuntu 24.04]
e indicame cómo alternar entre versiones LTS con update-alternatives"
```

Por si nos hemos olvidado, siempre podemos cambiar la versión de Java para compilar/ejecutar:

```
sudo update-alternatives --config java

Existen 4 opciones para la alternativa java (que provee /usr/bin/java).

Selección  Ruta                                     Prioridad  Estado
-----
* 0          /usr/lib/jvm/jdk-25.0.2-oracle-x64/bin/java      419446784  modo automático
 1          /usr/lib/jvm/java-11-openjdk-amd64/bin/java  1111      modo manual
 2          /usr/lib/jvm/jdk-17-oracle-x64/bin/java      285278208  modo manual
 3          /usr/lib/jvm/jdk-21-oracle-x64/bin/java      352321536  modo manual
 4          /usr/lib/jvm/jdk-25.0.2-oracle-x64/bin/java  419446784  modo manual
```

CREACIÓN DEL PROYECTO

Pasos:

- 1. Abrir Visual Studio Code
- 2. En VSC > Abrimos el Command Palette (CTRL + MAY + P).
- 3. Escribe: Spring Initializr: Create a Maven Project (con escribir Spring Init es suficiente)
- 4. Versión 3.5.15 (evitar Snapshot -versiones inestables- y M1 -milestones o betas-).
- 5. Language: Java
- 6. Seleccionamos las siguientes opciones:

Campo	Valor recomendado
Group	com.inserta
Artifact	crudalumnos
Name	crudalumnos
Description	Proyecto CRUD con Spring Boot
Package name	com.inserta.crudalumnos
Packaging	Jar
Java Version	25

- Dependencias:
 - Spring Web Web, Spring Data JPA SQL, MySQL Driver SQL, Lombok Developer Tools y Spring Boot DevTools
 - (En proyectos sucesivos) Añadir SpringDoc OpenAPI Web (que es Swagger)

 **Warning**
Seleccionar la carpeta `SpringBoot_Angular` pero **no abrirla**.
Hacer clic en `Generate into this folder` . Se creará una carpeta llamada `crudalumnos` .

VSCode preguntará si añadir el proyecto al espacio de trabajo.
Clic en `Add to Workspace` .

Aparece la estructura del proyecto creado en Maven:

```
crudalumnos/  
├─ .mvn/  
├─ .vscode/  
├─ src/  
├─ target/  
├─ .gitattributes  
├─ .gitignore  
├─ HELP.md  
├─ mvnw  
├─ mvnw.cmd  
└─ pom.xml
```

**Tip**

Si olvidaste alguna dependencia al crear el proyecto, puedes añadirla manualmente en `crudalumnos/pom.xml` en cualquier momento.

- En `crudalumnos/pom.xml`

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.5.13</version>
    <relativePath/> <!-- lookup parent from repository -->
  </parent>
  <groupId>com.inserta</groupId>
  <artifactId>crudalumnos</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>crudalumnos</name>
  <!-- ----- -->

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-devtools</artifactId>
      <scope>runtime</scope>
      <optional>true</optional>
    </dependency>
    <dependency>
      <groupId>com.mysql</groupId>
      <artifactId>mysql-connector-j</artifactId>
      <scope>runtime</scope>
    </dependency>
    <dependency>
      <groupId>org.projectlombok</groupId>
      <artifactId>lombok</artifactId>
      <optional>true</optional>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <!-- ----- -->
</project>

```




Paso2. Configurar aplicación y conectar con BBDD

Tabla de contenidos

- 2.1 Nos creamos la BBDD inserta_matriculas

```
CREATE DATABASE inserta_matriculas;
```

- 2.2 En src/main/resources/application.properties

```
# Conexión a MySQL
# OJO! Cuidado con poner espacios después de los valores!
# Por ejemplo: username=root NO LLEVA ESPACIO DESPUÉS
spring.application.name=crudalumnos
spring.datasource.url=jdbc:mysql://localhost:3306/inserta_matriculas?serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=root

# JPA / Hibernate
# ddl-auto opciones:
#   create → borra y recrea las tablas en cada arranque
#   update → crea si no existen, altera si hay cambios, NUNCA borra datos
#   validate → solo verifica que el esquema coincide (recomendado en producción)
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```



Tip

Añadiendo `createDatabaseIfNotExist=true` a la URL de conexión, Spring Boot crea la base de datos automáticamente si no existe.

Así el único requisito es tener MySQL en marcha — no hace falta entrar en la consola MySQL para crearla manualmente. Yo, personalmente, no lo recomiendo. Es una cadena que puede llegar a ser peligrosa.



Important

Antes de empezar a escribir código, ejecutar en la terminal del proyecto:

```
cd ~/SpringBoot_Angular/crudalumnos # la carpeta del proyecto
./mvnw clean install
```

Esto descarga todas las dependencias del `pom.xml` (Lombok, Spring Boot, etc.)

y evita que VSCode muestre falsos errores en las anotaciones `@Data`, `@NoArgsConstructor`, `@AllArgsConstructor`, etc.

Es muy probable que haya que reiniciar VSC

FASE 2 - Entidades, repositorios y datos iniciales

[Tabla de contenidos](#)

Paso 3. Crear entidades (modelo)

[Tabla de contenidos](#)

Las entidades nos van a servir para mapear la aplicación con la conexión de la Base de datos. En nuestro caso vamos a crearnos, por ahora, 3 entidades:

- Alumno → Tabla alumnos
- Asignatura → Tabla asignaturas
- AlumnoAsignatura → Tabla intermedia alumnos_asignaturas

Caution

Aunque SpringBoot permite el uso de ManyToMany, es mucho más aconsejable definir por nuestra cuenta la entidad intermedia. Da muchos problemas en la serialización (salida JSON) y con Swagger.

- 3.1 Creamos la entidad principal Alumno

Important

Crear la carpeta modelo en `src/main/java/com/inserta/crudalumnos`. Allí pondremos todas las entidades de la aplicación. Otra cosa: Saldrán errores en `Alumno.java` por una dependencia circular (esta entidad depende de `AlumnoAsignatura`). Es normal y esperado. Veremos errores en `@Data` y `@AllArgsConstructor`

- En `src/main/java/com/inserta/crudalumnos/modelo/Alumno.java`

```

package com.inserta.crudalumnos.modelo;

// Las importaciones ya saldrán mientras escribimos código
// IMPORTANTE!! Hay que añadir el @id (clave principal)
// para que no salgan como errores AllArgsConstructor y NoArgsConstructor

// Debemos evitar el .* -> Importar lo justo y necesario
import java.util.ArrayList;
import java.util.List;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.OneToOne;
import jakarta.persistence.Table;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Entity          // -> Esto es una clase para definir una entidad
@Data           // -> Lombok (Setter, Getter, hash...)
@NoArgsConstructor // -> Genera Constructor por defecto
@AllArgsConstructor // -> Genera constructor completo (todos los params)
@Table(name="alumnos") // -> Nombre Tabla (por debajo usar Hibernate)
public class Alumno {
    @Id           // -> Clave Principal (PK de la tabla)
    @Column(length = 9, columnDefinition = "CHAR(9)")
    private String nif; // SQL -> nif CHAR(9) NOT NULL PRIMARY KEY

    // Resto de los campos (siempre PRIVADOS!)
    @Column(length = 50, nullable = false)
    private String nombre; // SQL -> nombre VARCHAR(50) NOT NULL

    // Aquí por ejemplo NO le ponemos ninguna etiqueta (@Column)
    private Integer edad; // SQL -> edad INT NULL

    @Column(columnDefinition = "TINYINT(1)")
    // Se recomienda usar Boolean en vez de boolean. La clase envoltura es mas flexible
    private Boolean genero; // SQL -> genero BOOLEAN

    // Ahora la relación Uno a Muchos con la otra tabla
    // OJO: aquí NO va @JsonIgnore. Queremos que el JSON del alumno
    // incluya sus matriculas (las usará el accordion de la FASE 5).
    // El bucle infinito lo corta el @JsonIgnore de AlumnoAsignatura.alumno
    @OneToMany(mappedBy = "alumno")
    private List<AlumnoAsignatura> alumnoAsignaturas = new ArrayList<>();
    // IMPORTANTE: <AlumnoAsignatura> dará error hasta no crear AlumnoAsignatura.java
}

```

- 3.2 Creamos la entidad principal Asignatura, clase Asignatura.java:
 - En src/main/java/com/inserta/crudalumnos/modelo/Asignatura.java

```

package com.inserta.crudalumnos.modelo;

import java.util.ArrayList;
import java.util.List;

import com.fasterxml.jackson.annotation.JsonIgnore;

import jakarta.persistence.CollectionTable;
import jakarta.persistence.Column;
import jakarta.persistence.ElementCollection;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.OneToMany;
import jakarta.persistence.Table;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

/* strategy = GenerationType.IDENTITY
 * Define la forma de poner las ids, autoincrementales
 */
@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
@Table(name = "asignaturas")
public class Asignatura {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;          // SQL -> id INT AUTO_INCREMENT

    @Column(length = 50, nullable = false)
    private String denominacion;

    @Column(columnDefinition = "TINYINT", nullable = true)
    // Un int primitivo no puede ser NULL
    private Integer curso;       // SQL -> curso TINYINT NULL

    // Dentro de la clase Asignatura, debajo de los campos existentes
    // Colección de valores embebidos (tabla secundaria automática)
    @ElementCollection
    @CollectionTable(name = "asignaturas_aulas",
        joinColumns = @JoinColumn(name = "asignatura_id"))
    private List<Float> aulas = new ArrayList<>();

    @OneToMany(mappedBy = "asignatura")
    @JsonIgnore // Esto rompe el bucle en la serialización
    private List<AlumnoAsignatura> alumnoAsignaturas = new ArrayList<>();
}

```

- 3.3 Creamos la entidad derivada AlumnoAsignatura, clase AlumnoAsignatura.java:
 - En src/main/java/com/inserta/crudalumnos/modelo/AlumnoAsignatura.java

```

package com.inserta.crudalumnos.modelo;

import com.fasterxml.jackson.annotation.JsonIgnore;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;
import jakarta.persistence.UniqueConstraint;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
@Table(name="alumnos_asignaturas",
      uniqueConstraints = @UniqueConstraint(
          columnNames = {"alumno_nif", "asignatura_id"}
      )
)
public class AlumnoAsignatura {
    /**
     * Usaremos una ID autogenerada en vez de compuesta
     * Si vamos a añadir mas adelante (será el caso)
     * campos adicionales (como un array de notas)
     */
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    /**
     * @JsonIgnore sirve para parar la búsqueda en la serialización
     * Sin @JsonIgnore: Alumno → AlumnoAsignatura → Alumno → ∞
     */
    @ManyToOne
    @JoinColumn(name = "alumno_nif")
    @JsonIgnore
    private Alumno alumno;

    @ManyToOne
    @JoinColumn(name = "asignatura_id")
    private Asignatura asignatura;
}

```

Paso 4. Ejecutar la aplicación y crear las tablas

Tabla de contenidos

```

# Antes de nada hay que fijar el JAVA_HOME de Maven
export JAVA_HOME=/usr/lib/jvm/jdk-25.0.2-oracle-x64
echo $JAVA_HOME
# Entramos en la carpeta del proyecto
# NOTA: Hay que poner la carpeta del proyecto!
cd ~/SpringBoot_Angular/crudalumnos
./mvnw clean spring-boot:run
# Y si me equivoco con algo? CTRL + C!

```



Tip

Comprueba en MySQL que Hibernate ha creado las tablas automáticamente:

Deberían aparecer `alumnos`, `asignaturas`, `alumnos_asignaturas` y `asignaturas_aulas`. Esta última la genera `@ElementCollection` sin necesidad de crear una entidad propia para las aulas.

```
mysql -u root -p # root
USE inserta_matriculas;
SHOW TABLES;
```



Paso 5. Creamos los repositorios para todas las entidades.

Tabla de contenidos

- En primer lugar creamos una carpeta con esta ruta:
 - `src/main/java/com/inserta/crudalumnos/repositorio/`
- 5.1. Dentro creamos la **INTERFAZ** `AlumnoRepository`
 - En `src/main/java/com/inserta/crudalumnos/repositorio/AlumnoRepository.java`

```
package com.inserta.crudalumnos.repositorio;

// Importamos la clase padre de JPA para repositorios
import org.springframework.data.jpa.repository.JpaRepository;
import com.inserta.crudalumnos.modelo.Alumno;
/*
 * <Alumno, String>?
 * Alumno es la entidad; String es el tipo para su clave principal (nif)
 */
public interface AlumnoRepository extends JpaRepository<Alumno, String>{

}
```

- 5.2. Repetimos el proceso con la interfaz `AsignaturaRepository`
 - En `src/main/java/com/inserta/crudalumnos/repositorio/AsignaturaRepository.java`:

```
package com.inserta.crudalumnos.repositorio;
import org.springframework.data.jpa.repository.JpaRepository;
import com.inserta.crudalumnos.modelo.Asignatura;
/*
 * Usamos <Asignatura, Integer> porque la
 * clave primaria de Asignatura es id.
 */
public interface AsignaturaRepository extends JpaRepository<Asignatura, Integer>{

}
```

- 5.3. Y por último la interfaz `AlumnoAsignaturaRepository`
 - En `src/main/java/com/inserta/crudalumnos/repositorio/AlumnoAsignaturaRepository.java`

```
package com.inserta.crudalumnos.repositorio;

import com.inserta.crudalumnos.modelo.AlumnoAsignatura;
import org.springframework.data.jpa.repository.JpaRepository;
/*
 * Usamos <AlumnoAsignatura, Long> porque la
 * clave primaria de AlumnoAsignatura es id de tipo Long (BIGINT)
 */
public interface AlumnoAsignaturaRepository extends JpaRepository<AlumnoAsignatura, Long>{

}
```

■ Paso 6. Ahora vamos a introducir unos datos iniciales de prueba en la BBDD.

[Tabla de contenidos](#)

⚠ En producción no es recomendable introducir datos previos. Lo haremos para pruebas.

Vamos a crear un componente de inicialización. Es limpio, flexible y no ensucia ni las entidades ni los controladores. Usaremos `@Component` con `CommandLineRunner`.

- Spring Boot ejecuta cualquier clase que implemente `CommandLineRunner` al iniciar la aplicación.
- En primer lugar creamos una carpeta (package) con esta ruta:
 - `src/main/java/com/inserta/crudalumnos/carga/`
 - Dentro crearemos el archivo `DatosIniciales.java`
 - En `src/main/java/com/inserta/crudalumnos/carga/DatosIniciales.java`

```

package com.inserta.crudalumnos.carga;

import java.util.ArrayList;
import java.util.List;

import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

import com.inserta.crudalumnos.modelo.Alumno;
import com.inserta.crudalumnos.modelo.AlumnoAsignatura;
import com.inserta.crudalumnos.modelo.Asignatura;
import com.inserta.crudalumnos.repositorio.AlumnoAsignaturaRepository;
import com.inserta.crudalumnos.repositorio.AlumnoRepository;
import com.inserta.crudalumnos.repositorio.AsignaturaRepository;

@Component
public class DatosIniciales implements CommandLineRunner{

    // 1. Añadimos los repositorios!
    private final AlumnoRepository alumnoRepo;
    private final AsignaturaRepository asignaturaRepo;
    private final AlumnoAsignaturaRepository alumnoAsignaturaRepo;

    // 2. Creamos el constructor completo
    public DatosIniciales(AlumnoRepository alumnoRepo, AsignaturaRepository asignaturaRepo,
        AlumnoAsignaturaRepository alumnoAsignaturaRepo) {
        this.alumnoRepo = alumnoRepo;
        this.asignaturaRepo = asignaturaRepo;
        this.alumnoAsignaturaRepo = alumnoAsignaturaRepo;
    }

    // 3. Metemos los datos en el run
    @Override
    public void run(String... args) throws Exception {
        // Comprobamos que las tablas están vacías
        if (alumnoRepo.count() == 0 && asignaturaRepo.count() == 0) {

            // 3.1 Crear asignaturas
            Asignatura bbdd = new Asignatura(null, "Bases de datos", 1,
                List.of(1.1f, 2.2f), new ArrayList<>());
            Asignatura prog = new Asignatura(null, "Programación", 1,
                List.of(2.2f, 2.3f), new ArrayList<>());
            Asignatura fol = new Asignatura(null, "Form.Orientacion Lab", 1,
                List.of(2.2f, 2.3f), new ArrayList<>());
            asignaturaRepo.saveAll(List.of(bbdd, prog, fol));

            // 3.2 Crear alumnos
            Alumno parraga = new Alumno("11A", "Alex Párraga", 23, false, new ArrayList<>());
            Alumno rider = new Alumno("22B", "Miguel Rider", 39, false, new ArrayList<>());
            Alumno aleon = new Alumno("33C", "Alex León", 30, false, new ArrayList<>());
            Alumno jaime = new Alumno("44D", "Jaime Peña", 30, false, new ArrayList<>());
            Alumno angela = new Alumno("55E", "Angela Crespo", 20, true, new ArrayList<>());
            alumnoRepo.saveAll(List.of(parraga, rider, aleon, jaime, angela));

            // 3.3 Relacionamos alumnos con asignaturas
            List<AlumnoAsignatura> relaciones = List.of(
                new AlumnoAsignatura(null, jaime, fol),
                new AlumnoAsignatura(null, jaime, bbdd),
                new AlumnoAsignatura(null, jaime, prog),
                new AlumnoAsignatura(null, aleon, prog),
                new AlumnoAsignatura(null, rider, bbdd),
                new AlumnoAsignatura(null, rider, prog),
                new AlumnoAsignatura(null, parraga, fol),
                new AlumnoAsignatura(null, parraga, bbdd),
                new AlumnoAsignatura(null, angela, bbdd),
                new AlumnoAsignatura(null, angela, prog)
            );
            alumnoAsignaturaRepo.saveAll(relaciones);
        }
    }
}

```



```

    );

    alumnoAsignaturaRepo.saveAll(relaciones);
}

}

}

```

Y sin necesidad de borrar ni BBDD ni tablas, limpiamos la compilación y ejecutamos de nuevo (veremos los datos en la BBDD):

```

cd ~/SpringBoot_Angular/crudalumnos
# Paramos la compilación previa: CTRL + C!
./mvnw clean spring-boot:run
# Entramos en MySQL
mysql -u root -p # Clave root

```

- Y vemos en MySQL

```

USE inserta_matriculas;
DESCRIBE alumnos;
SELECT * FROM alumnos;
SELECT * FROM asignaturas;
SELECT * FROM alumnos_asignaturas;
SELECT * FROM asignaturas_aulas;

```

FASE 3: API REST Básico - Instalar Swagger/Postman

[Tabla de contenidos](#)

3.1. Crear Controlador API REST básico para Alumnos

[Tabla de contenidos](#)

- Nos creamos un Java Package en inserta/crudalumnos llamado controlador
- Dentro nos creamos una clase llamada AlumnoController
 - En crudalumnos/src/main/java/com/inserta/crudalumnos/controlador/AlumnoController.java:

```
package com.inserta.crudalumnos.controlador;

import java.util.List;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

// Importamos modelo y repo de Alumno
import com.inserta.crudalumnos.modelo.Alumno;
import com.inserta.crudalumnos.repositorio.AlumnoRepository;
// Para las descripciones de swagger
import io.swagger.v3.oas.annotations.Operation;

/**
 * Añadimos la anotación @RestController para API REST
 * Agregamos la BASE de la ruta de acceso con @RequestMapping
 */
@RestController
@RequestMapping("/api/alumnos")
public class AlumnoController {
    // Agregamos el repo de Alumno al controlador
    private final AlumnoRepository alumnoRepo;

    public AlumnoController (AlumnoRepository repo) {
        this.alumnoRepo = repo;
    }

    /* Sacamos todos los datos de la tabla
     * @Operation(summary) -> Descripción en Swagger
     */
    // 🔍 GET - R01 -> Consultar la tabla entera
    @GetMapping("/consultar")
    public List<Alumno> verTodosAlumnos() {
        return alumnoRepo.findAll();
    }
}
```

 Se prueba con <http://localhost:8080/api/alumnos/consultar>



3.2. Instalación de Postman

Tabla de contenidos

- En la página de Postman tenemos los instaladores
<https://www.postman.com/downloads/>
- Pasos con la consola

```
cd ~/Descargas
wget https://dl.pstmn.io/download/latest/linux64 -O postman.tar.gz
sudo tar -xzf postman.tar.gz -C /opt
sudo mv Postman /usr/share/postman
cd /usr/share/postman
# Ejecutamos una primera vez para comprobar...
./Postman

# Ahora creamos el acceso directo de escritorio
cd /usr/share/applications
sudo nano postman.desktop
```

- Dentro de postman.desktop pondremos:
 - En /usr/share/applications/postman.desktop:

```
[Desktop Entry]
Version=1.0
Type=Application
Name=Postman
GenericName=Postman
Comment=Postman
Exec=/usr/share/postman/Postman
TryExec=/usr/share/postman/Postman
Icon=/usr/share/postman/app/icons/icon_128x128.png
Terminal=false
```

- Es posible que tengamos que actualizar la BBDD de enlaces simbólicos y de escritorio

```
sudo update-desktop-database /usr/share/applications
```

- Ya solo nos queda iniciar Postman y probar un Endpoint:

```
cd ~/SpringBoot_Angular/crudalumnos
./mvnw clean spring-boot:run
```

- Abrimos Postman a **New** -> **HTTP** -> **GET**
- Ponemos el endpoint: <http://localhost:8080/api/alumnos>
- Pulsamos en **Send**

3.3. Instalación de Swagger (Springdoc OpenAPI v3)

Tabla de contenidos

✓ ¿Qué es Swagger?

Swagger (OpenAPI) genera documentación interactiva para los endpoints REST, lo cual es especialmente útil en proyectos profesionales o colaborativos.

- Abrimos el pom.xml que está en la raíz del proyecto
 - En ~/SpringBoot_Angular/crudalumnos/pom.xml
 - Añadimos lo siguiente:

```
<!-- Dependencias anteriores... -->
    <dependency>        <!-- Dependencia6: Swagger -->
        <groupId>org.springdoc</groupId>
        <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
        <version>2.8.9</version>
    </dependency>
</dependencies>
```

- Añadimos lo siguiente (recomendable) en application.properties:
 - En src/main/resources/application.properties

```
# Para Swagger
springdoc.swagger-ui.path=/swagger-ui.html
```

⚠ Antes de seguir, debemos asegurarnos que los test son correctos:

- En ~/SpringBoot_Angular/crudalumnos/src/test/java/com/inserta/crudalumnos/CrudalumnosApplicationTest.java

```
package com.inserta.crudalumnos;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;
@SpringBootTest
class CrudalumnosApplicationTests {
    @Test
    void contextLoads() {
    }
}
```

- Para probar que funciona, iniciamos de nuevo el proyecto. Al estar creadas las tablas y registros, la BBDD no se modifica

```
cd ~/SpringBoot_Angular/crudalumnos
# Paramos la compilación previa: CTRL + C!
./mvnw clean spring-boot:run
```

- Y lo vemos en el navegador:
 - <http://localhost:8080/swagger-ui/index.html>
 - Abrimos GET /api/alumnos -> Try it out -> Execute



3.4. Ampliando API REST

Tabla de contenidos

En este apartado vamos a definir el resto de opciones básicas del API REST: CREATE (insert), READ (select), UPDATE y DELETE.



Tip

Como curiosidad, podemos tener la aplicación iniciada mientras definimos el resto de Endpoints.

- En primer lugar añadimos un sufijo al endpoint genérico /api/alumnos definido al inicio de la clase:
 - En crudalumnos/src/main/java/com/inserta/crudalumnos/controlador/AlumnoController.java:

```

/* Sacamos todos los datos de la tabla
 * @Operation(summary) -> Descripción en Swagger
 * @GetMapping("/consultar") añade un sufijo al endpoint genérico
 * /api/alumnos/consultar
 */
// GET - R01 -> Consultar la tabla entera
@GetMapping("/consultar")
@Operation(summary = "Listar todos los alumnos")
public List<Alumno> verTodosAlumnos(){
    return alumnoRepo.findAll();
}

```

Y probamos:

```

cd ~/SpringBoot_Angular/crudalumnos
# Paramos la compilación previa: CTRL + C!
./mvnw clean spring-boot:run

```

- Y lo vemos en el navegador (hay que darle a F5):
 - <http://localhost:8080/swagger-ui/index.html>
 - Abrimos GET /api/alumnos/consultar -> Try it out -> Execute



Note

findAll(Sort sort) y findAll(Pageable pageable) son las dos sobrecargas que más se usan en una aplicación real: ordenar una tabla y paginarla. findAll(Example < S > example) (búsqueda por ejemplo / Query by Example) es potente pero menos intuitiva, así que la dejamos fuera del temario básico.

Vale, todo correcto.

Seguimos con el resto de opciones básicas del CRUD:

Empecemos con las consultas:

Antes de nada debemos definir varios métodos abstractos en el repositorio:

- En crudalumnos/src/main/java/com/inserta/crudalumnos/repositorio/AlumnoRepository.java:

```

package com.inserta.crudalumnos.repositorio;

import java.util.List;

import org.springframework.data.jpa.repository.JpaRepository;
import com.inserta.crudalumnos.modelo.Alumno;
/**
 * JpaRepository<Alumno,String>
 * Alumno es el modelo y el String es el tipo de la PK (nif)
 */

public interface AlumnoRepository extends JpaRepository<Alumno,String> {
    // GET R07 → /api/alumnos/consultar/nombre
    // Que nombre esté como parte del dato ignorando MAY/MIN
    List<Alumno> findByNombreContainingIgnoreCase(String nombre);

    // Esto sería para buscar EXACTAMENTE un nombre
    // List<Alumno> findByName(String nombre);

    // GET R08 y R09 → /api/alumnos/consultar/genero
    List<Alumno> findByGenero(boolean genero);

    // GET R10 y R11 → /api/alumnos/consultar/contar/
    // countBy<campo> -> Saca el entero(largo) de contar registros de un campo
    long countByGenero(boolean genero);
}

```

- Cada uno de los métodos abstractos anteriores serán usados en su GET correspondiente.
- Vamos a ver ahora TODOS los endpoints para Alumnos:
 - En crudalumnos/src/main/java/com/inserta/crudalumnos/controlador/AlumnoController.java:

```

// En el paquete del controlador
package com.inserta.crudalumnos.controlador;

// Librerías clasificadas
import com.inserta.crudalumnos.modelo.Alumno;
import com.inserta.crudalumnos.repositorio.AlumnoRepository;

import io.swagger.v3.oas.annotations.Operation;
import java.util.List;

import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Sort;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;

import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.PutMapping;

/**
 * Endpoints para Alumnos:
 * * http://localhost:8080/api/alumnos
 * * - GET R01 → /api/alumnos/consultar → Consultar todos los alumnos
 * * - GET R02 → /api/alumnos/consultar/{nif} → Consulta por PK (nif)
 * * - GET R03 → /api/alumnos/consultar/ordenado → Consulta ordenada por campo=edad
 * * - GET R04 → /api/alumnos/consultar/pagina → Consulta paginada (LIMIT 0,5)
 * * - GET R05 → /api/alumnos/existe/{nif} → Consulta booleana (¿Existe alumno?)
 * * - GET R06 → /api/alumnos/contar → Contar todos los alumnos
 * * - GET R07 → /api/alumnos/consultar/nombre → Listar alumnos que contengan "nombre"
 * * - GET R08 → /api/alumnos/consultar/genero/mujeres → Listar solo alumnas
 * * - GET R09 → /api/alumnos/consultar/genero/hombres → Listar solo alumnos
 * * - GET R10 → /api/alumnos/consultar/contar/mujeres → Contar solo alumnas
 * * - GET R11 → /api/alumnos/consultar/contar/hombres → Contar solo alumnos
 *
 * * - POST - C01 → /api/alumnos/crear/{nif}/{nombre}/{edad}/{genero} → Insertar por parámetros
 * * - POST - C02 → /api/alumnos/crear → Insertar por JSON
 * * - POST - C03 → /api/alumnos/crear/lote → insertar por JSON en lote
 *
 * * - PUT - U01 → /api/alumnos/actualizar/{nif} → Actualiza por param/JSON
 * * - PUT - U02 → /api/alumnos/actualizar → Actualiza por JSON
 * * - PUT - U03 → /api/alumnos/actualizar/{nif}/{nombre}/{edad}/{genero}
 *
 * * - DELETE - D01 → /api/alumnos/borrar/{nif}
 * * - DELETE - D02 → /api/alumnos/borrar/sencillo/{nif}
 * * - DELETE - D03 → /api/alumnos/borrar
 * * - DELETE - D04 → /api/alumnos/borrar/todo
 */

// Para la API/Rest (en el controlador) se pone la anotación @RestController
// @RequestMapping define el endpoint (la URL que sale en swagger/postman)
@RestController
@RequestMapping("/api/alumnos")
public class AlumnoController {
    // Agregamos el repo
    private final AlumnoRepository alumnoRepo;

    // Constructor del repo
    public AlumnoController(AlumnoRepository alumnoRepo) {

```

```

        this.alumnoRepo = alumnoRepo;
    }

    // Por cada endpoint se crea un método
    // Ej: http://localhost:8080/api/alumnos/consultar
    // GET R01 → /api/alumnos/consultar
    @GetMapping("/consultar")
    @Operation(summary = "Lista alumnos")
    public List<Alumno> verTodosAlumnos(){
        return alumnoRepo.findAll();
    }

    // Añadimos Swagger CTRL + MAY + P
    // Spring Initializr: Add Starter > SpringDoc Open Api Web
    // Se añade el paquete [Proceed] y se añade al Path
    // En application.properties se añade la url.path de swagger
    // http://localhost:8080/swagger-ui/index.html

    // GET por parámetro PK (findById) -> Parámetros van con { }
    // Ej: http://localhost:8080/api/alumnos/consultar/22B
    // GET R02 → /api/alumnos/consultar/{nif}
    @GetMapping("/consultar/{nif}")
    @Operation(summary = "Ver alumno por NIF")
    public ResponseEntity<Alumno> verAlumnoPorNIF(@PathVariable String nif){
        return alumnoRepo.findById(nif)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    // Ej: http://localhost:8080/api/alumnos/consultar/ordenado?campo=edad
    // GET R03 → /api/alumnos/consultar/ordenado
    @GetMapping("/consultar/ordenado")
    @Operation(summary = "Consultar alumnos ordenado por nombre descendente")
    public List<Alumno> verAlumnosOrdenados
        (@RequestParam (defaultValue = "nombre") String campo) {
        return alumnoRepo.findAll(Sort.by(campo).descending());
    }

    // Ej: http://localhost:8080/api/alumnos/consultar/pagina?pagina=0&tam=5
    // GET R04 → /api/alumnos/consultar/pagina
    @GetMapping("/consultar/pagina")
    @Operation(summary = "Consultar alumnos paginado por intervalos")
    public Page<Alumno> verAlumnosPaginados
        (@RequestParam (defaultValue = "0") int pagina,
        @RequestParam (defaultValue = "5") int tam) {
        return alumnoRepo.findAll(PageRequest.of(pagina, tam));
    }

    // Ej: http://localhost:8080/api/alumnos/existe/22C
    // GET R05 → /api/alumnos/existe/{nif}
    @GetMapping("/existe/{nif}")
    @Operation(summary = "¿Existe un alumno por su NIF?")
    public boolean existeAlumno (@PathVariable String nif) {
        return alumnoRepo.existsById(nif);
    }

    // Ej: http://localhost:8080/api/alumnos/contar
    // GET R06 → /api/alumnos/contar
    @GetMapping("/contar")
    public long contarTodosAlumnos() {
        return alumnoRepo.count();
    }

    // Ej: http://localhost:8080/api/alumnos/consultar/nombre?nombre=alex
    // GET R07 → /api/alumnos/consultar/nombre
    // Paso1: Al no ser método estandar, se añade al repo
    // Paso2: Se agrega el nuevo método del repo aquí en el controlador

```



```

@GetMapping("/consultar/nombre")
@Operation(summary = "Buscar alumnos que contengan parte de 'nombre'")
public List<Alumno> buscarPorNombre(@RequestParam String nombre) {
    return alumnoRepo.findByNombreContainingIgnoreCase(nombre);
}

// GET R08 → /api/alumnos/consultar/genero/mujeres
@GetMapping("/consultar/genero/mujeres")
@Operation(summary = "Sacar Listado Alumnas")
public List<Alumno> verAlumnas() {
    return alumnoRepo.findByGenero(true);
}

// GET R09 → /api/alumnos/consultar/genero/hombres
@GetMapping("/consultar/genero/hombres")
@Operation(summary = "Sacar Listado Alumnos")
public List<Alumno> verAlumnos() {
    return alumnoRepo.findByGenero(false);
}

// GET R10 → /api/alumnos/consultar/contar/mujeres
@GetMapping("/consultar/contar/mujeres")
@Operation(summary = "Contar Alumnas")
public long contarAlumnas() {
    return alumnoRepo.countByGenero(true);
}

// GET R11 → /api/alumnos/consultar/contar/hombres
@GetMapping("/consultar/contar/hombres")
@Operation(summary = "Contar Alumnos")
public long contarAlumnos() {
    return alumnoRepo.countByGenero(false);
}

// Ej: http://localhost:8080/api/alumnos/crear/88E/Carlos%20Hidalgo/34/false
// POST - C01 → /api/alumnos/crear/{nif}/{nombre}/{edad}/{genero}
// El orden de los parámetros DA IGUAL, pero los nombres deben ser iguales
// Consejo: En los PathVariable usar siempre OBJETOS (para llegadas nulas)
// En el SpringBoot en el formulario para rellenar datos sale el orden del @Path
@PostMapping("/crear/{nif}/{nombre}/{edad}/{genero}")
@Operation(summary = "Inserta Alumno por parámetros en la URL")
public Alumno crearAlumnoParametros(
    @PathVariable String nif,
    @PathVariable Boolean genero,
    @PathVariable String nombre,
    @PathVariable Integer edad
) {
    Alumno alumno = new Alumno(); // Objeto vacío
    alumno.setNombre(nombre);
    alumno.setEdad(edad);
    alumno.setGenero(genero);
    alumno.setNif(nif);

    return alumnoRepo.save(alumno);
}

// Ej: http://localhost:8080/api/alumnos/crear
// POST - C02 → /api/alumnos/crear
@PostMapping("/crear")
@Operation(summary = "Inserta alumno por JSON")
public Alumno crearAlumnoJSON(@RequestBody Alumno alumno) {
    return alumnoRepo.save(alumno);
}

// Ej: http://localhost:8080/api/alumnos/crear/lote
// POST - C03 → /api/alumnos/crear/lote
@PostMapping("/crear/lote")

```

```

@Operation(summary = "Inserta varios alumnos en lote por JSON")
public List<Alumno> crearAlumnoJSON(@RequestBody List<Alumno> alumnos) {
    return alumnoRepo.saveAll(alumnos);
}

// Ej: Nif: 33C → { "edad": 30 }
// PUT - U01 → /api/alumnos/actualizar/{nif}
@PutMapping("/actualizar/{nif}")
@Operation(summary = "Actualizar un alumno por NIF en la URL")
public Alumno actualizarAlumnoNIF(
    @PathVariable String nif, @RequestBody Alumno alumnoJSON) {
    // Busco el alumno para actualizar
    Alumno alumno = alumnoRepo.findById(nif)
        .orElseThrow(() ->
            new RuntimeException("Alumno no encontrado"));

    if(alumnoJSON.getNombre() != null){
        alumno.setNombre(alumnoJSON.getNombre());}
    if(alumnoJSON.getEdad() != null){
        alumno.setEdad(alumnoJSON.getEdad());}
    if(alumnoJSON.getGenero() != null){
        alumno.setGenero(alumnoJSON.getGenero());}

    return alumnoRepo.save(alumno);
}

// Ej: { "nif": "11A", "nombre": "Alejandro Párraga", "edad": 24 }
// PUT - U02 → /api/alumnos/actualizar
@PutMapping("/actualizar")
@Operation(summary = "Actualizar un alumno por JSON")
public Alumno actualizarAlumnoJSON(@RequestBody Alumno alumnoJSON) {
    // Busco el alumno para actualizarnif
    Alumno alumno = alumnoRepo.findById(alumnoJSON.getNif())
        .orElseThrow(() ->
            new RuntimeException("Alumno no encontrado"));

    if(alumnoJSON.getNombre() != null){
        alumno.setNombre(alumnoJSON.getNombre());}
    if(alumnoJSON.getEdad() != null){
        alumno.setEdad(alumnoJSON.getEdad());}
    if(alumnoJSON.getGenero() != null){
        alumno.setGenero(alumnoJSON.getGenero());}

    return alumnoRepo.save(alumno);
}

// Ej: http://localhost:8080/api/alumnos/actualizar/22B/Miguel%20%C3%81ngel%20Rider/39/false
// PUT - U03 → /api/alumnos/actualizar/{nif}/{nombre}/{edad}/{genero}
// Los parámetros TODOS son obligatorios
@PutMapping("/actualizar/{nif}/{nombre}/{edad}/{genero}")
@Operation(summary = "Actualizar un alumno por NIF en la URL")
public Alumno actualizarAlumno(
    @PathVariable String nif,
    @PathVariable String nombre,
    @PathVariable Integer edad,
    @PathVariable Boolean genero ) {
    // Busco el alumno para actualizar
    Alumno alumno = alumnoRepo.findById(nif)
        .orElseThrow(() ->
            new RuntimeException("Alumno no encontrado"));

    alumno.setNombre(nombre);
    alumno.setEdad(edad);
    alumno.setGenero(genero);

    return alumnoRepo.save(alumno);
}

```

```
// Ej: http://localhost:8080/api/alumnos/borrar/11A
// ESTE es el que usa a nivel profesional!
// DELETE - D01 → /api/alumnos/borrar/{nif}
@DeleteMapping("/borrar/{nif}")
@Operation(summary = "Borrar alumno por NIF (PK)")
public ResponseEntity<String> borrarAlumno(@PathVariable String nif){
    if(!alumnoRepo.existsById(nif)){
        return ResponseEntity.status(HttpStatus.NOT_FOUND).
            body("Alumno/a no encontrado");
    }
    alumnoRepo.deleteById(nif);
    return ResponseEntity.ok("Alumno/a borrado de la BBDD");
}

// Ej: http://localhost:8080/api/alumnos/borrar/sencillo/14234
// DELETE - D02 → /api/alumnos/borrar/sencillo/{nif}
@DeleteMapping("/borrar/sencillo/{nif}")
@Operation(summary = "Borrar alumno por NIF (PK) - Modo Sencillo")
public void borrarAlumnoSencillo (@PathVariable String nif){
    alumnoRepo.deleteById(nif);
}

// Ej: ["22B", "44D"] → Elimino a Rider y a Jaime
// DELETE - D03 → /api/alumnos/borrar
@DeleteMapping("/borrar")
@Operation(summary = "Borrar alumno por JSON (incluido LOTE)")
public void borrarVariosAlumnos(@RequestBody List<String> nifs){
    alumnoRepo.deleteAllById(nifs);
}

// DELETE - D04 → /api/alumnos/borrar/todo
@DeleteMapping("/borrar/todo")
@Operation(summary = "Borrar todos los alumnos")
public void borrarTodosAlumnos(){
    alumnoRepo.deleteAll();
}
}
```

- Para el caso del borrado, debemos empezar por borrar las asignaturas de un alumno.
 - Y luego borramos ese alumno por su nif.
 - Nos creamos tanto el repo como el controlador de AlumnoAsignatura.
- En crudalumnos/src/main/java/com/inserta/crudalumnos/repositorio/AlumnoAsignaturaRepository.java:

```
package com.inserta.crudalumnos.repositorio;

import org.springframework.data.jpa.repository.JpaRepository;
import com.inserta.crudalumnos.modelo.AlumnoAsignatura;
import jakarta.transaction.Transactional;

public interface AlumnoAsignaturaRepository extends JpaRepository<AlumnoAsignatura, Long>{
    // Paso 1a. Definir el método abstracto de contar asignaturas por NIF
    @Transactional
    long countByAlumnoNif(String nif);

    // Paso 2a. Definir el método abstracto de borrado de asignaturas
    // Operaciones con la BBDD que implica tocar una tabla relacionada
    @Transactional
    void deleteByAlumnoNif(String nif);
}
```

- En crudalumnos/src/main/java/com/inserta/crudalumnos/controlador/AlumnoAsignaturaControlador.java:

```

package com.inserta.crudalumnos.controlador;

import org.springframework.http.HttpStatus;
import org.springframework.http.HttpStatusCode;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.inserta.crudalumnos.repositorio.AlumnoAsignaturaRepository;

import io.swagger.v3.oas.annotations.Operation;
import jakarta.transaction.Transactional;

@RestController
@RequestMapping("/api/alumnos-asignaturas")
public class AlumnoAsignaturaController {
    // Agregamos el repo
    private final AlumnoAsignaturaRepository alumnoAsignaturaRepo;

    // Constructor del repo
    public AlumnoAsignaturaController(
        AlumnoAsignaturaRepository alumnoAsignaturaRepo) {
        this.alumnoAsignaturaRepo = alumnoAsignaturaRepo;
    }

    // Paso1. Vamos a borrar todas las asignaturas de un alumno
    // Paso2. Ya borradas, podremos borrar el alumno

    // Paso2b. Usamos el método abstracto y definimos el endpoint
    // Ej: http://localhost:8080/api/alumnos-asignaturas/borrar/{nif}
    @DeleteMapping("/borrar/{nif}")
    @Operation(summary = "Borrar asignaturas a partir del NIF de un alumno")
    @Transactional
    public ResponseEntity<String>
        borrarAsignaturasNIF (@PathVariable String nif){
        // Vemos las asignaturas por alumno
        long totalAsignaturas = alumnoAsignaturaRepo.countByAlumnoNif(nif);

        if(totalAsignaturas == 0){
            return ResponseEntity.status(HttpStatus.NOT_FOUND)
                .body("El alumno/a no tiene asignaturas matriculadas");
        }
        // Si tiene mas de 1 asignatura
        alumnoAsignaturaRepo.deleteByAlumnoNif(nif);
        return ResponseEntity.ok("Asignaturas borradas!");
    }
}

```



3.5. JOINS

[Tabla de contenidos](#)

Consultas con JOIN para los repositorios

Las sobrecargas de `findAll` y los métodos derivados (`findByNombre`, `findByEdadGreaterThan...`) cubren una sola tabla. Para cruzar tablas necesitamos JPQL con `@Query`. Los métodos van en `AlumnoAsignaturaRepository`, que es la entidad puente donde confluyen alumno y asignatura.

- En `crudalumnos/src/main/java/com/inserta/crudalumnos/repositorio/AlumnoAsignaturaRepository.java`:

```

// src/main/java/com/inserta/crudalumnos/repositorio/AlumnoAsignaturaRepository.java
package com.inserta.crudalumnos.repositorio;

import java.util.List;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Modifying;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;
import org.springframework.transaction.annotation.Transactional;

import com.inserta.crudalumnos.modelo.Alumno;
import com.inserta.crudalumnos.modelo.AlumnoAsignatura;

/**
 * Repositorio para gestionar la tabla intermedia de matriculas (Alumno ↔ Asignatura).
 *
 * * Permite:
 * * - Comprobar si un alumno ya está matriculado en una asignatura
 * * - Borrar una matrícula concreta
 * * - Listar todas las matrículas de un alumno
 * * - Listar todas las matrículas de una asignatura
 * * - Contar cuántos alumnos están matriculados en una asignatura
 * * - Listar los alumnos matriculados en una asignatura (por su denominación)
 */
@Repository
public interface AlumnoAsignaturaRepository extends JpaRepository<AlumnoAsignatura, Long> {

    /**
     * Verifica si existe una matrícula de un alumno en una asignatura concreta.
     * "" -> text block de java (desde v15), permite escribir cadenas multilinea
     * aa -> alias de AlumnoAsignatura
     * @param nif NIF del alumno
     * @param idAsignatura ID de la asignatura
     * @return true si ya existe, false si no
     */
    @Query("""
        SELECT CASE WHEN COUNT(aa) > 0
        THEN TRUE
        ELSE FALSE END
        FROM AlumnoAsignatura aa
        WHERE aa.alumno.nif = :nif
        AND aa.asignatura.id = :idAsignatura
        """)
    boolean existeMatricula(@Param("nif") String nif,
        @Param("idAsignatura") Integer idAsignatura);

    /**
     * Borra la matrícula de un alumno en una asignatura.
     *
     * @param nif NIF del alumno
     * @param idAsignatura ID de la asignatura
     */
    @Modifying // Va a OPERAR (no consultar) sobre la BBDD
    @Transactional // Se ejecuta en una transacción
    @Query("""
        DELETE FROM AlumnoAsignatura aa
        WHERE aa.alumno.nif = :nif
        AND aa.asignatura.id = :idAsignatura
        """)
    void borrarMatricula(@Param("nif") String nif,
        @Param("idAsignatura") Integer idAsignatura);

    /**
     * Busca todas las matrículas de un alumno por su NIF.
     *

```

```

    * @param nif NIF del alumno
    * @return lista de matrículas de ese alumno
    */
    @Query("""
        SELECT aa
        FROM AlumnoAsignatura aa
        WHERE aa.alumno.nif = :nif
        ORDER BY aa.asignatura.denominacion ASC
        """)
    List<AlumnoAsignatura> buscarPorAlumno(@Param("nif") String nif);

    /**
     * Busca todas las matrículas de una asignatura por su ID.
     */
    * @param idAsignatura ID de la asignatura
    * @return lista de matrículas en esa asignatura
    */
    @Query("""
        SELECT aa
        FROM AlumnoAsignatura aa
        WHERE aa.asignatura.id = :idAsignatura
        ORDER BY aa.alumno.nif ASC
        """)
    List<AlumnoAsignatura> buscarPorAsignatura(@Param("idAsignatura") Integer idAsignatura);

    /**
     * Cuenta cuántos alumnos están matriculados en una asignatura.
     */
    * @param idAsignatura ID de la asignatura
    * @return número de alumnos matriculados
    */
    @Query("""
        SELECT COUNT(aa)
        FROM AlumnoAsignatura aa
        WHERE aa.asignatura.id = :idAsignatura
        """)
    long contarPorAsignatura(@Param("idAsignatura") Integer idAsignatura);

    /**
     * Lista los alumnos matriculados en una asignatura (por su denominación).
     * JOIN entre AlumnoAsignatura y Asignatura: devolvemos el Alumno asociado.
     */
    * @param denominacion nombre de la asignatura
    * @return lista de alumnos matriculados en esa asignatura
    */
    @Query("""
        SELECT aa.alumno
        FROM AlumnoAsignatura aa
        JOIN aa.asignatura asig
        WHERE asig.denominacion = :denominacion
        """)
    List<Alumno> buscarAlumnosPorAsignatura(@Param("denominacion") String denominacion);
}

```

El método `existeMatricula` define una Query que empleará JPQL (Java Persistence Query Language, de JPA). Ese mismo comando en MySQL sería el siguiente:

```

SELECT CASE WHEN COUNT(*) > 0
THEN TRUE
ELSE FALSE END AS existeMatricula
FROM alumnos_asignaturas
WHERE alumno_nif = '11A'
AND asignatura_id = 1;

```



Important

*En JPQL escribimos aa.asignatura.denominacion, no la columna asignatura_id de SQL.
Hibernate traduce esa navegación entre objetos al JOIN ... ON real contra la base de datos.
Es la diferencia clave entre JPQL (orientado a entidades) y SQL (orientado a tablas).*

- Para probarlo tendremos que hacerlo en el controlador de MatriculaController
 - En crudalumnos/src/main/java/com/inserta/crudalumnos/controlador/MatriculaController.java:


```

// src/main/java/com/inserta/crudalumnos/controlador/MatriculaController.java
package com.inserta.crudalumnos.controlador;

import java.util.List;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import com.inserta.crudalumnos.modelo.Alumno;
import com.inserta.crudalumnos.modelo.AlumnoAsignatura;
import com.inserta.crudalumnos.modelo.Asignatura;
import com.inserta.crudalumnos.repositorio.AlumnoAsignaturaRepository;
import com.inserta.crudalumnos.repositorio.AlumnoRepository;
import com.inserta.crudalumnos.repositorio.AsignaturaRepository;

import io.swagger.v3.oas.annotations.Operation;

/**
 * Controlador REST para gestionar MATRÍCULAS (Alumno ↔ Asignatura).
 *
 * Endpoints:
 * - POST /api/matriculas/matricular/{nif}/{idAsignatura} → Matricular
 * - DELETE /api/matriculas/desmatricular/{nif}/{idAsignatura} → Desmatricular
 * - GET /api/matriculas/matriculados?denominacion=... → Alumnos de una asignatura
 */
@RestController
@RequestMapping("/api/matriculas")
public class MatriculaController {

    private final AlumnoRepository alumnoRepo;
    private final AsignaturaRepository asignaturaRepo;
    private final AlumnoAsignaturaRepository aluAsigRepo;

    public MatriculaController(AlumnoRepository alumnoRepo,
                               AsignaturaRepository asignaturaRepo,
                               AlumnoAsignaturaRepository aluAsigRepo) {
        this.alumnoRepo = alumnoRepo;
        this.asignaturaRepo = asignaturaRepo;
        this.aluAsigRepo = aluAsigRepo;
    }

    // + POST - Matricular un alumno en una asignatura
    @PostMapping("/matricular/{nif}/{idAsignatura}")
    @Operation(summary = "Matricular un alumno en una asignatura")
    public ResponseEntity<?>
        matricular(@PathVariable String nif,
                  @PathVariable Integer idAsignatura) {

        // 1) Validar existencia de Alumno y Asignatura
        final Alumno alumno = alumnoRepo.findById(nif).orElse(null);
        if (alumno == null)
            return ResponseEntity.notFound().build(); // 404 si el alumno no existe

        final Asignatura asig = asignaturaRepo.findById(idAsignatura).orElse(null);
        if (asig == null)
            return ResponseEntity.notFound().build(); // 404 si la asignatura no existe

        // 2) Evitar duplicados (además del unique constraint)
        if (aluAsigRepo.existeMatricula(nif, idAsignatura)) {
            return ResponseEntity.badRequest().body("El alumno ya está matriculado en esa asignatura");
        }

        // 3) Crear relación y devolverla
        AlumnoAsignatura rel = new AlumnoAsignatura(null, alumno, asig);
        return ResponseEntity.ok(aluAsigRepo.save(rel)); // 200 con la matricula creada
    }
}

```

```

// ✖ DELETE - Desmatricular un alumno de una asignatura
@DeleteMapping("/desmatricular/{nif}/{idAsignatura}")
@Operation(summary = "Desmatricular un alumno de una asignatura")
public ResponseEntity<>
    desmatricular(@PathVariable String nif,
                  @PathVariable Integer idAsignatura) {

    // 1) Si no existe la matrícula → 404 (nada que borrar)
    if (!aluAsigRepo.existeMatricula(nif, idAsignatura)) {
        return ResponseEntity.notFound().build();
    }

    // 2) Borrar relación
    aluAsigRepo.borrarMatricula(nif, idAsignatura);
    return ResponseEntity.noContent().build(); // 204 OK sin contenido
}

// 🔍 GET - Alumnos matriculados en una asignatura (por su denominación)
@GetMapping("/matriculados")
@Operation(summary = "Listar los alumnos matriculados en una asignatura")
public List<Alumno> matriculados(@RequestParam String denominacion) {
    // Ej: /api/matriculas/matriculados?denominacion=Programación
    return aluAsigRepo.buscarAlumnosPorAsignatura(denominacion);
}
}

```

🍕 FASE 4: Bootstrap 5 y Fetch API

Tabla de contenidos

En esta fase vamos a construir un frontend completo para nuestra Academia Arelance usando únicamente **Bootstrap 5** y la **Fetch API** nativa del navegador — sin frameworks, sin compiladores, sin instalaciones. Solo HTML, CSS y JavaScript vanilla.

El resultado será un conjunto de páginas en `src/main/resources/static/` del proyecto `crudalumnos` que consumen directamente los endpoints que hemos construido en las fases anteriores.



Tip

Al servir los archivos desde `static/` del propio Spring Boot, todo está en el mismo origen (`localhost:8080`) y no hay problemas de CORS. No necesitamos configurar nada extra en el backend.

La estructura de archivos que crearemos será:

```
crudalumnos/src/main/resources/static/
├─ index.html      ← Menú principal / Dashboard
├─ alumnos.html    ← CRUD completo de alumnos
├─ asignaturas.html ← Listado de asignaturas
├─ formularios.html ← Ejemplos de formularios Bootstrap
├─ componentes.html ← Galería de componentes Bootstrap
└─ grid.html       ← Ejemplos del sistema de rejilla
```

Con Spring Boot en marcha, todo quedará disponible en `http://localhost:8080/`.



4.1. ¿Qué es Bootstrap? CDN, descarga local y NPM

Tabla de contenidos

📖 Referencias oficiales:

- Introduction (estructura base y CDN) → <https://getbootstrap.com/docs/5.3/getting-started/introduction/>
- Download (descarga local y npm) → <https://getbootstrap.com/docs/5.3/getting-started/download/>
- Contents (qué contiene cada archivo `dist`) → <https://getbootstrap.com/docs/5.3/getting-started/contents/>
- List group (el menú del `index.html`) → <https://getbootstrap.com/docs/5.3/components/list-group/>

Bootstrap es el framework CSS más utilizado del mundo. Nos proporciona un sistema de rejilla responsive, componentes visuales listos para usar (tablas, botones, modales, formularios, navbars...) y un conjunto de utilidades para tipografía, colores, espaciado y más — todo sin escribir una línea de CSS propio.

La forma más rápida de incluirlo es mediante **CDN** (*Content Delivery Network*): enlazamos los archivos directamente desde servidores externos, sin instalación ni `npm`.

¿Qué CDN usamos?

Puede que hayas visto referencias a `bootstrapcdn.com` — es un CDN mantenido por la comunidad, que también aloja Font Awesome y Bootswatch, pero **no es el CDN recomendado por Bootstrap**. La propia documentación oficial en `getbootstrap.com` recomienda y usa **jsDelivr** (`cdn.jsdelivr.net`), que es el que usaremos nosotros.

CDN	Mantenido por	Versión más reciente disponible	¿Recomendado?
<code>cdn.jsdelivr.net</code>	jsDelivr (open source)	5.3.8	Sí — oficial en la doc Bootstrap
<code>bootstrapcdn.com</code>	Comunidad externa	5.1.3 (desactualizado)	No recomendado



Warning

Si buscas tutoriales en español sobre Bootstrap 5 puede que encuentres documentación de la versión 5.1 (bootstrap.esdocu.com). Es una traducción no oficial que se quedó desactualizada. La documentación oficial en inglés, que usaremos como referencia, está siempre en <https://getbootstrap.com/docs/5.3/>

Opción A — CDN (la que usaremos en esta fase)

Los links CDN que usaremos en **todos** nuestros archivos HTML son estos dos (versión **5.3.8**):

```
<!--  Bootstrap CSS — va en el <head> -->
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
      rel="stylesheet"
      integrity="sha384-sR14kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqukIAvNWLzeN8tE5YBujZqJLB"
      crossorigin="anonymous">

<!--  Bootstrap JS bundle (incluye Popper para modales/dropdowns) — antes del </body> -->
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"
        integrity="sha384-FKyoEForCGlyvwx9Hj09JcYn3nv7wiPvLz7YYwJrWVcXK/BmnVDxM+D2scQbITxI"
        crossorigin="anonymous"></script>
```



Note

El atributo `integrity` contiene un hash criptográfico que el navegador verifica antes de ejecutar el archivo. Si el archivo hubiese sido manipulado en el CDN (ataque de cadena de suministro), el navegador lo rechaza. Es una buena práctica incluirlo siempre.

Opción B — Descargar los archivos y servirlos desde `static/`

 **Referencia oficial:** Download · Bootstrap 5.3 → <https://getbootstrap.com/docs/5.3/getting-started/download/>

El CDN es cómodo, pero depende de tener conexión a Internet. La alternativa es **descargar los archivos compilados** de Bootstrap e incluirlos en nuestro proyecto, dentro de `static/`. Spring Boot los servirá igual que sirve nuestros HTML.

Descargamos el ZIP oficial (*Compiled CSS and JS*) desde la página de descargas, o directamente por consola:

```
cd ~/Descargas
# ZIP oficial con los archivos ya compilados (mismo contenido que el CDN)
wget https://github.com/twbs/bootstrap/releases/download/v5.3.8/bootstrap-5.3.8-dist.zip
unzip bootstrap-5.3.8-dist.zip
# Dentro hay dos carpetas: css/ y js/
ls bootstrap-5.3.8-dist
```

De todo el contenido solo necesitamos **dos archivos** (el resto son variantes: sin minificar, por módulos, mapas de depuración `.map` ...):

```
# Creamos las carpetas css/ y js/ dentro de static/ y copiamos lo necesario
cd ~/SpringBoot_Angular/crudalumnos/src/main/resources/static
mkdir -p css js
cp ~/Descargas/bootstrap-5.3.8-dist/css/bootstrap.min.css      css/
cp ~/Descargas/bootstrap-5.3.8-dist/js/bootstrap.bundle.min.js js/
```

La estructura queda así:

```
crudalumnos/src/main/resources/static/
├── css/
│   └── bootstrap.min.css
├── js/
│   └── bootstrap.bundle.min.js
├── index.html
└── ...resto de páginas
```

Y en el HTML sustituiríamos los enlaces al CDN por **rutas locales** (Spring Boot sirve `static/` en la raíz):

```
<!-- CSS local — va en el <head> -->
<link href="/css/bootstrap.min.css" rel="stylesheet">

<!-- JS local — antes del </body> -->
<script src="/js/bootstrap.bundle.min.js"></script>
```

**Note**

Con archivos locales **no se usa el atributo `integrity`**: ese hash solo tiene sentido cuando el archivo viene de un servidor externo (CDN) que podría ser manipulado. Nuestros archivos locales ya están bajo nuestro control.

**Tip**

¿CDN o local? El CDN es ideal para prototipos y ejemplos (cero instalación). La descarga local es mejor cuando: no hay garantía de Internet (aula, demo), queremos **congelar** la versión exacta, o la aplicación va a producción en una intranet.

Opción C — Instalar Bootstrap vía NPM

Referencia oficial: Download → *Install with package managers* · Bootstrap 5.3 → <https://getbootstrap.com/docs/5.3/getting-started/download/>

La tercera vía es la que usan los proyectos JavaScript profesionales: instalar Bootstrap como **dependencia npm**. `npm` es el gestor de paquetes de Node.js (lo instalaremos formalmente en el apartado 5.1 con NVM; si aún no lo tienes, puedes adelantar esa instalación).

```
# Creamos una carpeta de pruebas y la inicializamos como proyecto npm
mkdir -p ~/SpringBoot_Angular/pruebas-bootstrap
cd ~/SpringBoot_Angular/pruebas-bootstrap
npm init -y                # genera package.json

# Instalamos Bootstrap 5.3.8 como dependencia
npm install bootstrap@5.3.8

# Los archivos compilados quedan en node_modules:
ls node_modules/bootstrap/dist
# css/  js/  → el MISMO contenido que el ZIP de la Opción B
```

Con `npm install`, Bootstrap queda declarado en `package.json`:

```
"dependencies": {
  "bootstrap": "^5.3.8"
}
```

En un proyecto de páginas estáticas como el de esta fase, la forma de "añadirlo al proyecto" es copiar los archivos de `node_modules` a `static/` (igual que en la Opción B):

```
cp node_modules/bootstrap/dist/css/bootstrap.min.css \
~/SpringBoot_Angular/crudalumnos/src/main/resources/static/css/
cp node_modules/bootstrap/dist/js/bootstrap.bundle.min.js \
~/SpringBoot_Angular/crudalumnos/src/main/resources/static/js/
```

**Note**

La ventaja real de npm la veremos dos veces más adelante:

1. En **Angular (apartado 5.2)** haremos `npm install bootstrap@5.3.8` dentro del proyecto y `angular.json` empaquetará Bootstrap automáticamente — sin copiar nada a mano.
2. En el **apartado 4.12 (SASS)** usaremos el paquete npm de Bootstrap como código fuente para compilar nuestra propia versión con colores personalizados.

En esta fase seguiremos usando la **Opción A (CDN)** en todos los ejemplos, por simplicidad.

Creamos nuestra primera página. En `src/main/resources/static/index.html` :

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <!-- ⚠ Imprescindible para el responsive: sin esto el grid no funciona en móvil -->
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Academia Arelance</title>

  <!-- Bootstrap CSS (jsDelivr - CDN recomendado por la doc oficial) -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
        rel="stylesheet"
        integrity="sha384-sRI14kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqukuIAvNWLZeN8tE5YBujZqJLB"
        crossorigin="anonymous">
</head>
<body class="bg-light">

  <!-- container: centra el contenido y añade márgenes laterales automáticos -->
  <div class="container mt-5">
    <!-- display-4: clase tipográfica Bootstrap para títulos grandes -->
    <h1 class="display-4">🎓 Academia Arelance</h1>
    <!-- lead: párrafo introductorio ligeramente más grande -->
    <p class="lead text-muted">Panel de gestión académica</p>
    <hr>

    <!-- list-group: lista estilizada con bordes y hover incluidos -->
    <!-- w-50: width 50% (utilidad de anchura Bootstrap) -->
    <div class="list-group w-50">
      <!-- list-group-item-action: añade cursor pointer y efecto hover -->
      <a href="/alumnos.html" class="list-group-item list-group-item-action">👤 Alumnos</a>
      <a href="/asignaturas.html" class="list-group-item list-group-item-action">📖 Asignaturas</a>
      <a href="/grid.html" class="list-group-item list-group-item-action">📏 Grid y Responsive</a>
      <a href="/componentes.html" class="list-group-item list-group-item-action">🧩 Componentes</a>
      <a href="/formularios.html" class="list-group-item list-group-item-action">📄 Formularios</a>
    </div>
  </div>

  <!-- Bootstrap JS bundle - necesario para modales, dropdowns, tooltips... -->
  <!-- Va justo antes del </body> para no bloquear la carga del HTML -->
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"
        integrity="sha384-FKyoEForCGlyvw9Hj09JcYn3nv7wiPVlZ7YYwJrWVcXK/BmnVDxM+D2scQbITxI"
        crossorigin="anonymous"></script>
</body>
</html>
```

Iniciamos Spring Boot y comprobamos en `http://localhost:8080/` :

```
cd ~/SpringBoot_Angular/crudalumnos
./mvnw spring-boot:run
# Spring Boot sirve automáticamente los archivos de static/ en la raíz
# http://localhost:8080/ → carga index.html
```



4.2. Sistema de rejilla (Grid) y responsive

[Tabla de contenidos](#)

Referencias oficiales:

- Grid system → <https://getbootstrap.com/docs/5.3/layout/grid/>
- Breakpoints (sm, md, lg, xl, xxl) → <https://getbootstrap.com/docs/5.3/layout/breakpoints/>
- Columns → <https://getbootstrap.com/docs/5.3/layout/columns/>
- Gutters (separación g-*) → <https://getbootstrap.com/docs/5.3/layout/gutters/>
- Spacing (m-, p-) → <https://getbootstrap.com/docs/5.3/utilities/spacing/>

El **grid de Bootstrap** divide la pantalla en **12 columnas**. Usando clases como `col-6` (6 de 12 = mitad) o `col-md-4` (4 columnas solo en pantallas medianas en adelante) construimos layouts que se adaptan automáticamente a cualquier dispositivo.

Los **breakpoints** definen a qué ancho de pantalla aplica cada clase:

Clase	Breakpoint	Ancho mínimo
<code>col-</code>	xs (extra small)	< 576px
<code>col-sm-</code>	small	≥ 576px
<code>col-md-</code>	medium	≥ 768px
<code>col-lg-</code>	large	≥ 992px
<code>col-xl-</code>	extra large	≥ 1200px

Creamos `src/main/resources/static/grid.html` :

```

<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Grid Bootstrap</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
        rel="stylesheet"
        integrity="sha384-sRl14kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqukuIAvNWLzeN8tE5YBujZqJLB"
        crossorigin="anonymous">
  <style>
    /* Clase auxiliar para visualizar las columnas del grid durante el aprendizaje */
    .caja { background: #0d6efd22; border: 1px solid #0d6efd;
           padding: 1rem; text-align: center; margin-bottom: 1rem; }
  </style>
</head>
<body class="bg-light p-4">

  <h2 class="mb-4">🔧 Sistema de rejilla</h2>
  <!-- btn-outline-secondary: botón con borde pero sin fondo – bueno para acciones secundarias -->
  <a href="/" class="btn btn-outline-secondary btn-sm mb-4">← Volver</a>

  <!-- EJEMPLO 1: row + col-4 → 3 columnas iguales (3 × 4 = 12) -->
  <h5>3 columnas iguales (col-4)</h5>
  <div class="row mb-4">
    <!-- col-4: ocupa 4 de las 12 columnas disponibles -->
    <div class="col-4"><div class="caja">col-4</div></div>
    <div class="col-4"><div class="caja">col-4</div></div>
    <div class="col-4"><div class="caja">col-4</div></div>
  </div>

  <!-- EJEMPLO 2: responsive – apiladas en móvil, en fila en escritorio -->
  <!-- col-12: ocupa todo el ancho (móvil) -->
  <!-- col-md-4: desde breakpoint md (≥768px) pasa a 4/12 columnas -->
  <h5>Responsive: col-12 en móvil → col-md-4 en escritorio</h5>
  <div class="row mb-4">
    <div class="col-12 col-md-4"><div class="caja">Bloque A</div></div>
    <div class="col-12 col-md-4"><div class="caja">Bloque B</div></div>
    <div class="col-12 col-md-4"><div class="caja">Bloque C</div></div>
  </div>

  <!-- EJEMPLO 3: layout sidebar + contenido (3 + 9 = 12) -->
  <h5>Sidebar + contenido (col-md-3 + col-md-9)</h5>
  <div class="row mb-4">
    <div class="col-md-3"><div class="caja">Sidebar</div></div>
    <div class="col-md-9"><div class="caja">Contenido principal</div></div>
  </div>

  <!-- EJEMPLO 4: utilidades de espaciado -->
  <!-- m- = margin, p- = padding; números del 0 al 5 (0=0px, 5=3rem) -->
  <!-- t=top, b=bottom, s=start(izq), e=end(der), x=horizontal, y=vertical -->
  <h5>Utilidades de espaciado: m- (margin) y p- (padding)</h5>
  <div class="row">
    <div class="col-md-6">
      <div class="caja p-4 mb-2">p-4 (padding 1.5rem) + mb-2 (margin-bottom 0.5rem)</div>
      <div class="caja p-1">p-1 (padding 0.25rem)</div>
    </div>
    <div class="col-md-6">
      <!-- mt-4: margin-top de 1.5rem -->
      <div class="caja mt-4">mt-4 (margin-top 1.5rem)</div>
    </div>
  </div>

  <!-- Bootstrap JS – no es necesario para el grid, pero lo incluimos por coherencia -->
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"
        integrity="sha384-FKyoEForCGlyvvx9Hj09JcYn3nv7wiPVlZ7YYwJrWVcXK/BmnVDxM+D2scQbITxI"
        crossorigin="anonymous"></script>

```



```
</body>
</html>
```



Tip

Redimensiona la ventana del navegador mientras ves esta página para ver cómo las columnas se apilan automáticamente en pantallas pequeñas.



4.3. Tipografía, colores y utilidades

[Tabla de contenidos](#)

 Referencias oficiales:

- Typography → <https://getbootstrap.com/docs/5.3/content/typography/>
- Colors (text-*) → <https://getbootstrap.com/docs/5.3/utilities/colors/>
- Background (bg-*) → <https://getbootstrap.com/docs/5.3/utilities/background/>
- Spacing (m-, p-) → <https://getbootstrap.com/docs/5.3/utilities/spacing/>
- Buttons → <https://getbootstrap.com/docs/5.3/components/buttons/>
- Badges → <https://getbootstrap.com/docs/5.3/components/badge/>
- Alerts → <https://getbootstrap.com/docs/5.3/components/alerts/>
- Flex → <https://getbootstrap.com/docs/5.3/utilities/flex/>

Bootstrap incluye clases de utilidad para controlar tipografía, colores semánticos y espaciado sin escribir CSS propio. Los **colores semánticos** son los más importantes:

Clase	Color	Uso típico
<code>primary</code>	Azul	Acciones principales
<code>secondary</code>	Gris	Acciones secundarias
<code>success</code>	Verde	Confirmación / OK
<code>danger</code>	Rojo	Error / Borrar
<code>warning</code>	Amarillo	Advertencia
<code>info</code>	Cian	Información
<code>light</code> / <code>dark</code>	Blanco / Negro	Fondos y textos

Estos colores se aplican con prefijos como `text-` , `bg-` , `btn-` , `badge bg-` , `alert alert-` , etc.

Añadimos a `componentes.html` una sección de tipografía y colores:

```

<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Componentes Bootstrap</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
        rel="stylesheet"
        integrity="sha384-sRI14kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqukuIAvNWLzeN8tE5YBujZqJLB"
        crossorigin="anonymous">
</head>
<body class="bg-light p-4">

  <h2 class="mb-4">🌸 Tipografía y colores</h2>
  <a href="/" class="btn btn-outline-secondary btn-sm mb-4">← Volver</a>

  <!-- — TIPOGRAFÍA ————— -->
  <h5>Encabezados y texto</h5>
  <div class="card p-3 mb-4">
    <!-- display-6: clase Bootstrap para títulos visualmente grandes (no semántico) -->
    <h1 class="display-6">display-6 – Título grande</h1>
    <!-- lead: párrafo introductorio, algo más grande y ligero que el texto normal -->
    <p class="lead">lead – Párrafo destacado introductorio</p>
    <p>Texto normal. <strong>Negrita</strong>. <em>Cursiva</em>. <mark>Resaltado</mark>.</p>
    <!-- text-muted: color gris suave, ideal para texto secundario -->
    <p class="text-muted">text-muted – texto atenuado (notas, pie de página)</p>
    <small class="text-secondary">small + text-secondary</small>
  </div>

  <!-- — COLORES DE TEXTO ————— -->
  <!-- Prefijo text- aplicado al nombre del color semántico -->
  <h5>Colores de texto</h5>
  <div class="card p-3 mb-4">
    <p class="text-primary">text-primary (azul)</p>
    <p class="text-success">text-success (verde)</p>
    <p class="text-danger">text-danger (rojo)</p>
    <p class="text-warning">text-warning (amarillo)</p>
    <p class="text-info">text-info (cian)</p>
  </div>

  <!-- — BOTONES ————— -->
  <!-- btn: clase base obligatoria; btn-{color}: relleno sólido -->
  <!-- btn-outline-{color}: solo borde, sin fondo -->
  <!-- btn-sm / btn-lg: tamaño pequeño / grande -->
  <h5>Botones</h5>
  <div class="card p-3 mb-4">
    <!-- d-flex: display flex; flex-wrap: permite salto de línea; gap-2: espacio entre elementos -->
    <div class="d-flex flex-wrap gap-2">
      <button class="btn btn-primary">primary</button>
      <button class="btn btn-secondary">secondary</button>
      <button class="btn btn-success">success</button>
      <button class="btn btn-danger">danger</button>
      <button class="btn btn-warning">warning</button>
      <button class="btn btn-outline-primary">outline-primary</button>
      <button class="btn btn-sm btn-dark">btn-sm</button>
      <button class="btn btn-lg btn-info">btn-lg</button>
    </div>
  </div>

  <!-- — BADGES ————— -->
  <!-- badge: etiqueta pequeña para contadores o estados -->
  <!-- bg-{color}: color de fondo (prefijo bg-, no btn-) -->
  <h5>Badges</h5>
  <div class="card p-3 mb-4">
    <div class="d-flex gap-2">
      <span class="badge bg-primary">primary</span>
      <span class="badge bg-success">success</span>
    </div>
  </div>

```

```

<span class="badge bg-danger">danger</span>
<!-- text-dark: el fondo amarillo necesita texto oscuro para contraste -->
<span class="badge bg-warning text-dark">warning</span>
<span class="badge bg-secondary">secondary</span>
</div>
</div>

<!-- --- ALERTS --- -->
<!-- alert: caja de aviso con color de fondo suave -->
<!-- alert-{color}: variante del color semántico -->
<h5>Alerts</h5>
<div class="alert alert-success">✔ Operación completada correctamente.</div>
<div class="alert alert-danger">✖ Error al conectar con el servidor.</div>
<div class="alert alert-warning">⚠ Este alumno ya está matriculado en esa asignatura.</div>
<div class="alert alert-info">ℹ Los datos se actualizan automáticamente.</div>

<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"
  integrity="sha384-FKYoEForCGlyvwx9Hj09JcYn3nv7w1PVLz7YYwJrWVcXK/BmnVDxM+D2scQbITxI"
  crossorigin="anonymous"></script>

</body>
</html>

```



4.4. Componentes: navbar, cards, badges y alerts

Tabla de contenidos

Referencias oficiales:

- Navbar → <https://getbootstrap.com/docs/5.3/components/navbar/>
- Cards → <https://getbootstrap.com/docs/5.3/components/card/>
- Collapse (el menú hamburguesa de la navbar) → <https://getbootstrap.com/docs/5.3/components/collapse/>
- Badges → <https://getbootstrap.com/docs/5.3/components/badge/>
- Alerts → <https://getbootstrap.com/docs/5.3/components/alerts/>

Ahora añadimos una **navbar** de navegación y **cards** a nuestro `index.html`. La navbar es el componente de Bootstrap que actúa como barra de menú superior. Las cards son contenedores con cabecera, cuerpo y pie de página.

Actualizamos `src/main/resources/static/index.html` con la versión con navbar:

```

<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Academia Arelance</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
        rel="stylesheet"
        integrity="sha384-sRl14kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqukuIAvNWLzeN8tE5YBujZqJLB"
        crossorigin="anonymous">
</head>
<body class="bg-light">

  <!-- NAVBAR ----- -->
  <!-- navbar-expand-lg: en pantallas grandes muestra el menú horizontal; -->
  <!-- en pantallas pequeñas lo colapsa en un botón ☰ -->
  <!-- navbar-dark bg-primary: texto blanco sobre fondo azul primario -->
  <nav class="navbar navbar-expand-lg navbar-dark bg-primary">
    <div class="container">
      <!-- navbar-brand: logo o nombre del sitio, alineado a la izquierda -->
      <a class="navbar-brand" href="/">🏠 Academia Arelance</a>

      <!-- Botón hamburguesa ☰ – visible solo en pantallas pequeñas -->
      <!-- data-bs-toggle/target conectan el botón con el div#menu de abajo -->
      <button class="navbar-toggler" type="button"
            data-bs-toggle="collapse" data-bs-target="#menu">
        <span class="navbar-toggler-icon"></span>
      </button>

      <!-- El menú: se colapsa en móvil, se expande en escritorio -->
      <div class="collapse navbar-collapse" id="menu">
        <!-- ms-auto: margin-start automático → empuja el menú a la derecha -->
        <ul class="navbar-nav ms-auto">
          <!-- active: indica la página actual -->
          <li class="nav-item"><a class="nav-link active" href="/">Dashboard</a></li>
          <li class="nav-item"><a class="nav-link" href="/alumnos.html">Alumnos</a></li>
          <li class="nav-item"><a class="nav-link" href="/asignaturas.html">Asignaturas</a></li>
        </ul>
      </div>
    </div>
  </nav>

  <!-- CONTENIDO PRINCIPAL ----- -->
  <div class="container mt-4">
    <h2 class="mb-1">Dashboard</h2>
    <p class="text-muted mb-4">Resumen académico · curso 2025-2026</p>

    <!-- Cards de resumen – g-3: gap entre columnas y filas (gutter) -->
    <div class="row g-3 mb-4">

      <!-- border-{color}: borde de la card del color semántico indicado -->
      <div class="col-md-3">
        <div class="card border-primary">
          <div class="card-body text-center">
            <!-- id para que JS inyecte aquí el dato real del backend -->
            <h3 class="text-primary" id="totalAlumnos"></h3>
            <p class="card-text text-muted mb-0">Alumnos registrados</p>
          </div>
        </div>
      </div>
      <div class="col-md-3">
        <div class="card border-success">
          <div class="card-body text-center">
            <h3 class="text-success" id="totalMujeres"></h3>
            <p class="card-text text-muted mb-0">Alumnas</p>
          </div>
        </div>
      </div>
    </div>
  </div>

```

```

    </div>
    <div class="col-md-3">
      <div class="card border-info">
        <div class="card-body text-center">
          <h3 class="text-info" id="totalHombres"></h3>
          <p class="card-text text-muted mb-0">Alumnos</p>
        </div>
      </div>
    </div>
    <div class="col-md-3">
      <div class="card border-warning">
        <div class="card-body text-center">
          <h3 class="text-warning" id="totalAsignaturas"></h3>
          <p class="card-text text-muted mb-0">Asignaturas</p>
        </div>
      </div>
    </div>
  </div>

  <!-- CARDS DE ACCESO RÁPIDO -->
  <h5>Accesos rápidos</h5>
  <!-- h-100: height 100% -- todas las cards de la fila miden lo mismo -->
  <div class="row g-3">
    <div class="col-md-4">
      <div class="card h-100">
        <!-- card-header: cabecera de la card con color de fondo -->
        <div class="card-header bg-primary text-white"><img alt="User icon" data-bbox="288 405 303 415"/> Alumnos</div>
        <div class="card-body">
          <p class="card-text">Consulta, crea, edita y elimina alumnos.</p>
          <a href="/alumnos.html" class="btn btn-primary btn-sm">Gestionar</a>
        </div>
      </div>
    </div>
    <div class="col-md-4">
      <div class="card h-100">
        <div class="card-header bg-success text-white"><img alt="Book icon" data-bbox="288 515 303 525"/> Asignaturas</div>
        <div class="card-body">
          <p class="card-text">Consulta el catálogo de asignaturas disponibles.</p>
          <a href="/asignaturas.html" class="btn btn-success btn-sm">Ver catálogo</a>
        </div>
      </div>
    </div>
    <div class="col-md-4">
      <div class="card h-100">
        <div class="card-header bg-secondary text-white"><img alt="Gear icon" data-bbox="288 625 303 635"/> Componentes</div>
        <div class="card-body">
          <p class="card-text">Galería de componentes y utilidades Bootstrap.</p>
          <a href="/componentes.html" class="btn btn-secondary btn-sm">Ver galería</a>
        </div>
      </div>
    </div>
  </div>

  <!-- Bootstrap JS -- necesario para el comportamiento del navbar colapsable -->
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"
    integrity="sha384-FKYoEForCGlyvw9Hj09JcYn3nv7wiPVLz7YYwJrWVCXK/BmnVDxM+D2scQbITxI"
    crossorigin="anonymous"></script>

  <script>
    // -- FETCH para rellenar las cards del dashboard con datos reales --
    // Cada llamada es independiente: si una falla, las demás siguen funcionando
    // .catch(() => {}) captura el error silenciosamente (la card queda en "--")

    fetch('/api/alumnos/contar')
      .then(r => r.json())
      .then(n => document.getElementById('totalAlumnos').textContent = n)
      .catch(() => {});
  </script>

```

```

fetch('/api/alumnos/consultar/contar/mujeres')
  .then(r => r.json())
  .then(n => document.getElementById('totalMujeres').textContent = n)
  .catch(() => {});

fetch('/api/alumnos/consultar/contar/hombres')
  .then(r => r.json())
  .then(n => document.getElementById('totalHombres').textContent = n)
  .catch(() => {});

// Para asignaturas no tenemos endpoint /contar, así que contamos el array
fetch('/api/asignaturas/consultar')
  .then(r => r.json())
  .then(a => document.getElementById('totalAsignaturas').textContent = a.length)
  .catch(() => {});
</script>
</body>
</html>

```

**Note**

El botón `navbar-toggler` activa el menú hamburguesa en pantallas pequeñas. Requiere el JS de Bootstrap (`bootstrap.bundle.min.js`) para funcionar.



4.5. Tablas Bootstrap

Tabla de contenidos

Referencias oficiales:

- Tables → <https://getbootstrap.com/docs/5.3/content/tables/>
- Spinners (indicador de carga) → <https://getbootstrap.com/docs/5.3/components/spinners/>
- Shadows (shadow-sm) → <https://getbootstrap.com/docs/5.3/utilities/shadows/>

Las tablas de Bootstrap se construyen con la clase `table` y se pueden personalizar con modificadores como `table-striped` (filas alternas), `table-hover` (resaltado al pasar el ratón), `table-bordered` (bordes en todas las celdas) o `table-sm` (más compacta).

Creamos `src/main/resources/static/asignaturas.html` con datos hardcodeados primero:

```

<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Asignaturas – Academia Arelance</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
        rel="stylesheet"
        integrity="sha384-sRI14kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqukuIAvNWLzeN8tE5YBujZqJLB"
        crossorigin="anonymous">
</head>
<body class="bg-light">

  <nav class="navbar navbar-dark bg-success">
    <div class="container">
      <a class="navbar-brand" href="/">🏠 Academia Arelance</a>
      <a href="/" class="btn btn-outline-light btn-sm">Volver</a>
    </div>
  </nav>

  <div class="container mt-4">
    <div class="d-flex justify-content-between align-items-center mb-3">
      <h2>📚 Asignaturas</h2>
      <span class="badge bg-success fs-6" id="contadorAsig">—</span>
    </div>

    <!-- Alerta de error (oculta por defecto) -->
    <div id="alertaError" class="alert alert-danger d-none">
      ❌ No se pudieron cargar las asignaturas. ¿Está Spring Boot en marcha?
    </div>

    <!-- Tabla -->
    <div class="card shadow-sm">
      <div class="card-body p-0">
        <table class="table table-striped table-hover align-middle mb-0">
          <thead class="table-success">
            <tr>
              <th>ID</th>
              <th>Denominación</th>
              <th>Curso</th>
              <th>Aulas</th>
            </tr>
          </thead>
          <tbody id="tablaAsignaturas">
            <tr>
              <td colspan="4" class="text-center text-muted py-3">
                <div class="spinner-border spinner-border-sm text-success me-2"></div>
                Cargando asignaturas...
              </td>
            </tr>
          </tbody>
        </table>
      </div>
    </div>
  </div>

  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"
        integrity="sha384-FKyoeFForCGlyvwx9Hj09JcYn3nv7wiPVlZ7YYwJrWvcXK/BmnVDxM+D2scQbITxI"
        crossorigin="anonymous"></script>
  <script>
    fetch('/api/asignaturas/consultar')
      .then(res => {
        if (!res.ok) throw new Error('Error HTTP ' + res.status);
        return res.json();
      })
      .then(asignaturas => {
        const tbody = document.getElementById('tablaAsignaturas');

```

```

document.getElementById('contadorAsig').textContent = asignaturas.length + ' asignaturas';

if (asignaturas.length === 0) {
  tbody.innerHTML = '<tr><td colspan="4" class="text-center text-muted">No hay asignaturas registradas.</td></tr>';
  return;
}

tbody.innerHTML = asignaturas.map(a => `
  <tr>
    <td><span class="badge bg-secondary">${a.id}</span></td>
    <td><strong>${a.denominacion}</strong></td>
    <td><span class="badge bg-success">${a.curso}</span></td>
    <td>${a.aulas} && a.aulas.length ? a.aulas.join(' ') : '<span class="text-muted"></span>`</td>
  </tr>
`).join('');
})
.catch(() => {
  document.getElementById('tablaAsignaturas').innerHTML =
    '<tr><td colspan="4" class="text-center text-danger">Error al cargar</td></tr>';
  document.getElementById('alertaError').classList.remove('d-none');
});
</script>
</body>
</html>

```

**Tip**

El `spinner-border` de Bootstrap es un indicador de carga CSS puro — no requiere ninguna librería adicional. Aparece mientras el `fetch()` está en curso y desaparece cuando llegan los datos.



4.6. Modales

Tabla de contenidos

Referencias oficiales:

- Modal → <https://getbootstrap.com/docs/5.3/components/modal/>
- Buttons → <https://getbootstrap.com/docs/5.3/components/buttons/>

Los **modales** son ventanas emergentes que se superponen al contenido. Bootstrap los gestiona con atributos `data-bs-toggle="modal"` y `data-bs-target="#idModal"`, sin necesidad de escribir JavaScript para abrirlos y cerrarlos.

Son perfectos para formularios de confirmación de borrado o inserción de datos. Añadimos a `componentes.html` un ejemplo de modal:


```

<!-- Botón que abre el modal -->
<button type="button" class="btn btn-danger"
        data-bs-toggle="modal" data-bs-target="#modalConfirmar">
  🗑 Borrar alumno
</button>

<!-- Definición del modal -->
<div class="modal fade" id="modalConfirmar" tabindex="-1">
  <div class="modal-dialog">
    <div class="modal-content">

      <div class="modal-header bg-danger text-white">
        <h5 class="modal-title">⚠ Confirmar borrado</h5>
        <button type="button" class="btn-close btn-close-white"
          data-bs-dismiss="modal"></button>
      </div>

      <div class="modal-body">
        ¿Estás seguro de que quieres eliminar este alumno?
        Esta acción <strong>no se puede deshacer</strong>.
      </div>

      <div class="modal-footer">
        <button type="button" class="btn btn-secondary"
          data-bs-dismiss="modal">Cancelar</button>
        <button type="button" class="btn btn-danger" id="btnConfirmarBorrado">
          Sí, borrar
        </button>
      </div>
    </div>
  </div>
</div>

```

Usaremos este modal en la página de alumnos para confirmar los borrados.



Warning

Los modales requieren el archivo `bootstrap.bundle.min.js` para funcionar. Sin él, el modal no se abrirá al pulsar el botón.



4.7. Formularios y validación visual

Tabla de contenidos

Referencias oficiales:

- Forms overview → <https://getbootstrap.com/docs/5.3/forms/overview/>
- Form controls (inputs) → <https://getbootstrap.com/docs/5.3/forms/form-control/>
- Select → <https://getbootstrap.com/docs/5.3/forms/select/>
- Checks & radios → <https://getbootstrap.com/docs/5.3/forms/checks-radios/>
- Validation → <https://getbootstrap.com/docs/5.3/forms/validation/>

Bootstrap incluye clases para estilizar formularios de forma consistente. Las clases clave son `form-label`, `form-control` (para inputs y textareas), `form-select` (para selects) y `form-check` (para checkboxes y radios).

La **validación visual** se activa añadiendo las clases `is-valid` o `is-invalid` a los campos, y mostrando mensajes con `valid-feedback` / `invalid-feedback`.

Creemos `src/main/resources/static/formularios.html` :

```

<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Formularios – Academia Arelance</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
        rel="stylesheet"
        integrity="sha384-sRI14kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqukuIAvNWLzeN8tE5YBujZqJLB"
        crossorigin="anonymous">
</head>
<body class="bg-light">

  <nav class="navbar navbar-dark bg-secondary">
    <div class="container">
      <a class="navbar-brand" href="/">🏠 Academia Arelance</a>
      <a href="/" class="btn btn-outline-light btn-sm">- Volver</a>
    </div>
  </nav>

  <div class="container mt-4">
    <h2 class="mb-4">📁 Formularios Bootstrap</h2>

    <div class="row g-4">

      <!-- Formulario básico -->
      <div class="col-md-6">
        <div class="card shadow-sm">
          <div class="card-header">Formulario básico</div>
          <div class="card-body">
            <div class="mb-3">
              <label class="form-label">NIF</label>
              <input type="text" class="form-control" placeholder="12345678A">
            </div>
            <div class="mb-3">
              <label class="form-label">Nombre completo</label>
              <input type="text" class="form-control" placeholder="Ana García López">
            </div>
            <div class="mb-3">
              <label class="form-label">Edad</label>
              <input type="number" class="form-control" min="16" max="99" placeholder="22">
            </div>
            <div class="mb-3">
              <label class="form-label">Género</label>
              <select class="form-select">
                <option value="">Selecciona...</option>
                <option value="false">♂ Hombre</option>
                <option value="true">♀ Mujer</option>
              </select>
            </div>
            <button class="btn btn-primary">Guardar</button>
          </div>
        </div>
      </div>

      <!-- Formulario con validación visual -->
      <div class="col-md-6">
        <div class="card shadow-sm">
          <div class="card-header">Validación visual</div>
          <div class="card-body">
            <div class="mb-3">
              <label class="form-label">NIF</label>
              <input type="text" class="form-control is-valid" value="12345678A">
              <div class="valid-feedback">✅ NIF válido</div>
            </div>
            <div class="mb-3">
              <label class="form-label">Nombre</label>
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>

```

```

        <input type="text" class="form-control is-invalid" value="">
        <div class="invalid-feedback">❌ El nombre es obligatorio</div>
      </div>
      <div class="mb-3">
        <label class="form-label">Edad</label>
        <input type="number" class="form-control is-invalid" value="200">
        <div class="invalid-feedback">❌ La edad debe estar entre 16 y 99</div>
      </div>
      <div class="form-check mb-3">
        <input class="form-check-input" type="checkbox" id="checkGenero">
        <label class="form-check-label" for="checkGenero">Mujer</label>
      </div>
      <button class="btn btn-success" onclick="validarFormulario()">Validar</button>
    </div>
  </div>
</div>

<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"
  integrity="sha384-FKyoEForCGlyvvw9Hj09JcYn3nv7wiPVlZ7YYwJrWVcXK/BmnVDxM+D2scQbITxI"
  crossorigin="anonymous"></script>
<script>
  function validarFormulario() {
    // Ejemplo de validación manual con clases Bootstrap
    const campos = document.querySelectorAll('.form-control');
    campos.forEach(campo => {
      if (campo.value.trim() === '') {
        campo.classList.add('is-invalid');
        campo.classList.remove('is-valid');
      } else {
        campo.classList.add('is-valid');
        campo.classList.remove('is-invalid');
      }
    });
  }
</script>
</body>
</html>

```



4.8. Fetch API: GET — listar datos reales

[Tabla de contenidos](#)

Referencias oficiales:

- Fetch API (MDN) → https://developer.mozilla.org/es/docs/Web/API/Fetch_API/Using_Fetch
- Input group (buscador) → <https://getbootstrap.com/docs/5.3/forms/input-group/>
- Tables → <https://getbootstrap.com/docs/5.3/content/tables/>
- Spinners → <https://getbootstrap.com/docs/5.3/components/spinners/>

La **Fetch API** es la forma moderna de hacer peticiones HTTP desde el navegador. Es una API nativa — no hay que instalar nada. Funciona con **Promesas** (`then` / `catch`) y devuelve un objeto `Response` que hay que convertir a JSON con `.json()` .

```
// Estructura básica de un fetch GET
fetch('/api/alumnos/consultar') // 1. Hacemos la petición
  .then(response => response.json()) // 2. Convertimos la respuesta a JSON
  .then(datos => { // 3. Usamos los datos
    console.log(datos);
  })
  .catch(error => { // 4. Capturamos errores de red
    console.error('Error:', error);
  });
```

Creamos `src/main/resources/static/alumnos.html` con el listado real desde el backend:

```

<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Alumnos - Academia Arelance</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
        rel="stylesheet"
        integrity="sha384-sRI14kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqukuIAvNWLzen8tE5YBujZqJLB"
        crossorigin="anonymous">
</head>
<body class="bg-light">

  <nav class="navbar navbar-dark bg-primary">
    <div class="container">
      <a class="navbar-brand" href="/">🏠 Academia Arelance</a>
      <a href="/" class="btn btn-outline-light btn-sm">- Volver</a>
    </div>
  </nav>

  <div class="container mt-4">
    <div class="d-flex justify-content-between align-items-center mb-3">
      <h2>👤 Alumnos</h2>
      <span class="badge bg-primary fs-6" id="contador">-</span>
    </div>

    <!-- Barra de búsqueda -->
    <div class="input-group mb-3 w-50">
      <input type="text" id="buscador" class="form-control"
        placeholder="🔍 Buscar por nombre...">
      <button class="btn btn-outline-primary" onclick="buscarPorNombre()">Buscar</button>
      <button class="btn btn-outline-secondary" onclick="cargarAlumnos()">Ver todos</button>
    </div>

    <!-- Alerta -->
    <div id="alerta" class="d-none mb-3"></div>

    <!-- Tabla -->
    <div class="card shadow-sm">
      <div class="card-body p-0">
        <table class="table table-striped table-hover align-middle mb-0">
          <thead class="table-primary">
            <tr>
              <th>NIF</th>
              <th>Nombre</th>
              <th>Edad</th>
              <th>Género</th>
            </tr>
          </thead>
          <tbody id="tablaAlumnos">
            <tr>
              <td colspan="4" class="text-center py-3">
                <div class="spinner-border spinner-border-sm text-primary me-2"></div>
                Cargando...
              </td>
            </tr>
          </tbody>
        </table>
      </div>
    </div>
  </div>

  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"
        integrity="sha384-FKyoEForCGlyvw9Hj09JcYn3nv7wiPVlz7YYwJrWcXK/BmnVDxM+D2scQbITxI"
        crossorigin="anonymous"></script>
  <script>
    const tbody = document.getElementById('tablaAlumnos');
  </script>

```

```

const contador = document.getElementById('contador');

// Renderiza un array de alumnos en la tabla
function renderTabla(alumnos) {
  contador.textContent = alumnos.length + ' alumnos';
  if (alumnos.length === 0) {
    tbody.innerHTML = '<tr><td colspan="4" class="text-center text-muted">Sin resultados</td></tr>';
    return;
  }
  tbody.innerHTML = alumnos.map(a => `
    <tr>
      <td><code>${a.nif}</code></td>
      <td>${a.nombre}</td>
      <td>${a.edad}</td>
      <td>
        <span class="badge ${a.genero ? 'bg-danger' : 'bg-info'}">
          ${a.genero ? '♀ Mujer' : '♂ Hombre'}
        </span>
      </td>
    </tr>
  `).join('');
}

// GET todos los alumnos
function cargarAlumnos() {
  document.getElementById('buscador').value = '';
  fetch('/api/alumnos/consultar')
    .then(r => r.json())
    .then(renderTabla)
    .catch(() => mostrarAlerta('danger', '❌ Error al conectar con el servidor'));
}

// GET búsqueda por nombre
function buscarPorNombre() {
  const nombre = document.getElementById('buscador').value.trim();
  if (!nombre) { cargarAlumnos(); return; }
  fetch(`/api/alumnos/consultar/nombre?nombre=${encodeURIComponent(nombre)}`)
    .then(r => r.json())
    .then(renderTabla)
    .catch(() => mostrarAlerta('danger', '❌ Error en la búsqueda'));
}

function mostrarAlerta(tipo, mensaje) {
  const alerta = document.getElementById('alerta');
  alerta.className = `alert alert-${tipo}`;
  alerta.textContent = mensaje;
  alerta.classList.remove('d-none');
  setTimeout(() => alerta.classList.add('d-none'), 4000);
}

// Cargamos al inicio
cargarAlumnos();
</script>
</body>
</html>

```

**Note**

`encodeURIComponent()` codifica caracteres especiales en la URL (tildes, espacios, etc.) para que la petición llegue correctamente al backend. Por ejemplo, "García" se convierte en "Garc%C3%ADa".

4.9. Fetch API: POST – insertar alumno

Tabla de contenidos

Referencias oficiales:

- Fetch API (MDN) → https://developer.mozilla.org/es/docs/Web/API/Fetch_API/Using_Fetch
- Modal → <https://getbootstrap.com/docs/5.3/components/modal/>
- Validation → <https://getbootstrap.com/docs/5.3/forms/validation/>

Un `fetch` POST necesita especificar el método, las cabeceras (`Content-Type: application/json`) y el cuerpo serializado con `JSON.stringify()` :

```
fetch('/api/alumnos/crear', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ nif: '11A', nombre: 'Ana', edad: 22, genero: true })
})
.then(r => r.json())
.then(alumno => console.log('Creado:', alumno))
.catch(err => console.error(err));
```

Ampliamos `alumnos.html` añadiendo un botón que abre un modal con el formulario de inserción. Añadimos al `<body>` , antes del `</body>` :

```

<!-- Botón añadir (encima de la tabla) -->
<!-- Añadir junto al h2: -->
<button class="btn btn-success btn-sm"
        data-bs-toggle="modal" data-bs-target="#modalNuevoAlumno">
    + Nuevo alumno
</button>

<!-- Modal de inserción -->
<div class="modal fade" id="modalNuevoAlumno" tabindex="-1">
    <div class="modal-dialog">
        <div class="modal-content">
            <div class="modal-header bg-success text-white">
                <h5 class="modal-title">+ Nuevo alumno</h5>
                <button type="button" class="btn-close btn-close-white"
                    data-bs-dismiss="modal"></button>
            </div>
            <div class="modal-body">
                <div class="mb-3">
                    <label class="form-label">NIF <span class="text-danger">*</span></label>
                    <input type="text" id="inNif" class="form-control" placeholder="12345678A">
                    <div class="invalid-feedback">El NIF es obligatorio</div>
                </div>
                <div class="mb-3">
                    <label class="form-label">Nombre <span class="text-danger">*</span></label>
                    <input type="text" id="inNombre" class="form-control" placeholder="Ana García López">
                    <div class="invalid-feedback">El nombre es obligatorio</div>
                </div>
                <div class="mb-3">
                    <label class="form-label">Edad <span class="text-danger">*</span></label>
                    <input type="number" id="inEdad" class="form-control" min="16" max="99" placeholder="22">
                    <div class="invalid-feedback">Edad entre 16 y 99</div>
                </div>
                <div class="mb-3">
                    <label class="form-label">Género <span class="text-danger">*</span></label>
                    <select id="inGenero" class="form-select">
                        <option value="">Selecciona...</option>
                        <option value="false">♂ Hombre</option>
                        <option value="true">♀ Mujer</option>
                    </select>
                    <div class="invalid-feedback">Selecciona un género</div>
                </div>
            </div>
            <div class="modal-footer">
                <button type="button" class="btn btn-secondary" data-bs-dismiss="modal">Cancelar</button>
                <button type="button" class="btn btn-success" onclick="crearAlumno()">Guardar</button>
            </div>
        </div>
    </div>
</div>

```

Y la función JavaScript en el `<script>` :


```
function crearAlumno() {
  const nif = document.getElementById('inNif').value.trim();
  const nombre = document.getElementById('inNombre').value.trim();
  const edad = parseInt(document.getElementById('inEdad').value);
  const genero = document.getElementById('inGenero').value;

  // Validación básica
  if (!nif || !nombre || isNaN(edad) || genero === '') {
    mostrarAlerta('warning', '⚠️ Rellena todos los campos obligatorios');
    return;
  }

  fetch('/api/alumnos/crear', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ nif, nombre, edad, genero: genero === 'true' })
  })
  .then(r => {
    if (!r.ok) throw new Error('Error ' + r.status);
    return r.json();
  })
  .then(() => {
    // Cerrar modal, recargar tabla y mostrar éxito
    bootstrap.Modal.getInstance(document.getElementById('modalNuevoAlumno')).hide();
    cargarAlumnos();
    mostrarAlerta('success', '✅ Alumno creado correctamente');
  })
  .catch(() => mostrarAlerta('danger', '❌ Error al crear el alumno. ¿Ya existe ese NIF?'));
}
```



4.10. Fetch API: PUT y DELETE — editar y borrar (CRUD completo)

[Tabla de contenidos](#)

Referencias oficiales:

- Fetch API (MDN) → https://developer.mozilla.org/es/docs/Web/API/Fetch_API/Using_Fetch
- Modal → <https://getbootstrap.com/docs/5.3/components/modal/>
- Buttons → <https://getbootstrap.com/docs/5.3/components/buttons/>

Los métodos `PUT` y `DELETE` siguen la misma estructura que `POST`, cambiando únicamente el `method`. En el caso de `DELETE` habitualmente no se envía cuerpo.

Completamos `alumnos.html` añadiendo las columnas de acciones en la tabla y las funciones correspondientes.

Actualizamos el `map` de `renderTabla` para incluir los botones de editar y borrar:

```
tbody.innerHTML = alumnos.map(a => `
<tr>
<td><code>${a.nif}</code></td>
<td>${a.nombre}</td>
<td>${a.edad}</td>
<td>
<span class="badge ${a.genero ? 'bg-danger' : 'bg-info'}">
  ${a.genero ? '♀ Mujer' : '♂ Hombre'}
</span>
</td>
<td>
<button class="btn btn-warning btn-sm me-1"
  onclick="abrirEditar('${a.nif}', '${a.nombre}', ${a.edad}, ${a.genero})">
    ✎ Editar
  </button>
<button class="btn btn-danger btn-sm"
  onclick="confirmarBorrar('${a.nif}')">
    🗑 Borrar
  </button>
</td>
</tr>
`).join('');
```

Modal de edición y función PUT:

```

<!-- Modal de edición -->
<div class="modal fade" id="modalEditar" tabindex="-1">
  <div class="modal-dialog">
    <div class="modal-content">
      <div class="modal-header bg-warning">
        <h5 class="modal-title">✎ Editar alumno</h5>
        <button type="button" class="btn-close" data-bs-dismiss="modal"></button>
      </div>
      <div class="modal-body">
        <input type="hidden" id="editNif">
        <div class="mb-3">
          <label class="form-label">Nombre</label>
          <input type="text" id="editNombre" class="form-control">
        </div>
        <div class="mb-3">
          <label class="form-label">Edad</label>
          <input type="number" id="editEdad" class="form-control" min="16" max="99">
        </div>
        <div class="mb-3">
          <label class="form-label">Género</label>
          <select id="editGenero" class="form-select">
            <option value="false">♂ Hombre</option>
            <option value="true">♀ Mujer</option>
          </select>
        </div>
      </div>
      <div class="modal-footer">
        <button class="btn btn-secondary" data-bs-dismiss="modal">Cancelar</button>
        <button class="btn btn-warning" onclick="guardarEdicion()">Guardar cambios</button>
      </div>
    </div>
  </div>
</div>

<!-- Modal de confirmación de borrado -->
<div class="modal fade" id="modalBorrar" tabindex="-1">
  <div class="modal-dialog">
    <div class="modal-content">
      <div class="modal-header bg-danger text-white">
        <h5 class="modal-title">⚠ Confirmar borrado</h5>
        <button type="button" class="btn-close btn-close-white" data-bs-dismiss="modal"></button>
      </div>
      <div class="modal-body">
        ¿Seguro que quieres borrar al alumno <strong id="borrarNif"></strong>?
        Esta acción <strong>no se puede deshacer</strong>.
      </div>
      <div class="modal-footer">
        <button class="btn btn-secondary" data-bs-dismiss="modal">Cancelar</button>
        <button class="btn btn-danger" onclick="ejecutarBorrado()">Sí, borrar</button>
      </div>
    </div>
  </div>
</div>
</div>

```

Funciones JavaScript para editar y borrar:

```

let nifABorrar = null;

function abrirEditar(nif, nombre, edad, genero) {
  document.getElementById('editNif').value = nif;
  document.getElementById('editNombre').value = nombre;
  document.getElementById('editEdad').value = edad;
  document.getElementById('editGenero').value = String(genero);
  new bootstrap.Modal(document.getElementById('modalEditar')).show();
}

function guardarEdicion() {
  const nif = document.getElementById('editNif').value;
  const nombre = document.getElementById('editNombre').value.trim();
  const edad = parseInt(document.getElementById('editEdad').value);
  const genero = document.getElementById('editGenero').value === 'true';

  fetch(`/api/alumnos/actualizar/${nif}`, {
    method: 'PUT',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ nombre, edad, genero })
  })
  .then(r => { if (!r.ok) throw new Error(); return r.json(); })
  .then(() => {
    bootstrap.Modal.getInstance(document.getElementById('modalEditar')).hide();
    cargarAlumnos();
    mostrarAlerta('success', '✅ Alumno actualizado correctamente');
  })
  .catch(() => mostrarAlerta('danger', '❌ Error al actualizar el alumno'));
}

function confirmarBorrar(nif) {
  nifABorrar = nif;
  document.getElementById('borrarNif').textContent = nif;
  new bootstrap.Modal(document.getElementById('modalBorrar')).show();
}

function ejecutarBorrado() {
  fetch(`/api/alumnos/borrar/${nifABorrar}`, { method: 'DELETE' })
  .then(r => { if (!r.ok) throw new Error(); })
  .then(() => {
    bootstrap.Modal.getInstance(document.getElementById('modalBorrar')).hide();
    cargarAlumnos();
    mostrarAlerta('success', `✅ Alumno ${nifABorrar} eliminado`);
    nifABorrar = null;
  })
  .catch(() => mostrarAlerta('danger', '❌ Error al borrar el alumno'));
}

```

Important

Recuerda que el endpoint `DELETE /api/alumnos/borrar/{nif}` del backend primero comprueba si existe el alumno y devuelve 404 si no lo encuentra. En el frontend comprobamos `r.ok` para detectar ese caso y mostrar el mensaje de error adecuado.



4.11. Personalizar los colores de Bootstrap

[Tabla de contenidos](#)

 Referencias oficiales:

- Color → <https://getbootstrap.com/docs/5.3/customize/color/>
- CSS variables → <https://getbootstrap.com/docs/5.3/customize/css-variables/>
- Color modes (tema claro/oscuro) → <https://getbootstrap.com/docs/5.3/customize/color-modes/>
- Sass → <https://getbootstrap.com/docs/5.3/customize/sass/>

El azul de `primary`, el verde de `success` ... son los colores por defecto de Bootstrap y se reconocen a la legua. Para darle identidad propia a la Academia Arelance tenemos **dos caminos**:

Método	Dificultad	Qué permite cambiar
A. Variables CSS (sin instalar nada)	★ Fácil	Utilidades (<code>text-</code> , <code>bg-</code> , <code>border-</code>) y componentes concretos, uno a uno
B. Compilar con SASS (apartado 4.12)	★★ Media	TODO de una vez: cambias <code>\$primary</code> y se regeneran botones, alerts, badges, formularios...

Método A – Variables CSS (el modo más sencillo)

Desde la versión 5.3, Bootstrap expone sus colores como **variables CSS** (*custom properties*) con el prefijo `--bs-`. Podemos sobrescribirlas en una hoja de estilos propia cargada **después** de Bootstrap, sin compilar nada.

Creemos `src/main/resources/static/css/arelance.css` :

```
/* css/arelance.css – Personalización de colores de la Academia Arelance */
/* IMPORTANTE: este archivo debe enlazarse DESPUÉS del CSS de Bootstrap */

/* 1 Utilidades: text-primary, bg-primary, border-primary...
   usan --bs-primary y --bs-primary-rgb, así que basta redefinirlas */
:root {
  --bs-primary: #123c69;           /* azul marino corporativo */
  --bs-primary-rgb: 18, 60, 105;   /* el MISMO color en formato R, G, B */
}

/* 2 Componentes: los botones NO leen --bs-primary directamente,
   sino sus propias variables de componente (--bs-btn-*).
   Se sobrescriben en la clase del componente: */
.btn-primary {
  --bs-btn-bg: #123c69;
  --bs-btn-border-color: #123c69;
  --bs-btn-hover-bg: #0d2c4e;      /* un poco más oscuro para el hover */
  --bs-btn-hover-border-color: #0d2c4e;
  --bs-btn-active-bg: #0a2340;
  --bs-btn-active-border-color: #0a2340;
}

/* 3 Otro ejemplo: la navbar con el color corporativo */
.navbar.bg-primary {
  background-color: var(--bs-primary) !important;
}
```

Y lo enlazamos en el `<head>` de nuestras páginas, **debajo** del CSS de Bootstrap:

```
<!-- 1º Bootstrap (CDN) -->
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
      rel="stylesheet"
      integrity="sha384-sRI14kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuquIAvNWLzeN8tE5YBujZqJLB"
      crossorigin="anonymous">
<!-- 2º Nuestra personalización (el orden importa: lo último gana) -->
<link href="/css/arelance.css" rel="stylesheet">
```



¿Por qué no basta con cambiar `--bs-primary` para que cambien los botones? Porque los componentes de Bootstrap se **compilaron** con el azul original: cada componente lee sus propias variables (`--bs-btn-bg`, `--bs-alert-bg` ...), que ya traen el color "cocinado". Redefinir `:root` solo afecta a las **utilidades** que usan la variable en vivo (`text-`, `bg-`, `border-`). Para cambiarlo **TODO** de raíz, hay que recompilar con SASS (siguiente apartado).

Bonus: modo oscuro en una línea

Bootstrap 5.3 incluye **color modes**: añadiendo `data-bs-theme="dark"` al `<html>` toda la página pasa a tema oscuro, sin escribir CSS:

```
<!-- Tema oscuro para TODA la página -->  
<html lang="es" data-bs-theme="dark">  
  
<!-- ...o solo para un componente concreto -->  
<div class="card" data-bs-theme="dark">...</div>
```



4.12. SASS: instalación y compilación básica

Tabla de contenidos

Referencias oficiales:

- Sass · Bootstrap 5.3 → <https://getbootstrap.com/docs/5.3/customize/sass/>
- Options (variables de configuración) → <https://getbootstrap.com/docs/5.3/customize/options/>
- Instalar Sass → <https://sass-lang.com/install/>

SASS (*Syntactically Awesome Style Sheets*) es un **preprocesador de CSS**: un lenguaje que amplía CSS con variables, anidamiento, funciones e importaciones, y que se **compila** a CSS normal. Bootstrap está escrito íntegramente en SASS — el `bootstrap.min.css` que enlazamos por CDN es el resultado de compilar su código fuente `.scss` con los valores por defecto.

La idea: en lugar de "parchear" los colores con CSS (método A del apartado anterior), **sobrescribimos las variables SASS de Bootstrap y compilamos nuestra propia versión**. Cambiando `$primary` una sola vez, se regeneran botones, alerts, badges, enlaces, formularios... todo.

Paso 1 — Instalar SASS

El compilador oficial (*Dart Sass*) se instala con npm (necesitamos Node — apartado 5.1):

```
# Instalación global del compilador
npm install -g sass

# Verificamos
sass --version
```

Paso 2 — Preparar el proyecto de compilación

Creamos una carpeta de trabajo con Bootstrap como dependencia npm (su código fuente `.scss` viene incluido en el paquete):

```
mkdir -p ~/SpringBoot_Angular/arelance-theme/scss
cd ~/SpringBoot_Angular/arelance-theme
npm init -y
npm install bootstrap@5.3.8
# El código fuente SASS está en node_modules/bootstrap/scss/
ls node_modules/bootstrap/scss/ | head
```

Paso 3 — Crear nuestro archivo de personalización

Creamos `scss/arelance.scss`. La regla de oro: **nuestras variables van ANTES del import de Bootstrap** (SASS usa `!default` en sus variables: solo aplica el valor por defecto si nadie lo definió antes).

```
// scss/arelance.scss — Tema de la Academia Arelance

// 1 Sobrescribimos variables ANTES de importar Bootstrap
$primary: #123c69; // azul marino corporativo
$success: #1e7a4f; // verde corporativo
$danger: #b33939;
$border-radius: .75rem; // esquinas más redondeadas en TODOS los componentes
$font-family-sans-serif: Roboto, system-ui, sans-serif;

// 2 Importamos TODO Bootstrap (se compila con NUESTROS valores)
@import "../node_modules/bootstrap/scss/bootstrap";
```

Paso 4 – Compilar

```
cd ~/SpringBoot_Angular/arelance-theme

# Compilación única, minificada, directamente a static/ del backend
sass --style=compressed --no-source-map \
  scss/arelance.scss \
  ~/SpringBoot_Angular/crudalumnos/src/main/resources/static/css/bootstrap-arelance.css

# ...o en modo vigilancia: recompila automáticamente al guardar el .scss
sass --watch scss/arelance.scss \
  ~/SpringBoot_Angular/crudalumnos/src/main/resources/static/css/bootstrap-arelance.css
```

Y en el HTML, este archivo **sustituye por completo** al Bootstrap del CDN (es Bootstrap entero, ya personalizado):

```
<head>
<!-- Nuestro Bootstrap compilado con los colores de la academia -->
<link href="/css/bootstrap-arelance.css" rel="stylesheet">
</head>
```



Warning

Al compilar Bootstrap desde el código fuente es normal ver avisos de *deprecation* de SASS (Bootstrap 5.3 usa la sintaxis `@import`, que Dart Sass está retirando en favor de `@use`). Son **avisos, no errores**: el CSS se genera correctamente. Bootstrap 6 migrará a la sintaxis nueva.



Tip

¿Qué variables puedo tocar? Todas las de `node_modules/bootstrap/scss/_variables.scss` (¡son más de mil líneas!). Las más útiles: los colores del tema (`$primary`, `$secondary`, `$success`, `$danger`, `$warning`, `$info`), `$border-radius`, `$font-family-sans-serif` y `$enable-shadows`.



4.13. De Fetch a Angular: por qué dar el salto

[Tabla de contenidos](#)

Con lo que hemos construido en esta fase tenemos un frontend funcional y completo. Sin embargo, a medida que la aplicación crece, el enfoque vanilla empieza a mostrar sus limitaciones:

Problema con Fetch + HTML vanilla	Solución en Angular
El HTML se genera como cadenas de texto (<code>innerHTML</code>) — frágil y difícil de mantener	Plantillas HTML con binding declarativo (<code>{{ dato }}</code> , <code>@for</code>)
El estado (lista de alumnos, modal abierto, formulario) se gestiona a mano con variables globales	Signals y componentes con estado encapsulado
Cada petición <code>fetch</code> repite la misma estructura (método, headers, error)	<code>HttpClient</code> centraliza y tipifica todas las peticiones
Si la misma tabla de alumnos aparece en dos páginas, hay que duplicar el código	Componentes reutilizables
No hay routing — navegamos con <code></code> recargando la página completa	Router de Angular con navegación SPA sin recarga
No hay tipado — un error de nombre de campo no se detecta hasta ejecutar	TypeScript detecta errores en tiempo de desarrollo

En la siguiente fase veremos cómo Angular resuelve todos estos problemas de forma estructurada, usando exactamente los mismos endpoints del backend que hemos estado consumiendo aquí con `fetch()` .



Tip

El conocimiento de Fetch API que has adquirido en esta fase no se descarta al aprender Angular — `HttpClient` hace internamente lo mismo que `fetch()` , pero añade tipado, gestión reactiva con Observables y una capa de abstracción que hace el código más mantenible.

FASE 5: FRONT END con Angular y Bootstrap

[Tabla de contenidos](#)

Introducción a TypeScript

[Tabla de contenidos](#)

TypeScript es un **superconjunto de JavaScript**: todo el código JS válido es también TS válido. La diferencia principal es el **tipado estático**, que permite detectar errores en tiempo de compilación en lugar de en ejecución.

Como ya conocemos JavaScript con POO, solo necesitamos ver las tres características que usaremos constantemente en Angular.

Tipado estático

```
// JavaScript
let nombre = "Ana";
nombre = 42;           // JS no se queja → error silencioso

// TypeScript
let nombre: string = "Ana";
nombre = 42;           // ❌ Error en compilación → lo detectamos antes
```

Interfaces

Las interfaces son la herramienta clave en Angular para definir la forma de los datos. Son equivalentes a las clases de Java, pero solo definen estructura, sin lógica.

```
// Una interfaz define qué campos tiene un objeto y de qué tipo son
export interface Alumno {
  nif: string;
  nombre: string;
  edad: number;
  genero: boolean;
}

// Uso: TypeScript comprueba que el objeto cumple la interfaz
const alumno: Alumno = {
  nif: "12345678A",
  nombre: "Ana Garcia",
  edad: 22,
  genero: false
};
```

El símbolo `?` marca un campo como opcional:

```
export interface AlumnoAsignatura {
  id: number;
  asignatura: Asignatura;
  notaFinal?: number; // puede estar o no
}
```

Imports y exports

TypeScript organiza el código en módulos. Cada archivo exporta lo que define e importa lo que necesita:

```
// alumno.model.ts — exportamos la interfaz
export interface Alumno { ... }

// alumno-service.ts — la importamos donde la necesitamos
import { Alumno } from '../modelos/alumno.model';
```



Note

No necesitamos profundizar más en TypeScript para seguir el curso. El resto de características (decoradores, genéricos, tipos union) las iremos viendo de forma natural cuando aparezcan en el código Angular.



5.1. Instalar Angular

Tabla de contenidos

- Debemos instalar lo siguiente:
 - NVM (Node Version Manager) que permite gestionar y alternar entre distintas versiones de Node/Npm
 - Node
 - NPM
- Empezamos por instalar NVM (aquí viene el comando):
 - <https://github.com/nvm-sh/nvm>

```
cd ~/Descargas
wget -qO- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.5/install.sh | bash

# Reiniciamos el bash
source ~/.bashrc
# Vemos la versión de nvm
nvm --version # 0.40.5
```

- Ahora, usando NVM, instalamos la versión LTS de NodeJS

```
cd ~/Descargas
nvm install --lts
nvm use --lts

# NOTA IMPORTANTE: Si en el nvm install da problemas, puede ser porque tengamos instalado curl por snap.
# Si ese es el caso, copiar la salida a la IA, desinstalarlo e instalar con sudo apt install curl

nvm alias default lts/*
# Para ver las versiones de NodeJS y NPM
nvm ls

# Instalar globalmente Angular21
npm install -g @angular/cli@21

# Verificar
ng version
```

- Las versiones que usaremos serán:
 - NVM -> 0.40.5
 - NodeJS -> 24.16.0
 - NPM -> 11.13.0
 - Angular 21.2.15
- Por último, y no menos importante, es MUY RECOMENDABLE instalar algunas extensiones para Angular y Bootstrap en VSC. Son las siguientes:
 - Angular Language Service (Angular)
 - Angular Snippets 18 (John Papa)
 - Bootstrap 5 Quick Snippets (Anbuselvan Rocky)



5.2. Crear proyecto FrontEnd

Tabla de contenidos

```
# Nos situamos en la raíz del repositorio
cd ~/SpringBoot_Angular/
# Dentro tendremos crudalumnos (Backend)
ng new frontendalumnos --style=css
# NOTA: ya no hace falta --routing (el enrutado viene incluido desde Angular 17)
# NOTA2: la convención Angular es kebab-case (con guion), pero mantendremos frontendalumnos

# ? Do you want to enable Server-Side Rendering (SSR)...? No
# ? Which AI tools do you want to configure...? None
cd frontendalumnos
# Instalamos Bootstrap
npm install bootstrap@5.3.8
```



Note

Angular 21 crea los proyectos nuevos como **zoneless** (sin la librería `zone.js`): la detección de cambios se apoya en signals, que es justo lo que usaremos en todo el curso. No hay que configurar nada.

- Y Añadimos Bootstrap en angular.json
 - En ~/SpringBoot_Angular/frontendalumnos/angular.json



Hay que tener cuidado donde se pone el styles y el scripts. No se ponen al final del JSON (en test). Se ponen en projects -> frontendalumnos -> architect -> build -> options

```
"styles": [
  "node_modules/bootstrap/dist/css/bootstrap.min.css",
  "src/styles.css"
],
"scripts": [
  "node_modules/bootstrap/dist/js/bootstrap.bundle.min.js"
]
```

- La estructura general del proyecto Frontend será:

```
src/
├─ app/
│  ├─ modelos/
│  │  ├─ alumno.model.ts
│  │  └─ asignatura.model.ts
│  ├─ servicios/
│  │  ├─ alumno-service.ts
│  │  └─ asignatura-service.ts
│  ├─ componentes/
│  │  ├─ alumno-lista/
│  │  ├─ asignatura-lista/
│  │  ├─ dashboard/
│  │  └─ matriculas/
│  ├─ app.routes.ts      ← rutas standalone
│  ├─ app.config.ts      ← configuración (providers, router)
│  ├─ app.ts             ← componente raíz (clase App)
│  ├─ app.html            ← plantilla del componente raíz
│  └─ app.css             ← estilos del componente raíz
```



Nomenclatura de Angular 21 (guía de estilo oficial desde la v20): los generadores del CLI ya **no añaden el sufijo de tipo**. `ng new` crea `app.ts` (clase `App`) en vez de `app.component.ts`, y `ng g c alumno-lista` genera `alumno-lista.ts` con la clase `AlumnoLista` — sin `.component` ni `Component`. Si ves tutoriales con `user.component.ts` / `UserComponent`, es el estilo

antiguo (se puede recuperar con `"type": "component"` en los *schematics* de `angular.json`, pero seguiremos el estilo nuevo). Para los servicios que creamos a mano usaremos el archivo `alumno-service.ts` (clase `AlumnoService`): así no choca con el modelo `alumno.model.ts`.



Este punto de la documentación es esencial.

A partir de ahora SIEMPRE tendremos que tener abierto el servidor Angular y Swagger.

Ya podemos iniciar el servidor de Angular:

```
cd ~/SpringBoot_Angular/frontendalumnos
ng serve
# Would you like to share pseudonymous usage data... No
# Y probamos en el navegador: http://localhost:4200/
```

Y por si lo hemos olvidado, iniciamos SpringBoot:

```
cd ~/SpringBoot_Angular/crudalumnos
./mvnw clean spring-boot:run
# Recomiendo iniciar Swagger!
# http://localhost:8080/swagger-ui/index.html
```



5.3. El MVC para Alumnos y Asignaturas

Tabla de contenidos

Ya tenemos toda la infraestructura montada e iniciada. Por un lado el backend SpringBoot que llama a una BBDD montada en MySQL y por otro lado un servidor Angular que nos mostrará los endpoints generados por el backend en un frontend que incluye Bootstrap.

Ahora vamos a ver el paradigma MVC en Angular:

- **Modelo:** Define interfaces para Asignaturas y Alumnos (como las clases de Java para SpringBoot)
- **Vista:** Presenta los datos en una página web usando componentes de Bootstrap
- **Controlador:** Son los componentes Angular (archivos `.ts`). Gestionan la lógica, llaman al servicio y actualizan la vista. En Angular 21 son "standalone" por defecto: cada componente declara sus propias dependencias sin necesidad de un módulo central. Tendremos menos archivos, el arranque será más rápido y las importaciones serán más claras. Hasta Angular 17 esto no era así, y el desarrollo era mucho más pesado.



5.3.1 Modelos en Angular

Tabla de contenidos

- Empezamos con el Modelo Asignatura (creamos la carpeta modelos)
 - `src/app/modelos/asignatura.model.ts`



Note

En lo sucesivo, para cada elemento de Angular (modelos, servicios, componentes) tendremos que crear su carpeta y archivo correspondiente manualmente.

```
// src/app/modelos/asignatura.model.ts
export interface Asignatura {
  id: number;
  denominacion: string;
  curso: number;
  // Aulas será un array de floats
  aulas: number[]; // List<Float> en backend → number[] en TS
}
```

- Seguimos con el Modelo Alumno-Asignatura
 - `src/app/modelos/alumno-asignatura.model.ts`

```
// src/app/modelos/alumno-asignatura.model.ts
// Lo primero es importar la otra interfaz
import { Asignatura } from "../asignatura.model";

export interface AlumnoAsignatura {
  id: number;
  // OJO! El segundo elemento es la interfaz (objeto)
  asignatura: Asignatura;
}
```

- Y terminamos con Alumno (es decir, al revés de lo definido en el Backend)
 - En `src/app/modelos/alumno.model.ts`

```
// src/app/modelos/alumno.model.ts
import { AlumnoAsignatura } from "./alumno-asignatura.model";

export interface Alumno {
  nif: string;
  nombre: string;
  edad: number;
  genero: boolean;
  // Array de interfaces (objetos)
  // Esto lo meteremos en el Accordion
  alumnoAsignaturas: AlumnoAsignatura[];
}
```




5.3.2 Servicios en Angular

Tabla de contenidos

Como ya hemos comentado previamente, Angular 21 es

`standalone` por defecto, permitiendo que se active HTTP en la configuración general de Angular.

Activamos el HTTP en `app.config.ts`

- En `~/SpringBoot_Angular/frontendalumnos/src/app/app.config.ts`

```
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import { routes } from './app.routes';

// Activamos HTTP: importamos...
import { provideHttpClient } from '@angular/common/http';

export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes),
    // Y añadimos a la aplicación el HTTP...
    provideHttpClient()
  ]
};
```

- Y ahora definimos el Servicio que nos permitirá llamar al endpoint
 - En `~/SpringBoot_Angular/frontendalumnos/src/app/servicios/alumno-service.ts`

```
// src/app/servicios/alumno-service.ts

// Importaciones básicas:
// a. Injectable -> Permite inyectar el servicio en otros componentes
// b. HttpClient -> peticiones HTTP (GET, POST, PUT, DELETE...).
// c. Observable -> Emitirá datos cuando estén listos (llegue el endpoint)
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

// El modelo
import { Alumno } from '../modelos/alumno.model';

@Injectable({ providedIn: 'root' })
export class AlumnoService {
  private baseUrl = "http://localhost:8080/api/alumnos";

  // Constructor por defecto
  constructor(private http: HttpClient) {}

  // Creamos el primer método getAlumnos
  // Devuelve array de objetos Alumno
  getAlumnos(): Observable<Alumno[]> {
    return this.http.get<Alumno[]>(`${this.baseUrl}/consultar`);
  }
}
```



5.3.3 Vista de Consulta en Angular

[Tabla de contenidos](#)

Para presentar los datos en la web necesitamos crear un componente que pasará los datos del servicio al HTML que incluye Bootstrap. Lo primero es generar el componente por consola:

```
cd ~/SpringBoot_Angular/ frontendalumnos/src/app/  
ng g c componentes/alumno-lista  
# Se generan alumno-lista.ts, alumno-lista.html, alumno-lista.css  
# y alumno-lista.spec.ts (SIN sufijo .component: estilo Angular 21)
```

Desarrollamos el componente que es el intermediario entre el servicio y la vista final (HTML/CSS)

- En ~/SpringBoot_Angular/frontendalumnos/src/app/componentes/alumno-lista/alumno-lista.ts

```
// src/app/componentes/alumno-lista/alumno-lista.ts
import { Component } from '@angular/core';

// Nuevas importaciones
import { OnInit } from '@angular/core';
import { signal } from '@angular/core';
import { CommonModule } from '@angular/common';

// Modelo y Servicio
import { Alumno } from '../../../modelos/alumno.model';
import { AlumnoService } from '../../../servicios/alumno-service';

@Component({
  // selector -> URL final de la vista
  selector: 'app-alumno-lista',
  // standalone ya no se escribe: es el valor por defecto desde Angular 19
  imports: [CommonModule], // Añadimos el CommonModule
  templateUrl: './alumno-lista.html',
  styleUrls: ['./alumno-lista.css']
})

// Implementamos la interfaz OnInit
export class AlumnoLista implements OnInit {

  // Definimos los objetos a devolver
  // tipo a devolver | valores por defecto
  alumnos = signal<Alumno[] | null>(null);
  error = signal<string | null>(null); // Si hay error
  cargando = signal<boolean>(false);

  // El constructor
  constructor(private alumnoServ: AlumnoService){}

  // Nos obliga a meter el método ngOnInit
  ngOnInit(): void {
    this.cargando.set(true);
    this.alumnoServ.getAlumnos().subscribe({
      next: (data) => {
        this.alumnos.set(data);
        this.cargando.set(false);
      },
      error: (e) => {
        this.error.set('No se pudieron cargar los alumnos');
        this.cargando.set(false);
      }
    });
  }

  // Usamos la función trackBy para asociar la PK
  trackByNif(i: number, a: Alumno) {
    return a.nif;
  }
}
```

Y ya solo nos falta la presentación en si:

- En ~/SpringBoot_Angular/frontendalumnos/src/app/componentes/alumno-lista/alumno-lista.html

⚠ Las directivas estructurales clásicas (*ngIf, *ngFor...) están oficialmente **deprecadas desde Angular 20** y se eliminarán en una versión futura. En este primer ejemplo las pongo para que las reconozcas en código antiguo, pero en el apartado 5.5 usaré los nuevos formatos (@if / @for).

```

<!-- frontendalumnos/src/app/componentes/alumno-lista/alumno-lista.html-->

<main class="container my-4">
  <header class="mb-3">
    <h2>Listado de alumnos</h2>
  </header>

  <!-- *ngIf (if) -> https://angular.dev/api/common/NgIf -->
  <p *ngIf="cargando()">Cargando...</p>
  <p *ngIf="error()" class="alert alert-danger">{{ error() }}</p>

  <table class="table table-striped align-middle" *ngIf="alumnos() as lista">
    <thead class="table-light">
      <tr>
        <th scope="col">NIF</th>
        <th scope="col">Nombre</th>
        <th scope="col">Edad</th>
        <th scope="col">Género</th>
        <th scope="col">Asignaturas</th>
      </tr>
    </thead>
    <tbody>
      <!-- *ngIf (For) -> https://angular.dev/api/common/NgFor -->
      <tr *ngFor="let a of lista; trackBy: trackByNif">
        <td>{{ a.nif }}</td>
        <td>{{ a.nombre }}</td>
        <td>{{ a.edad }}</td>
        <td>{{ a.genero ? 'F' : 'M' }}</td>
        <td>
          <!-- Accordion -->
          <section class="accordion" [id]=" 'acc-' + a.nif">
            <article class="accordion-item">
              <header class="accordion-header" [id]=" 'h-' + a.nif">
                <button class="accordion-button collapsed" type="button"
                  data-bs-toggle="collapse"
                  [attr.data-bs-target]=" '#c-' + a.nif"
                  aria-expanded="false"
                  [attr.aria-controls]=" 'c-' + a.nif">
                  Ver asignaturas ({{ a.alumnoAsignaturas.length || 0 }})
                </button>
              </header>
              <section [id]=" 'c-' + a.nif" class="accordion-collapse collapse"
                [attr.aria-labelledby]=" 'h-' + a.nif"
                [attr.data-bs-parent]=" '#acc-' + a.nif">
                <section class="accordion-body p-0">
                  <table class="table mb-0">
                    <thead>
                      <tr>
                        <th scope="col">#ID</th>
                        <th scope="col">Denominación</th>
                        <th scope="col">Curso</th>
                        <th scope="col">Aulas</th>
                      </tr>
                    </thead>
                    <tbody>
                      <tr *ngFor="let rel of a.alumnoAsignaturas">
                        <td>{{ rel.asignatura.id }}</td>
                        <td>{{ rel.asignatura.denominacion }}</td>
                        <td>{{ rel.asignatura.curso }}</td>
                        <td>
                          <span *ngIf="rel.asignatura.aulas?.length; else sinAulas">
                            {{ rel.asignatura.aulas.join(', ') }}
                          </span>
                          <ng-template #sinAulas--></ng-template>
                        </td>
                      </tr>
                    </tbody>
                  </table>
                </section>
              </section>
            </article>
          </section>
        </td>
      </tr>
    </tbody>
  </table>

```

```

        </table>
      </section>
    </section>
  </article>
</section>
</td>
</tr>
</tbody>
</table>
</main>

```

- Y las rutas
 - En ~/SpringBoot_Angular/frontendalumnos/src/app/app.routes.ts

```

import { Routes } from '@angular/router';
// Importamos el componente
import { AlumnoLista } from '../componentes/alumno-lista/alumno-lista';

export const routes: Routes = [
  {
    path: '',
    redirectTo: 'alumnos',
    pathMatch: 'full'
  },
  {
    path: 'alumnos',
    component: AlumnoLista
  }
];

```

Un último detalle: `ng new` deja en `app.html` una plantilla de bienvenida enorme. La sustituimos ENTERA por el `<router-outlet />`, que es donde el router pintará cada componente (`app.ts` ya lo importa por defecto):

- En ~/SpringBoot_Angular/frontendalumnos/src/app/app.html

```

<!-- src/app/app.html -->
<router-outlet />

```

Vale, y ahora como lo probamos?

Debemos configurar CORS...



5.4 CORS

Tabla de contenidos

CORS (Cross-Origin Resource Sharing, Compartición de Recursos de Orígenes Cruzados), es un mecanismo de seguridad del navegador que controla si una página web (frontend) puede hacer peticiones a un servidor (backend) que está en un dominio diferente. Aunque pensemos que el dominio es el mismo (estamos en la misma máquina), en realidad esto NO es así, pues el puerto forma parte del origen.

Así tendremos:

- Backend SpringBoot → <http://localhost:8080>
- Frontend Angular → <http://localhost:4200>

Cuando Angular hace una llamada como:

- `this.http.get("http://localhost:8080/api/alumnos/consultar")`
el navegador primero envía una petición preliminar (OPTIONS, llamada preflight) para preguntar al backend:
"¿Aceptas que este origen (<http://localhost:4200>) te pida datos?"
- Si el backend responde con cabeceras CORS correctas (Access-Control-Allow-Origin, etc.), el navegador deja pasar la petición real (GET).
- Si no las responde → el navegador bloquea la respuesta aunque el backend haya procesado la solicitud.

Debemos volver a SpringBoot.

Para empezar debemos añadir una nueva dependencia al pom, el spring-boot-starter-security:

- En crudalumnos/pom.xml

```
<dependency>      <!-- Dependencia6: Swagger -->
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.8.9</version>
</dependency>
<dependency>      <!-- Dependencia7: Spring Security -->
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
  </dependency>
</dependencies>
```

- Nos vamos a ~/SpringBoot_Angular/crudalumnos/src/main/java/com/inserta/crudalumnos
- Nos creamos el paquete config y dentro la clase SecurityConfig.java
 - ~/SpringBoot_Angular/crudalumnos/src/main/java/com/inserta/crudalumnos/config/SecurityConfig.java

```
// src/main/java/com/inserta/crudalumnos/config/SecurityConfig.java
package com.inserta.crudalumnos.config;

import java.util.List;

// Las importaciones saldrán solas...
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

// Metemos la anotación Configuration
@Configuration
public class SecurityConfig {

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            // 🛡️ CSRF deshabilitado para permitir pruebas y peticiones REST desde Angular
            .csrf(csrf -> csrf.disable())

            // 🌐 Activamos CORS con la configuración que definimos abajo
            .cors(Customizer.withDefaults())

            // 🔑 Configuraciones de autorización
            // Lo retocaremos cuando veamos JWT
            .authorizeHttpRequests(
                auth -> auth
                    // ✅ Swagger/OpenAPI
                    .requestMatchers(
                        "/v3/api-docs/**",
                        "/swagger-ui/**",
                        "/swagger-ui.html")
                    .permitAll()
                    // API Angular
                    .requestMatchers(
                        "/api/**")
                    .permitAll()
                    .anyRequest().permitAll());
            return http.build();
    }

    @Bean
    CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration cfg = new CorsConfiguration();

        // 🌐 Permitimos Angular en local
        cfg.setAllowedOriginPatterns(
            List.of("http://localhost:4200", "http://127.0.0.1:4200"));
        // Métodos HTTP que Angular puede usar
        // OPTIONS es una petición previa del navegador para métodos "complejos"
        // como PUT (Actualizar) y DELETE (borrar). Hay que añadirlo...
        cfg.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));

        // Cabeceras permitidas
        cfg.setAllowedHeaders(List.of("*"));
        // Permitir credenciales (cookies, auth headers)
        cfg.setAllowCredentials(true);

        // Aplicar configuración a todas las rutas
        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", cfg);
    }
}
```

```
        return source;
    }
}
```

Y ahora si, debería funcionar:

Para empezar, paramos Angular y SpringBoot (que a lo mejor tenemos iniciados en sendas pestañas de la consola):

CTRL + C

Luego hacemos lo siguiente:

- Iniciamos el Backend...

```
cd ~/SpringBoot_Angular/crudalumnos
# Con el cambio en la seguridad, es recomendable hacer un install
./mvnw clean install
# Iniciamos el servidor SpringBoot
./mvnw clean spring-boot:run
```

- Iniciamos el FrontEnd...

```
cd ~/SpringBoot_Angular/frontendalumnos
ng serve
# Y probamos en el navegador:
# http://localhost:4200/alumnos
# http://localhost:8080/swagger-ui/index.html
```




5.5 Resto de Endpoints en Angular

[Tabla de contenidos](#)

Ahora vamos a definir, en la misma página (/alumnos) el resto de opciones del CRUD. En esta parte NO tocaremos SpringBoot, puesto que ya tenemos los endpoints necesarios. El modelo alumnos en Angular se mantendrá igual.



5.5.1. Ampliación del Servicio

[Tabla de contenidos](#)

Lo primero que debemos hacer es ampliar el servicio:

- En ~/SpringBoot_Angular/frontendalumnos/src/app/servicios/alumno-service.ts

```

// src/app/servicios/alumno-service.ts
// Importaciones previas:

export class AlumnoService {
  // Código inicial de la clase...

  getAlumnos(): Observable<Alumno[]> {
    return this.http.get<Alumno[]>(`${this.baseUrl}/consultar`);
  }

  // Resto de métodos para trabajar con el Backend
  // 🕒 Por NIF, que se pasa por parámetro
  // encodeURIComponent -> se usa para escapar caracteres especiales
  // No hace falta usarlo para números.
  getPorNIF(nif: string): Observable<Alumno> {
    return this.http.get<Alumno>(
      `${this.baseUrl}/consultar/${encodeURIComponent(nif)}`
    );
  }

  // + Crear (vía parámetros en URL, como en el backend)
  // OJO! 'nif' | 'nombre' | es UNION, no un OR. Significa:
  // "Dame un tipo con solo las propiedades de Alumno cuyos nombres están en esta unión de claves."
  // Pick sirve para tipar exactamente qué propiedades esperamos
  // en cada operación, evitando sobrecarga de parámetros y entradas "ruidosas".
  crearAlumno(alumno: Pick<Alumno, 'nif' | 'nombre' | 'edad' | 'genero'>): Observable<Alumno> {
    const { nif, edad, genero, nombre } = alumno;
    // OJO al ORDEN: el backend espera /crear/{nif}/{nombre}/{edad}/{genero}
    return this.http.post<Alumno>(
      `${this.baseUrl}/crear/${encodeURIComponent(nif)}/${encodeURIComponent(nombre)}/${edad}/${genero}`,
      null
    );
  }

  // 🕒 Actualizar (por NIF original en ruta)
  // OJO! el nif (que es por donde buscamos el alumno para actualizar)
  // va aparte de los atributos del alumno
  actualizarAlumno(nif: string,
    alumno: Pick<Alumno, 'nombre' | 'edad' | 'genero'>): Observable<Alumno> {
    const { edad, genero, nombre } = alumno;
    // OJO al ORDEN: el backend espera /actualizar/{nif}/{nombre}/{edad}/{genero}
    return this.http.put<Alumno>(
      `${this.baseUrl}/actualizar/${encodeURIComponent(nif)}/${encodeURIComponent(nombre)}/${edad}/${genero}`,
      null
    );
  }

  // ✖ Borrar por NIF
  borrarAlumno(nif: string): Observable<void> {
    return this.http.delete<void>(`${this.baseUrl}/borrar/${encodeURIComponent(nif)}`);
  }
}

```



5.5.2. Ampliación de la vista

[Tabla de contenidos](#)

Ampliamos el componente (intermediario entre el servicio y la vista final, HTML/CSS).

- En ~/SpringBoot_Angular/frontendalumnos/src/app/componentes/alumno-lista/alumno-lista.ts

```

//src/app/componentes/alumno-lista/alumno-lista.ts
import { Component } from '@angular/core';

// Nuevas importaciones
import { OnInit } from '@angular/core';
import { signal } from '@angular/core';
import { CommonModule } from '@angular/common';

// Modelo y Servicio
import { Alumno } from '../../../modelos/alumno.model';
import { AlumnoService } from '../../../servicios/alumno-service';

// [NUEVO] Nuevas importaciones
import { inject, ViewChild, ElementRef } from '@angular/core';
import { ReactiveFormsModule, FormBuilder, Validators } from '@angular/forms';
import { finalize } from 'rxjs';

@Component({
  // selector -> URL final de la vista
  selector: 'app-alumno-lista',
  // standalone ya no se escribe: es el valor por defecto desde Angular 19
  // Con el control flow @if/@for NO es necesario CommonModule, pero se puede dejar sin problema.
  imports: [CommonModule, ReactiveFormsModule], // Añadimos ReactiveFormsModule
  templateUrl: './alumno-lista.html',
  styleUrls: ['./alumno-lista.css'],
})

// Implementamos la interfaz OnInit
export class AlumnoLista implements OnInit {
  // Definimos los objetos a devolver
  // tipo a devolver | valores por defecto
  alumnos = signal<Alumno[]>([]);
  error = signal<string | null>(null); // Si hay error
  cargando = signal<boolean>(false);

  // [NUEVO] Atributos nuevos (inject va privado)
  // Señal opcional para bloquear el botón de enviar durante POST/PUT
  guardando = signal<boolean>(false);
  private fb = inject(FormBuilder);
  modoEdicion = signal<boolean>(false); // true cuando estamos actualizando
  nifOriginal: string | null = null; // guarda NIF del registro cargado para edición

  // [NUEVO] Referencia al <dialog> y modales de BS en la vista:
  // <dialog #dialogoConfirmarBS>...</dialog>
  @ViewChild('dialogoConfirmarBS', { static: false })
  dialogoConfirmarBS!: ElementRef<HTMLDialogElement>;
  // Modales de Bootstrap como alternativa...
  @ViewChild('abrirModalBS', { static: false })
  abrirModalBS!: ElementRef<HTMLButtonElement>;
  @ViewChild('cerrarModalBS', { static: false })
  cerrarModalBS!: ElementRef<HTMLButtonElement>;

  pendienteEliminar = signal<Alumno | null>(null); // alumno seleccionado para eliminar
  eliminando = signal<boolean>(false); // true mientras se hace DELETE

  // [NUEVO] Formulario reactivo
  // Importante: 'genero' se maneja como string 'true'/'false' en el <select>.
  form = this.fb.group({
    nif: ['', [Validators.required, Validators.minLength(2)]],
    nombre: ['', [Validators.required, Validators.minLength(2)]],
    edad: [18, [Validators.required, Validators.min(0), Validators.max(120)]],
    genero: ['true', [Validators.required]], // por defecto 'true' (femenino)
  });

  // El constructor (con inyección)
  constructor(private alumnoServ: AlumnoService) {}

```

```

// La interfaz nos obliga a meter el método ngOnInit (Carga inicial)
// IMPORTANTE: el método subscribe de la interfaz Observer de RxJS
// incluye dos métodos: next y error. Los nombres NO se pueden cambiar
ngOnInit(): void {
  this.cargando.set(true);
  this.alumnoServ.getAlumnos().pipe(
    // finalize se ejecuta SIEMPRE cuando termina la suscripción (complete/error/unsubscribe)
    finalize(() => this.cargando.set(false))).subscribe({
      next: (data) => {
        this.alumnos.set(data);
      },
      error: (e) => {
        this.error.set('No se pudieron cargar los alumnos');
      },
    });
}

// Usamos la función verPorNif para asociar la PK (Utilidades)
verPorNif(a: Alumno): string {
  return a.nif;
}

// [NUEVO] Métodos adicionales
// -----
// Para resetear con valores por defecto el formulario
restaurarForm(): void {
  this.form.reset({ nif: '', nombre: '', edad: 18, genero: 'true' });
  this.modaEdicion.set(false);
  this.nifOriginal = null;
}

// Para editar un registro, usamos el mismo formulario
// Y cargamos los valores del alumno seleccionado
editar(a: Alumno): void {
  this.form.setValue({
    nif: a.nif,
    nombre: a.nombre,
    edad: a.edad,
    genero: a.genero ? 'true' : 'false',
  });
  this.nifOriginal = a.nif;
  this.modaEdicion.set(true);
}

// Me echo para atrás en el borrado (y limpia estado de eliminación)
cancelarEliminar(): void {
  const dlg = this.dialogoConfirmarBS?.nativeElement; // Objeto dialog
  this.eliminando.set(false); // NO elimino!
  this.pendienteEliminar.set(null);
  dlg?.close(); // Cerramos dialog
  // Si hay BS, cerramos el modal
  if(this.cerrarModalBS)
    this.cerrarModalBS.nativeElement.click();
}

// Ahora si, ELIMINO el alumno! (y actualiza la lista en memoria)
eliminarConfirmado(): void {
  const a = this.pendienteEliminar();
  if (!a)
    return;

  this.eliminando.set(true);
  this.alumnoServ.borrarAlumno(a.nif).pipe(
    // Garantiza que 'eliminando' vuelve a false aunque falle la petición
    finalize(() => this.eliminando.set(false))).subscribe({
      next: () => {

```

```

// ✅ Con signal no-nulo, podemos usar update + filter directamente
this.alumnos.update(
  alumnosActuales => alumnosActuales.filter(x => x.nif !== a.nif));

// Si estábamos editando justo este alumno, resetea el formulario
if (this.modaEdicion() && this.nifOriginal === a.nif)
  this.restaurarForm();

this.pendienteEliminar.set(null);
// 📌 Cierre del modal de Bootstrap sin usar su API
if (this.cerrarModalBS)
  this.cerrarModalBS.nativeElement.click();

// Si tenemos también <dialog>, por si estaba abierto:
this.dialogoConfirmarBS?.nativeElement.close();
},
error: () => {
  this.error.set('No se pudo eliminar el alumno');
  // Deja el diálogo abierto para que el usuario vea el error o reintente
},
});
}

// Para eliminar, preguntaré antes
// 🗨 Mostrar diálogo nativo para confirmar borrado
eliminar(a: Alumno): void {
  this.error.set(null);
  this.pendienteEliminar.set(a);

  // 1) Si está cargado el JS de Bootstrap, abrimos el modal con el botón oculto
  const tieneBS = typeof (window as any).bootstrap !== 'undefined';
  if (tieneBS && this.abrirModalBS) {
    this.abrirModalBS.nativeElement.click(); // abre #confirmBsModal
    return;
  }

  // 2) Si no hay Bootstrap, usamos <dialog> nativo si está disponible
  const dlg = this.dialogoConfirmarBS?.nativeElement;
  if (dlg && typeof dlg.showModal === 'function') {
    dlg.showModal();
    return;
  }

  // 3) Última opción: window.confirm (plan C)
  if (window.confirm(`¿Eliminar al alumno ${a.nombre} (${a.nif})?`)) {
    this.eliminarConfirmado();
  } else {
    this.cancelarEliminar();
  }
}

// Por último, el submit normal para crear/editar
submit(): void {
  // Si no valida el formulario, o no se está guardando, adios!
  if (this.form.invalid || this.guardando())
    return;

  // Normaliza valores del formulario
  const nif = this.form.value.nif!.trim(); // Quito espacios al final
  const datosForm = {
    nombre: this.form.value.nombre!.trim(),
    edad: Number(this.form.value.edad), // Convierto a número
    genero: this.form.value.genero === 'true',
  };

  this.guardando.set(true);

```

```

// UPDATE: Si estamos en editar (no en crear)
if (this.modoEdicion()) {
  const nifObjetivo = this.nifOriginal ?? nif;
  this.alumnoServ.actualizarAlumno(nifObjetivo, datosForm).pipe(
    finalize(() => this.guardando.set(false))).subscribe({
      next: (alumnoActualizado) => {
        this.alumnos.update(listadoAlumnos =>
          listadoAlumnos.map(x => x.nif === nifObjetivo ? alumnoActualizado : x)
        );
        this.restaurarForm();
      },
      error: () =>
        this.error.set('No se pudo actualizar el alumno')
    });
} // CREATE: SI no es editar, es para CREAR (POST)
else {
  this.alumnoServ.crearAlumno({ nif, ...datosForm }).pipe(
    finalize(() => this.guardando.set(false))).subscribe({
      next: (alumnoCreado) => {
        this.alumnos.update(alumnosActuales => [alumnoCreado, ...alumnosActuales]);
        this.restaurarForm();
      },
      error: () =>
        this.error.set('No se pudo crear el alumno (¿NIF duplicado?)')
    });
}
}

// Método especial para conseguir que el accordion de la vista funcione abrir/cerrar
// 3 días que me he pasado para conseguirlo!!
toggleAcordeon(i: number, ev?: Event): void {
  const id = `collapse-${i}`; // id distintos para cada alumno
  const el = document.getElementById(id); // Saco el acordeon completo
  if (!el)
    return;

  // Si hay Bootstrap (bundle) usa su API
  const bs = (window as any).bootstrap;
  if (bs?.Collapse) { // Si la propiedad está Collapse -> CIERRO!
    const instancia = bs.Collapse.getOrCreateInstance(el, { toggle: false });
    instancia.toggle();
  } else { // En caso contrario, ABRO!
    // Degradación: alterna la clase "show"
    el.classList.toggle('show');
  }

  // Actualiza el botón (aria-expanded + clase collapsed) para accesibilidad/estilos
  const boton = ev?.currentTarget as HTMLButtonElement | undefined;
  if (boton) {
    const abierto = el.classList.contains('show');
    boton.setAttribute('aria-expanded', String(abierto));
    boton.classList.toggle('collapsed', !abierto);
  }
}
}

```

Y por último, realizamos varios añadidos en la vista final HTML.



Note

Pongo el código completo, al igual que el componente, por claridad.

- En ~/SpringBoot_Angular/frontendalumnos/src/app/componentes/alumno-lista/alumno-lista.html


```

<!-- src/app/componentes/alumno-lista/alumno-lista.html -->
<main class="container my-4">
  <header class="mb-3">
    <h2>Alumnado</h2>
    <p class="text-muted m-0">Listado, alta y edición en una sola pantalla.</p>
  </header>

  <!-- Formulario crear/editar -->
  <form (ngSubmit)="submit()" [formGroup]="form" class="card mb-4">
    <section class="card-body">
      <fieldset class="row g-3">
        <section class="col-12 col-md-3">
          <label class="form-label" for="nif">NIF</label>
          <input id="nif" type="text" class="form-control" formControlName="nif" [readonly]="modoEdicion()">
          @if (modoEdicion()) {
            <small class="form-text">Editando registro existente.</small>
          }
        </section>

        <section class="col-12 col-md-4">
          <label class="form-label" for="nombre">Nombre</label>
          <input id="nombre" type="text" class="form-control" formControlName="nombre">
        </section>

        <section class="col-6 col-md-2">
          <label class="form-label" for="edad">Edad</label>
          <input id="edad" type="number" min="0" max="120" class="form-control" formControlName="edad">
        </section>

        <section class="col-6 col-md-3">
          <label class="form-label" for="genero">Género</label>
          <select id="genero" class="form-select" formControlName="genero">
            <option value="false">Masculino</option>
            <option value="true">Femenino</option>
          </select>
        </section>
      </fieldset>

      <section class="mt-3 d-flex gap-2">
        <button class="btn btn-primary" type="submit" [disabled]="form.invalid || guardando()">
          {{ modoEdicion() ? 'Actualizar' : 'Crear' }}
        </button>
        @if (modoEdicion()) {
          <button class="btn btn-secondary" type="button" (click)="restaurarForm()" [disabled]="guardando()">Cancelar</button>
        }
      </section>
    </section>
  </form>

  <!-- Mensajes globales -->
  @if (cargando()) { <p class="alert alert-info">Cargando...</p> }
  @if (error()) { <p class="alert alert-danger">{{ error() }}</p> }

  <!-- Tabla si hay alumnos -->
  @if (alumnos().length > 0) {
    <section class="table-responsive">
      <table class="table table-striped align-middle">
        <thead class="table-light">
          <tr>
            <th scope="col">NIF</th>
            <th scope="col">Nombre</th>
            <th scope="col">Edad</th>
            <th scope="col">Género</th>
            <th scope="col">Asignaturas</th>
            <th scope="col" class="text-start">Acciones</th>
          </tr>
        </thead>

```

```

<tbody>
  @for (a of alumnos(); track verPorNif(a); let i = $index) {
    <tr>
      <td><code>{{ a.nif }}</code></td>
      <td>{{ a.nombre }}</td>
      <td>{{ a.edad }}</td>
      <td>{{ a.genero ? 'F' : 'M' }}</td>

      <td><!-- OJO: el accordion SIEMPRE dentro de una celda <td> -->
      <!-- Parte del accordion -->
      <!-- ACCORDION Bootstrap con control por TS: toggleAcordeon(i, $event) -->
      <!-- (click) al pulsar, llamo al método del TS -->
      <section class="accordion" [id]='acc-' + i">
        <article class="accordion-item">
          <h2 class="accordion-header" [id]='heading-' + i">
            <button
              class="accordion-button collapsed text-primary"
              type="button"
              (click)="toggleAcordeon(i, $event)"
              [attr.aria-controls]='collapse-' + i"
              aria-expanded="false">
                Ver asignaturas ({{ a.alumnoAsignaturas.length }})
            </button>
          </h2>

          <!-- SIN data-bs-parent para poder cerrar el único panel -->
          <section
            class="accordion-collapse collapse"
            [id]='collapse-' + i"
            [attr.aria-labelledby]='heading-' + i">
            <section class="accordion-body p-0">
              <table class="table mb-0">
                <thead>
                  <tr>
                    <th scope="col">#ID</th>
                    <th scope="col">Denominación</th>
                    <th scope="col">Curso</th>
                    <th scope="col">Aulas</th>
                  </tr>
                </thead>
                <tbody>
                  @for (rel of a.alumnoAsignaturas; track rel.asignatura.id) {
                    <tr>
                      <td>{{ rel.asignatura.id }}</td>
                      <td>{{ rel.asignatura.denominacion }}</td>
                      <td>{{ rel.asignatura.curso }}</td>
                      <td>
                        @if (rel.asignatura.aulas.length) {
                          {{ rel.asignatura.aulas.join(', ') }}
                        } @else { - }
                      </td>
                    </tr>
                  }
                </tbody>
              </table>
            </section>
          </article>
        </section>

        <!-- Fin Parte del accordion -->
      </td>

      <td class="text-start">
        <button class="btn btn-sm btn-outline-primary me-2" (click)="editar(a)">Editar</button>
        <button class="btn btn-sm btn-outline-danger" (click)="eliminar(a)">Eliminar</button>
      </td>
    </tr>
  }

```

```

        </td>
      </tr>
    }
  </tbody>
</table>
</section>
}

<!-- Estado vacío -->
@if (!cargando() && alumnos().length === 0) {
  <p class="text-center text-muted my-3">No hay alumnos.</p>
}

<!-- === Botones ocultos para el modal de Bootstrap === -->
<button #abrirModalBS type="button" class="d-none"
  data-bs-toggle="modal" data-bs-target="#confirmBsModal"></button>

<button #cerrarModalBS type="button" class="d-none"
  data-bs-dismiss="modal"></button>

<!-- === Modal Bootstrap de confirmación === -->
<section class="modal fade" id="confirmBsModal" tabindex="-1" aria-labelledby="confirmBsModalLabel" aria-hidden="true">
  <section class="modal-dialog modal-dialog-centered">
    <article class="modal-content">
      <header class="modal-header">
        <h5 class="modal-title" id="confirmBsModalLabel">Confirmar eliminación</h5>
        <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Cerrar" [disabled]="eliminando()"></button>
      </header>

      <section class="modal-body">
        @if (pendienteEliminar(); as alu) {
          ¿Eliminar a <strong>{{ alu.nombre }}</strong> (<code>{{ alu.nif }}</code>)?
        } @else {
          ¿Eliminar el alumno seleccionado?
        }
        @if (error()) {
          <p class="alert alert-danger mt-3 mb-0">{{ error() }}</p>
        }
      </section>

      <footer class="modal-footer">
        <button type="button" class="btn btn-secondary" data-bs-dismiss="modal" [disabled]="eliminando()">
          Cancelar
        </button>
        <button type="button" class="btn btn-danger" (click)="eliminarConfirmado()" [disabled]="eliminando()">
          @if (eliminando()) { Eliminando... } @else { Eliminar }
        </button>
      </footer>
    </article>
  </section>
</section>

<!-- === Fallback: <dialog> nativo (si no hay Bootstrap) === -->
<dialog #dialogoConfirmarBS aria-labelledby="titulo-dialogo-confirmacion">
  <form method="dialog" class="m-0">
    <h3 id="titulo-dialogo-confirmacion" class="h5 mb-3">Confirmar eliminación</h3>
    @if (pendienteEliminar(); as alu2) {
      <p>¿Eliminar a <strong>{{ alu2.nombre }}</strong> (<code>{{ alu2.nif }}</code>)?</p>
    } @else {
      <p>¿Eliminar el alumno seleccionado?</p>
    }
    @if (error()) {
      <p class="alert alert-danger">{{ error() }}</p>
    }
    <menu class="mt-3 d-flex gap-2 justify-content-end">
      <button class="btn btn-secondary" type="button" (click)="cancelarEliminar()" [disabled]="eliminando()">Cancelar</button>
      <button class="btn btn-danger" type="button" (click)="eliminarConfirmado()" [disabled]="eliminando()">

```

```
        @if (eliminando()) { Eliminando... } @else { Eliminar }  
      </button>  
    </menu>  
  </form>  
</dialog>  
</main>
```

FASE 6: AMPLIAR PROYECTO: Asignaturas

Tabla de contenidos

Ya tenemos definido el CRUD para alumnos, con una sección adicional para visualizar las asignaturas matriculadas para cada registro. Ahora vamos a definir el mismo CRUD para asignaturas, permitiendo el alta/baja en las matriculaciones y la asignación de aulas para cada asignatura.

Reutilizaremos mucho trabajo previo.

6.1: Backend para asignaturas

Tabla de contenidos

Empezamos por el repositorio y el controlador de matrículas... ¡que ya están hechos! En el apartado **3.5 (JOINS)** dejamos completos `AlumnoAsignaturaRepository` y `MatriculaController`. En esta fase los **reutilizamos tal cual** – no hay que crear ni tocar nada de matrículas. Como recordatorio, esto es lo que ya tenemos disponible:

Capa	Elemento	Qué hace
Repositorio	<code>existeMatricula(nif, id)</code>	¿Ya está matriculado? (JPQL con CASE)
Repositorio	<code>borrarMatricula(nif, id)</code>	Borra una matrícula (<code>@Modifying</code> + <code>@Transactional</code>)
Repositorio	<code>buscarPorAlumno(nif)</code>	Matrículas de un alumno
Repositorio	<code>buscarPorAsignatura(id)</code>	Matrículas de una asignatura
Repositorio	<code>contarPorAsignatura(id)</code>	Nº de alumnos de una asignatura
Repositorio	<code>buscarAlumnosPorAsignatura(denominacion)</code>	Alumnos de una asignatura (JOIN)
Controlador	<code>POST /api/matriculas/matricular/{nif}/{id}</code>	Matricular
Controlador	<code>DELETE /api/matriculas/desmatricular/{nif}/{id}</code>	Desmatricular
Controlador	<code>GET /api/matriculas/matriculados?denominacion=</code>	Alumnos de una asignatura

Lo único **nuevo** de esta fase en el backend es el CRUD de asignaturas.

Vamos con el nuevo controlador:

Un CRUD completo para Asignaturas...

- En ~/SpringBoot_Angular/crudalumnos/src/main/java/com/inserta/crudalumnos/controlador/AsignaturaController.java



El controlador AsignaturaController.java debe crearse en la carpeta controlador como New Java File > Class

```

// src/main/java/com/inserta/crudalumnos/controlador/AsignaturaController.java
package com.inserta.crudalumnos.controlador;

import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

// Importamos el modelo y su repositorio
import com.inserta.crudalumnos.modelo.Asignatura;
import com.inserta.crudalumnos.repositorio.AsignaturaRepository;

// Y las anotaciones para Swagger
import io.swagger.v3.oas.annotations.Operation;

/**
 * Controlador REST para gestionar ASIGNATURAS.
 */
* Endpoints:
* - GET    /api/asignaturas/consultar           → Listar todas
* - GET    /api/asignaturas/consultar/{id}      → Buscar por ID
* - POST   /api/asignaturas/crear/{den}/{curso} → Crear (sin JSON, den=denominacion)
* - PUT    /api/asignaturas/actualizar/{id}/{den}/{curso} → Actualizar (sin JSON)
* - DELETE /api/asignaturas/borrar/{id}         → Eliminar
* - PUT    /api/asignaturas/{id}/aulas?lista=1.1,2.2 → Definir aulas (CSV)
*/

@RestController
@RequestMapping("/api/asignaturas")
public class AsignaturaController {

    private final AsignaturaRepository asignaturaRepo;

    public AsignaturaController(AsignaturaRepository asignaturaRepo) {
        this.asignaturaRepo = asignaturaRepo;
    }

    // 🔍 GET - Listar todas las asignaturas
    @GetMapping("/consultar")
    @Operation(summary = "Listar todas las asignaturas")
    public List<Asignatura> verAsignaturas() {
        return asignaturaRepo.findAll();
    }

    // 🔍 GET - Buscar una asignatura por ID
    @GetMapping("/consultar/{id}")
    @Operation(summary = "Buscar una asignatura por ID")
    public ResponseEntity<Asignatura> verAsignaturaPorId(@PathVariable Integer id) {
        return asignaturaRepo.findById(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build()); // 404 si no existe
    }

    // ➕ POST - Crear asignatura (usando parámetros en ruta, sin JSON)
    @PostMapping("/crear/{denominacion}/{curso}")
    @Operation(summary = "Crear una asignatura (sin JSON)")
    public Asignatura
        crearAsignatura(@PathVariable String denominacion,
            @PathVariable Integer curso) {
        Asignatura a = new Asignatura();
        a.setDenominacion(denominacion);
        a.setCurso(curso);
        // List<Float> aulas se deja vacío; se puede definir luego vía /{id}/aulas
        return asignaturaRepo.save(a);
    }
}

```

```

// 🛠️ PUT - Actualizar asignatura (también sin JSON, para mantener la sencillez)
@PutMapping("/actualizar/{id}/{denominacion}/{curso}")
@Operation(summary = "Actualizar una asignatura (sin JSON)")
public ResponseEntity<Asignatura>
    actualizarAsignatura(
        @PathVariable Integer id,
        @PathVariable String denominacion,
        @PathVariable Integer curso) {
    return asignaturaRepo.findById(id)
        .map(a -> {
            a.setDenominacion(denominacion);
            a.setCurso(curso);
            return ResponseEntity.ok(asignaturaRepo.save(a));
        })
        .orElse(ResponseEntity.notFound().build()); // 404 si no existe
}

// ❌ DELETE - Eliminar asignatura por ID
@DeleteMapping("/borrar/{id}")
@Operation(summary = "Eliminar una asignatura por ID")
public ResponseEntity<Void> borrarAsignatura(@PathVariable Integer id) {
    if (!asignaturaRepo.existsById(id)) {
        return ResponseEntity.notFound().build(); // 404 si no existe
    }
    // Si usas ON DELETE CASCADE, borrará matrículas automáticamente.
    // Si no, asegúrate de borrar primero relaciones en el servicio/repositorio.
    asignaturaRepo.deleteById(id);
    return ResponseEntity.noContent().build(); // 204 OK sin contenido
}

// 🏠 PUT - Definir/actualizar aulas con un CSV en ?lista=1.1,2.2,3.3
@PutMapping("/{id}/aulas")
@Operation(summary = "Definir las aulas donde se imparte (CSV en query param ?lista=1.1,2.2)")
public ResponseEntity<?> definirAulas(@PathVariable Integer id,
    @RequestParam(name = "lista") String csv) {
    // Parseo robusto del CSV → List<Float>
    final List<Float> aulas;
    try {
        aulas = Arrays.stream(csv.split(","))
            .map(String::trim)
            .filter(s -> !s.isEmpty())
            .map(Float::parseFloat)
            .collect(Collectors.toList());
    } catch (NumberFormatException e) {
        return ResponseEntity.badRequest()
            .body("El formato de 'lista' es inválido. Usa comas y punto decimal. Ej: 1.1, 2.2");
    }

    return asignaturaRepo.findById(id)
        .map(a -> {
            a.getAulas().clear();
            a.getAulas().addAll(aulas);
            return ResponseEntity.ok(asignaturaRepo.save(a));
        })
        .orElse(ResponseEntity.notFound().build()); // 404 si no existe
}
}

```




6.2: Frontend para asignaturas

[Tabla de contenidos](#)

- Y ahora definimos el Servicio que nos permitirá llamar a los endpoints de asignaturas y que, como ya hemos visto previamente, es el intermediario entre el Backend y el componente (que luego crearemos).
 - En ~/SpringBoot_Angular/frontendalumnos/src/app/servicios/asignatura-service.ts

```

// src/app/servicios/asignatura-service.ts
import { Injectable } from "@angular/core";
import { HttpClient } from "@angular/common/http";
import { Observable } from "rxjs";

import { Asignatura } from "../modelos/asignatura.model";

/**
 * Servicio para llamar a los endpoints del controlador de Asignaturas.
 */
@Injectable({ providedIn: "root" })
export class AsignaturaService {
  private baseUrl = "http://localhost:8080/api/asignaturas";

  // Constructor por defecto
  constructor(private http: HttpClient) {}

  // 🔍 (GET) Listar todas las asignaturas (devuelve array)
  getAsignaturas(): Observable<Asignatura[]> {
    return this.http.get<Asignatura[]>({
      url: `${this.baseUrl}/consultar`,
    });
  }

  // 🔍 (GET) Buscar por ID
  getAsignaturaPorId(id: number): Observable<Asignatura> {
    return this.http.get<Asignatura>({
      url: `${this.baseUrl}/consultar/${id}`,
    });
  }

  // ➕ (POST) Crear (sin JSON, vía parámetros en ruta)
  // Pick<Asignatura, 'denominacion' | 'curso'> tipa exactamente lo que esperamos.
  // null, por si no devuelve nada el Backend
  crearAsignatura(asig: Pick<Asignatura, "denominacion" | "curso">):
    Observable<Asignatura> {
    const { denominacion, curso } = asig;
    return this.http.post<Asignatura>({
      url: `${this.baseUrl}/crear/${encodeURIComponent(denominacion)}/${curso}`,
      body: null,
    });
  }

  // 🖋️ (PUT) Actualizar (por ID en ruta, sin JSON)
  actualizarAsignatura(id: number,
    asig: Pick<Asignatura, "denominacion" | "curso">):
    Observable<Asignatura> {
    const { denominacion, curso } = asig;
    return this.http.put<Asignatura>({
      url: `${this.baseUrl}/actualizar/${id}/${encodeURIComponent(denominacion)}/${curso}`,
      body: null,
    });
  }

  // ✖️ (DELETE) Borrar por ID
  borrarAsignatura(id: number): Observable<void> {
    return this.http.delete<void>({
      url: `${this.baseUrl}/borrar/${id}`,
    });
  }

  // 🏠 (PUT) Definir aulas desde CSV en query param (?lista=1.1,2.2)
  definirAulasAsignaturas(id: number, csvAulas: string):
    Observable<Asignatura> {
    return this.http.put<Asignatura>({
      url: `${this.baseUrl}/${id}/aulas?lista=${encodeURIComponent(csvAulas)}`,
      body: null,
    });
  }
}

```

Ahora toca crear el componente para asignaturas y probar que getAsignaturas funciona correctamente. Por consola:

```
cd ~/SpringBoot_Angular/frontendalumnos/src/app  
ng g c componentes/asignatura-lista
```

Y ahora definimos el ts del componente:

- En ~/SpringBoot_Angular/frontendalumnos/src/app/componentes/asignatura-lista/asignatura-lista.ts

```

// src/app/componentes/asignatura-lista/asignatura-lista.ts
import { Component } from '@angular/core';

// Nuevas importaciones
import { OnInit, signal } from '@angular/core';

// Servicio y modelo
import { AsignaturaService } from '../../../servicios/asignatura-service';
import { Asignatura } from '../../../modelos/asignatura.model';

/**
 * Componente para mostrar el listado de Asignaturas
 * Paso inicial → solo consulta (GET) y listado en tabla.
 */
@Component({
  selector: 'app-asignatura-lista',
  // standalone ya no se escribe: es el valor por defecto desde Angular 19.
  // Tampoco hace falta CommonModule: usamos el control flow nativo @if/@for
  templateUrl: './asignatura-lista.html',
  styleUrls: ['./asignatura-lista.css']
})

export class AsignaturaLista implements OnInit {

  // Estado del componente con signals (como en alumnos)
  asignaturas = signal<Asignatura[]>([]); // listado de asignaturas
  error = signal<string | null>(null); // mensaje de error si ocurre
  cargando = signal<boolean>(false); // true mientras carga

  // Constructor con inyección del servicio
  constructor(private asigServ: AsignaturaService) {}

  // Al iniciar → llamada al servicio
  ngOnInit(): void {
    this.cargarAsignaturas(); // Mas abajo...
  }

  // Método para cargar todas las asignaturas desde el backend
  cargarAsignaturas(): void {
    this.cargando.set(true);
    this.error.set(null);

    this.asigServ.getAsignaturas().subscribe({
      next: (data) => {
        this.asignaturas.set(data);
        this.cargando.set(false);
      },
      error: (err) => {
        console.error('Error cargando asignaturas', err);
        this.error.set('No se pudieron cargar las asignaturas');
        this.cargando.set(false);
      }
    });
  }

  // NOTA: con el control flow @for ya no hace falta el método trackBy:
  // el seguimiento se declara en la propia plantilla con "track a.id"
}

```

La vista en HTML:

- En ~/SpringBoot_Angular/frontendalumnos/src/app/componentes/asignatura-lista/asignatura-lista.html

```

<!-- componentes/asignatura-lista/asignatura-lista.html -->
<main class="container my-4">
  <header class="mb-3">
    <h2>Listado de asignaturas</h2>
  </header>

  <!-- Mensajes de estado (control flow nativo @if, como en 5.5) -->
  @if (cargando()) { <p>Cargando asignaturas...</p> }
  @if (error()) { <p class="alert alert-danger">{{ error() }}</p> }

  <!-- Tabla de asignaturas -->
  @if (asignaturas().length > 0) {
    <table class="table table-striped align-middle">
      <thead class="table-light">
        <tr>
          <th>ID</th>
          <th>Denominación</th>
          <th>Curso</th>
          <th>Aulas</th>
        </tr>
      </thead>
      <tbody>
        <!-- track a.id sustituye al antiguo trackBy -->
        @for (a of asignaturas(); track a.id) {
          <tr>
            <td>{{ a.id }}</td>
            <td>{{ a.denominacion }}</td>
            <td>{{ a.curso }}</td>
            <td>
              <!-- Si hay aulas las muestro separadas por coma -->
              @if (a.aulas.length) {
                {{ a.aulas.join(', ') }}
              } @else { - }
            </td>
          </tr>
        }
      </tbody>
    </table>
  }

  <!-- Estado vacío -->
  @if (!cargando() && asignaturas().length === 0) {
    <p class="text-muted">No hay asignaturas registradas.</p>
  }
</main>

```

Y añadimos la ruta para visualizarlo en el navegador:

- En ~/SpringBoot_Angular/frontendalumnos/src/app/app.routes.ts

```
// src/app/app.routes.ts
import { Routes } from '@angular/router';
// Importamos el componente
import { AlumnoLista } from '../componentes/alumno-lista/alumno-lista';

// Importamos el componente de Asignatura...
import { AsignaturaLista } from '../componentes/asignatura-lista/asignatura-lista';

export const routes: Routes = [
  {
    path: '',
    redirectTo: 'alumnos',
    pathMatch: 'full'
  },
  {
    path: 'alumnos',
    component: AlumnoLista
  },
  // Nueva ruta para asignaturas...
  {
    path: 'asignaturas',
    component: AsignaturaLista
  }
];
```

Y probamos como en ocasiones anteriores (OJO, backend y frontend en pestañas distintas de la consola)

- Iniciamos el Backend...

```
cd ~/SpringBoot_Angular/crudalumnos
./mvnw clean install
./mvnw clean spring-boot:run
```

- Iniciamos el FrontEnd...

```
cd ~/SpringBoot_Angular/frontendalumnos
ng serve
# Y probamos en el navegador:
# http://localhost:8080/swagger-ui/index.html
# http://localhost:4200/alumnos
# http://localhost:4200/asignaturas
```

🍕 FASE 7: SPA completa — las pantallas de los mockups

Tabla de contenidos

Hasta ahora hemos construido las piezas: entidades, API REST, servicios y componentes Angular sueltos. En esta fase las ensamblamos en una **SPA completa** con el aspecto de los mockups de diseño de la Academia Arelance (los cuatro que aparecen al inicio de este documento):

Pantalla	Ruta	Qué muestra
Dashboard	/dashboard	Tarjetas de resumen, matrículas recientes, alumnos por asignatura y alertas
Alumnos	/alumnos	Tabla con buscador, filtros por género, paginación, nº de matrículas y nota media
Asignaturas	/asignaturas	Tabla con buscador, filtro por curso, aulas, nº de alumnos y nota media
Matrículas	/matriculas	Formulario de nueva matrícula + historial del alumno seleccionado

Las entradas *Estadísticas* 📊 y *Carga CSV* 📄 del menú lateral quedan reservadas para la fase de **Python/FastAPI**; *Cursos* y *Aulas* se proponen como ejercicio.

El plan de trabajo:

1. **Backend (7.1):** migrar las entidades con los campos que faltan (`fechaMatricula` , `notaFinal` , `descuento` , `horas`) y crear los endpoints de estadística que necesitan las pantallas.
2. **Frontend (7.2 a 7.7):** actualizar modelos y servicios, montar el layout con menú lateral y construir las cuatro pantallas.

📌 7.1. Backend: migración de campos y endpoints de estadística

Tabla de contenidos

Las pantallas de los mockups muestran datos que **todavía no existen** en nuestro modelo: la fecha de matrícula, la nota final, el descuento aplicado y las horas de cada asignatura. También necesitan agregaciones (contar, medias) que aún no tienen endpoint. Vamos a resolverlo todo aquí.

7.1.1. El enum TipoDescuento

Empezamos por el tipo de descuento de la matrícula. Es un conjunto **cerrado** de valores → un `enum` de Java es la herramienta perfecta. Lo creamos en el paquete `modelo` :

- En `~/SpringBoot_Angular/crudalumnos/src/main/java/com/inserta/crudalumnos/modelo/TipoDescuento.java`

```
// src/main/java/com/inserta/crudalumnos/modelo/TipoDescuento.java
package com.inserta.crudalumnos.modelo;

/**
 * Tipos de descuento aplicables a una matrícula.
 * Cada valor lleva asociado su porcentaje de descuento.
 */
public enum TipoDescuento {
    NINGUNO(0),
    DESEMPLEADO(10),
    FAMILIA_NUMEROSA(20),
    VULNERABLE(30);

    // Los enum de Java pueden tener atributos y métodos
    private final int porcentaje;

    // Constructor del enum (siempre privado implícitamente)
    TipoDescuento(int porcentaje) {
        this.porcentaje = porcentaje;
    }

    public int getPorcentaje() {
        return porcentaje;
    }
}
```

7.1.2. Migración de las entidades

Ampliamos `AlumnoAsignatura` con los tres campos de la matrícula. Gracias a `spring.jpa.hibernate.ddl-auto=update`, Hibernate añadirá las columnas nuevas **sin borrar los datos existentes**.

- En `~/SpringBoot_Angular/crudalumnos/src/main/java/com/inserta/crudalumnos/modelo/AlumnoAsignatura.java`


```

// src/main/java/com/inserta/crudalumnos.modelo/AlumnoAsignatura.java
package com.inserta.crudalumnos.modelo;

// [NUEVO] Imports para los campos de la migración
import java.math.BigDecimal;
import java.time.LocalDate;

import com.fasterxml.jackson.annotation.JsonIgnore;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.EnumType;
import jakarta.persistence.Enumerated;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;
import jakarta.persistence.UniqueConstraint;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
@Table(name="alumnos_asignaturas",
    uniqueConstraints = @UniqueConstraint(
        columnNames = {"alumno_nif", "asignatura_id"}
    )
)
public class AlumnoAsignatura {
    /**
     * Usaremos una ID autogenerada en vez de compuesta
     * porque la entidad tiene campos adicionales propios
     * (fecha, nota, descuento)
     */
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    /**
     * @JsonIgnore sirve para parar la búsqueda en la serialización
     * Sin @JsonIgnore: Alumno → AlumnoAsignatura → Alumno → ∞
     */
    @ManyToOne
    @JoinColumn(name = "alumno_nif")
    @JsonIgnore
    private Alumno alumno;

    @ManyToOne
    @JoinColumn(name = "asignatura_id")
    private Asignatura asignatura;

    // [NUEVO] — Campos de la migración —————

    // LocalDate → columna DATE (solo fecha, sin hora)
    private LocalDate fechaMatricula;    // SQL -> fecha_matricula DATE

    // BigDecimal → DECIMAL(4,2): 4 dígitos en total, 2 decimales (0.00 a 99.99)
    // NUNCA usar float/double para notas o dinero: tienen errores de redondeo
    @Column(precision = 4, scale = 2)
    private BigDecimal notaFinal;        // SQL -> nota_final DECIMAL(4,2)

```

```
// El enum se guarda como texto ("DESEMPLEADO"), NUNCA como número
@Enumerated(EnumType.STRING)
@Column(length = 20)
private TipoDescuento descuento;    // SQL -> descuento VARCHAR(20)
}
```

! Caution

`@Enumerated(EnumType.STRING)` es **obligatorio**. El valor por defecto de JPA es `ORDINAL`, que guarda la **posición** del enum (0, 1, 2...). Si un día alguien reordena o inserta un valor en el enum, TODOS los datos existentes pasan a significar otra cosa. Con `STRING` la base de datos guarda el nombre y es inmune a reordenaciones.

Ahora ampliamos `Asignatura` con las horas del módulo:

- En `~/SpringBoot_Angular/crudalumnos/src/main/java/com/inserta/crudalumnos/modelo/Asignatura.java`

```
// src/main/java/com/inserta/crudalumnos/modelo/Asignatura.java
package com.inserta.crudalumnos.modelo;

import java.util.ArrayList;
import java.util.List;

import com.fasterxml.jackson.annotation.JsonIgnore;

import jakarta.persistence.CollectionTable;
import jakarta.persistence.Column;
import jakarta.persistence.ElementCollection;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.OneToMany;
import jakarta.persistence.Table;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

/* strategy = GenerationType.IDENTITY
 * Define la forma de poner las ids, autoincrementales
 */
@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
@Table(name = "asignaturas")
public class Asignatura {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;           // SQL -> id INT AUTO_INCREMENT

    @Column(length = 50, nullable = false)
    private String denominacion;

    @Column(columnDefinition = "TINYINT", nullable = true)
    // Un int primitivo no puede ser NULL
    private Integer curso;       // SQL -> curso TINYINT NULL

    // [NUEVO] Horas del módulo. Usamos int primitivo: una asignatura
    // SIEMPRE tiene horas, así que no tiene sentido permitir NULL
    @Column(nullable = false)
    private int horas;           // SQL -> horas INT NOT NULL

    // Colección de valores embebidos (tabla secundaria automática)
    @ElementCollection
    @CollectionTable(name = "asignaturas_aulas",
        joinColumns = @JoinColumn(name = "asignatura_id"))
    private List<Float> aulas = new ArrayList<>();

    @OneToMany(mappedBy = "asignatura")
    @JsonIgnore // Esto rompe el bucle en la serialización
    private List<AlumnoAsignatura> alumnoAsignaturas = new ArrayList<>();
}
```

🔴 Al añadir campos a una entidad con Lombok, **cambia la firma de @AllArgsConstructor** (el constructor incluye todos los campos, en el orden en el que están declarados). En cuanto guardes estos archivos, el compilador marcará en rojo **todas** las llamadas `new Asignatura(...)` y `new AlumnoAsignatura(...)` del proyecto (`DatosIniciales` y `MatriculaController`). Es lo esperado: las actualizamos a continuación.

**Note**

Al arrancar, Hibernate ejecutará los `ALTER TABLE` necesarios. En MySQL, añadir una columna `NOT NULL` a una tabla con datos rellena las filas existentes con el valor por defecto del tipo (`0` para `INT`). Por eso las asignaturas ya creadas aparecerán con `horas = 0` hasta que las actualicemos.

7.1.3. DatosIniciales con los campos nuevos

Actualizamos la carga inicial con valores de ejemplo para **todos** los campos de la migración: horas reales de módulos de DAW, fechas de matrícula del curso 2025-2026, notas (algunas todavía sin poner → `null`) y descuentos variados.

- En ~/SpringBoot_Angular/crudalumnos/src/main/java/com/inserta/crudalumnos/carga/DatosIniciales.java

```

// src/main/java/com/inserta/crudalumnos/carga/DatosIniciales.java
package com.inserta.crudalumnos.carga;

// [NUEVO] Imports para fechas y decimales
import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

import com.inserta.crudalumnos.modelo.Alumno;
import com.inserta.crudalumnos.modelo.AlumnoAsignatura;
import com.inserta.crudalumnos.modelo.Asignatura;
import com.inserta.crudalumnos.modelo.TipoDescuento;
import com.inserta.crudalumnos.repositorio.AlumnoAsignaturaRepository;
import com.inserta.crudalumnos.repositorio.AlumnoRepository;
import com.inserta.crudalumnos.repositorio.AsignaturaRepository;

@Component
public class DatosIniciales implements CommandLineRunner{

    // 1. Añadimos los repositorios!
    private final AlumnoRepository alumnoRepo;
    private final AsignaturaRepository asignaturaRepo;
    private final AlumnoAsignaturaRepository alumnoAsignaturaRepo;

    // 2. Creamos el constructor completo
    public DatosIniciales(AlumnoRepository alumnoRepo, AsignaturaRepository asignaturaRepo,
        AlumnoAsignaturaRepository alumnoAsignaturaRepo) {
        this.alumnoRepo = alumnoRepo;
        this.asignaturaRepo = asignaturaRepo;
        this.alumnoAsignaturaRepo = alumnoAsignaturaRepo;
    }

    // 3. Metemos los datos en el run
    @Override
    public void run(String... args) throws Exception {
        // Comprobamos que las tablas están vacías
        if (alumnoRepo.count() == 0 && asignaturaRepo.count() == 0) {

            // 3.1 Crear asignaturas
            // [NUEVO] 4º parámetro: horas del módulo (horas reales de DAW)
            Asignatura bbdd = new Asignatura(null, "Bases de datos", 1, 192,
                List.of(1.1f, 2.2f), new ArrayList<>());
            Asignatura prog = new Asignatura(null, "Programación", 1, 256,
                List.of(2.2f, 2.3f), new ArrayList<>());
            Asignatura fol = new Asignatura(null, "Form.Orientacion Lab", 1, 96,
                List.of(2.2f, 2.3f), new ArrayList<>());
            asignaturaRepo.saveAll(List.of(bbdd, prog, fol));

            // 3.2 Crear alumnos
            Alumno parraga = new Alumno("11A", "Alex Párraga", 23, false, new ArrayList<>());
            Alumno rider = new Alumno("22B", "Miguel Rider", 39, false, new ArrayList<>());
            Alumno aleon = new Alumno("33C", "Alex León", 30, false, new ArrayList<>());
            Alumno jaime = new Alumno("44D", "Jaime Peña", 30, false, new ArrayList<>());
            Alumno angela = new Alumno("55E", "Angela Crespo", 20, true, new ArrayList<>());
            alumnoRepo.saveAll(List.of(parraga, rider, aleon, jaime, angela));

            // 3.3 Relacionamos alumnos con asignaturas
            // [NUEVO] Parámetros: fechaMatricula, notaFinal, descuento
            // Las notas a null son matriculas aún sin evaluar (alimentan
            // la alerta "alumnos sin nota" del Dashboard)
            List<AlumnoAsignatura> relaciones = List.of(
                new AlumnoAsignatura(null, jaime, fol,

```

```

        LocalDate.of(2025, 9, 15), new BigDecimal("8.50"), TipoDescuento.NINGUNO),
        new AlumnoAsignatura(null, jaime, bbdd,
            LocalDate.of(2025, 9, 15), new BigDecimal("6.75"), TipoDescuento.NINGUNO),
        new AlumnoAsignatura(null, jaime, prog,
            LocalDate.of(2025, 10, 1), null, TipoDescuento.NINGUNO),
        new AlumnoAsignatura(null, aleon, prog,
            LocalDate.of(2025, 10, 1), new BigDecimal("9.10"), TipoDescuento.DESEMPLEADO),
        new AlumnoAsignatura(null, rider, bbdd,
            LocalDate.of(2025, 9, 20), new BigDecimal("4.20"), TipoDescuento.FAMILIA_NUMEROSA),
        new AlumnoAsignatura(null, rider, prog,
            LocalDate.of(2025, 9, 20), new BigDecimal("5.90"), TipoDescuento.FAMILIA_NUMEROSA),
        new AlumnoAsignatura(null, parraga, fol,
            LocalDate.of(2025, 10, 10), new BigDecimal("7.75"), TipoDescuento.NINGUNO),
        new AlumnoAsignatura(null, parraga, bbdd,
            LocalDate.of(2025, 10, 10), null, TipoDescuento.VULNERABLE),
        new AlumnoAsignatura(null, angela, bbdd,
            LocalDate.of(2025, 9, 15), new BigDecimal("8.90"), TipoDescuento.NINGUNO),
        new AlumnoAsignatura(null, angela, prog,
            LocalDate.of(2025, 9, 15), new BigDecimal("7.30"), TipoDescuento.NINGUNO)
    );

    alumnoAsignaturaRepo.saveAll(relaciones);
}

}

}

```

**Warning**

Recuerda que `DatosIniciales` solo inserta si las tablas están **vacías**. Si ya tienes datos del curso anterior, tienes dos opciones: (a) actualizar a mano las matrículas existentes con `UPDATE` en MySQL, o (b) vaciar la BBDD para regenerarla:

```
DROP DATABASE inserta_matriculas; CREATE DATABASE inserta_matriculas; y arrancar de nuevo.
```

7.1.4. DTOs con records de Java

Las pantallas necesitan datos **agregados** (por ejemplo: "por cada asignatura, cuántos alumnos y qué nota media"). Devolver entidades enteras sería ineficiente y ambiguo. La solución moderna en Java son los **records** (desde Java 16): clases inmutables de una línea, perfectas como DTO (*Data Transfer Object*).

Creamos el paquete `dto` y dentro tres records:

- En `~/SpringBoot_Angular/crudalumnos/src/main/java/com/inserta/crudalumnos/dto/AlumnoResumen.java`

```
// src/main/java/com/inserta/crudalumnos/dto/AlumnoResumen.java
package com.inserta.crudalumnos.dto;

/**
 * DTO de resumen por alumno: nº de matriculas y nota media.
 * Un record genera automáticamente constructor, getters,
 * equals, hashCode y toString. Inmutable.
 * notaMedia es Double (envoltura): será null si el alumno
 * no tiene ninguna nota todavía.
 */
public record AlumnoResumen(String nif, Long totalMatriculas, Double notaMedia) {}
```

- En `~/SpringBoot_Angular/crudalumnos/src/main/java/com/inserta/crudalumnos/dto/AsignaturaResumen.java`

```
// src/main/java/com/inserta/crudalumnos/dto/AsignaturaResumen.java
package com.inserta.crudalumnos.dto;

/**
 * DTO de resumen por asignatura: nº de alumnos matriculados y nota media.
 */
public record AsignaturaResumen(Integer id, String denominacion,
                                Long totalAlumnos, Double notaMedia) {}
```

- En `~/SpringBoot_Angular/crudalumnos/src/main/java/com/inserta/crudalumnos/dto/MatriculaReciente.java`

```
// src/main/java/com/inserta/crudalumnos/dto/MatriculaReciente.java
package com.inserta.crudalumnos.dto;

import java.math.BigDecimal;
import java.time.LocalDate;

/**
 * DTO para la tabla "Matriculas recientes" del Dashboard.
 * Incluye el nombre del alumno, que la entidad AlumnoAsignatura
 * no serializa (su campo alumno lleva @JsonIgnore).
 */
public record MatriculaReciente(String nif, String nombreAlumno,
                                String denominacion,
                                LocalDate fechaMatricula,
                                BigDecimal notaFinal) {}
```



Note

En el JSON de salida, los nombres de los campos del record se convierten directamente en claves:

```
{ "nif": "11A", "totalMatriculas": 2, "notaMedia": 7.62 } . Jackson soporta records de forma nativa.
```


7.1.5. Repositorio: consultas de agregación

Ampliamos `AlumnoAsignaturaRepository` (el que completamos en el apartado 3.5) con las consultas que alimentan las pantallas. Aparecen tres técnicas nuevas de JPQL: **funciones de agregación** (`AVG`), **expresiones constructoras** (`SELECT new ...`) que crean DTOs directamente desde la consulta, y **LEFT JOIN** para no perder los registros sin matrículas.

- En `~/SpringBoot_Angular/crudalumnos/src/main/java/com/inserta/crudalumnos/repositorio/AlumnoAsignaturaRepository.java`

```

// src/main/java/com/inserta/crudalumnos/repositorio/AlumnoAsignaturaRepository.java
package com.inserta.crudalumnos.repositorio;

import java.util.List;

import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Modifying;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;
import org.springframework.transaction.annotation.Transactional;

import com.inserta.crudalumnos.dto.AlumnoResumen;
import com.inserta.crudalumnos.dto.AsignaturaResumen;
import com.inserta.crudalumnos.dto.MatriculaReciente;
import com.inserta.crudalumnos.modelo.Alumno;
import com.inserta.crudalumnos.modelo.AlumnoAsignatura;

/**
 * Repositorio para gestionar la tabla intermedia de matrículas (Alumno ↔ Asignatura).
 *
 * Consultas básicas (desde 3.5):
 * - Comprobar si un alumno ya está matriculado en una asignatura
 * - Borrar una matrícula concreta
 * - Listar todas las matrículas de un alumno / de una asignatura
 * - Contar los alumnos matriculados en una asignatura
 * - Listar los alumnos matriculados en una asignatura (por su denominación)
 *
 * Consultas de estadística (FASE 7):
 * - Contar las matrículas de un alumno
 * - Nota media de un alumno / global
 * - Últimas matrículas (Dashboard)
 * - Resumen agregado por alumno y por asignatura (DTOs)
 */
@Repository
public interface AlumnoAsignaturaRepository extends JpaRepository<AlumnoAsignatura, Long> {

    /**
     * Verifica si existe una matrícula de un alumno en una asignatura concreta.
     * "" -> text block de java (desde v15), permite escribir cadenas multilínea
     * aa -> alias de AlumnoAsignatura
     * @param nif NIF del alumno
     * @param idAsignatura ID de la asignatura
     * @return true si ya existe, false si no
     */
    @Query("""
        SELECT CASE WHEN COUNT(aa) > 0
            THEN TRUE
            ELSE FALSE END
        FROM AlumnoAsignatura aa
        WHERE aa.alumno.nif = :nif
            AND aa.asignatura.id = :idAsignatura
        """)
    boolean existeMatricula(@Param("nif") String nif,
        @Param("idAsignatura") Integer idAsignatura);

    /**
     * Borra la matrícula de un alumno en una asignatura.
     *
     * @param nif NIF del alumno
     * @param idAsignatura ID de la asignatura
     */
    @Modifying // Va a OPERAR (no consultar) sobre la BBDD
    @Transactional // Se ejecuta en una transacción
    @Query("""
        DELETE FROM AlumnoAsignatura aa
    """)

```

```

        WHERE aa.alumno.nif = :nif
        AND aa.asignatura.id = :idAsignatura
    """)
    void borrarMatricula(@Param("nif") String nif,
        @Param("idAsignatura") Integer idAsignatura);

/**
 * Busca todas las matriculas de un alumno por su NIF.
 *
 * @param nif NIF del alumno
 * @return lista de matriculas de ese alumno
 */
@Query("""
    SELECT aa
    FROM AlumnoAsignatura aa
    WHERE aa.alumno.nif = :nif
    ORDER BY aa.asignatura.denominacion ASC
    """)
List<AlumnoAsignatura> buscarPorAlumno(@Param("nif") String nif);

/**
 * Busca todas las matrículas de una asignatura por su ID.
 *
 * @param idAsignatura ID de la asignatura
 * @return lista de matriculas en esa asignatura
 */
@Query("""
    SELECT aa
    FROM AlumnoAsignatura aa
    WHERE aa.asignatura.id = :idAsignatura
    ORDER BY aa.alumno.nif ASC
    """)
List<AlumnoAsignatura> buscarPorAsignatura(@Param("idAsignatura") Integer idAsignatura);

/**
 * Cuenta cuántos alumnos están matriculados en una asignatura.
 *
 * @param idAsignatura ID de la asignatura
 * @return número de alumnos matriculados
 */
@Query("""
    SELECT COUNT(aa)
    FROM AlumnoAsignatura aa
    WHERE aa.asignatura.id = :idAsignatura
    """)
long contarPorAsignatura(@Param("idAsignatura") Integer idAsignatura);

/**
 * Lista los alumnos matriculados en una asignatura (por su denominación).
 * JOIN entre AlumnoAsignatura y Asignatura: devolvemos el Alumno asociado.
 *
 * @param denominacion nombre de la asignatura
 * @return lista de alumnos matriculados en esa asignatura
 */
@Query("""
    SELECT aa.alumno
    FROM AlumnoAsignatura aa
    JOIN aa.asignatura asig
    WHERE asig.denominacion = :denominacion
    """)
List<Alumno> buscarAlumnosPorAsignatura(@Param("denominacion") String denominacion);

// -----
// [NUEVO] Consultas de estadística para las pantallas (FASE 7)
// -----

/**

```

```

    * Cuenta cuántas matrículas tiene un alumno.
    * (Columna MATRÍCULAS de la pantalla Alumnos)
    *
    * @param nif NIF del alumno
    * @return número de matrículas del alumno
    */
    @Query("""
        SELECT COUNT(aa)
        FROM AlumnoAsignatura aa
        WHERE aa.alumno.nif = :nif
        """)
    long contarPorAlumno(@Param("nif") String nif);

    /**
     * Nota media de un alumno.
     * AVG ignora los null (matrículas sin evaluar no cuentan).
     * (Columna NOTA MEDIA de la pantalla Alumnos)
     *
     * @param nif NIF del alumno
     * @return media de sus notas, o null si no tiene ninguna nota
     */
    @Query("""
        SELECT AVG(aa.notaFinal)
        FROM AlumnoAsignatura aa
        WHERE aa.alumno.nif = :nif
        """)
    Double notaMediaPorAlumno(@Param("nif") String nif);

    /**
     * Nota media global de la academia.
     * (Tarjeta "Nota media global" del Dashboard)
     *
     * @return media de todas las notas, o null si no hay ninguna
     */
    @Query("""
        SELECT AVG(aa.notaFinal)
        FROM AlumnoAsignatura aa
        """)
    Double notaMediaGlobal();

    /**
     * Últimas matrículas, ordenadas por fecha descendente.
     * SELECT new ... = "expresión constructora": la consulta llama
     * directamente al constructor del record (ruta COMPLETA del DTO).
     * JPQL no tiene LIMIT: el tope se pasa con Pageable desde el controlador.
     * (Tabla "Matrículas recientes" del Dashboard)
     *
     * @param pageable tamaño de página (ej: PageRequest.of(0, 5) → las 5 últimas)
     * @return lista de DTOs con alumno, asignatura, fecha y nota
     */
    @Query("""
        SELECT new com.inserta.crudalumnos.dto.MatriculaReciente(
            a.nif, a.nombre, asig.denominacion,
            aa.fechaMatricula, aa.notaFinal)
        FROM AlumnoAsignatura aa
        JOIN aa.alumno a
        JOIN aa.asignatura asig
        ORDER BY aa.fechaMatricula DESC
        """)
    List<MatriculaReciente> ultimasMatriculas(Pageable pageable);

    /**
     * Resumen agregado por alumno: nº de matrículas y nota media.
     * LEFT JOIN para que los alumnos SIN matrículas también salgan
     * (con COUNT = 0 y media null).
     *
     * @return un AlumnoResumen por cada alumno de la academia
     */

```

```

    */
    @Query("""
        SELECT new com.inserta.crudalumnos.dto.AlumnoResumen(
            a.nif, COUNT(aa), AVG(aa.notaFinal))
        FROM Alumno a
        LEFT JOIN AlumnoAsignatura aa ON aa.alumno = a
        GROUP BY a.nif
        """)
    List<AlumnoResumen> resumenPorAlumno();

    /**
     * Resumen agregado por asignatura: nº de alumnos y nota media.
     * LEFT JOIN para que las asignaturas SIN alumnos también salgan
     * (alimenta la alerta "asignatura sin alumnos" del Dashboard).
     *
     * @return un AsignaturaResumen por cada asignatura
     */
    @Query("""
        SELECT new com.inserta.crudalumnos.dto.AsignaturaResumen(
            asig.id, asig.denominacion, COUNT(aa), AVG(aa.notaFinal))
        FROM Asignatura asig
        LEFT JOIN AlumnoAsignatura aa ON aa.asignatura = asig
        GROUP BY asig.id, asig.denominacion
        """)
    List<AsignaturaResumen> resumenPorAsignatura();
}

```

**Tip**

Fíjate en que el repositorio puede consultar **cualquier entidad**, no solo la suya: `resumenPorAlumno()` arranca `FROM Alumno`. El `JpaRepository<AlumnoAsignatura, Long>` solo condiciona los métodos heredados (`findAll`, `save` ...), no las `@Query` que escribamos.

7.1.6. Controladores actualizados

Presentamos `MatriculaController` **completo** con todos los endpoints nuevos, y `AsignaturaController` actualizado para que `crear` y `actualizar` reciban las horas.

- En `~/SpringBoot_Angular/crudalumnos/src/main/java/com/inserta/crudalumnos/controlador/MatriculaController.java`

```

// src/main/java/com/inserta/crudalumnos/controlador/MatriculaController.java
package com.inserta.crudalumnos.controlador;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.List;

import org.springframework.data.domain.PageRequest;
import org.springframework.format.annotation.DateTimeFormat;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import com.inserta.crudalumnos.dto.AlumnoResumen;
import com.inserta.crudalumnos.dto.AsignaturaResumen;
import com.inserta.crudalumnos.dto.MatriculaReciente;
import com.inserta.crudalumnos.modelo.Alumno;
import com.inserta.crudalumnos.modelo.AlumnoAsignatura;
import com.inserta.crudalumnos.modelo.Asignatura;
import com.inserta.crudalumnos.modelo.TipoDescuento;
import com.inserta.crudalumnos.repositorio.AlumnoAsignaturaRepository;
import com.inserta.crudalumnos.repositorio.AlumnoRepository;
import com.inserta.crudalumnos.repositorio.AsignaturaRepository;

import io.swagger.v3.oas.annotations.Operation;

/**
 * Controlador REST para gestionar MATRÍCULAS (Alumno ↔ Asignatura).
 *
 * * Endpoints básicos (desde 3.5):
 * * - POST /api/matriculas/matricular/{nif}/{idAsignatura} → Matricular (+fecha/nota/descuento opcionales)
 * * - DELETE /api/matriculas/desmatricular/{nif}/{idAsignatura} → Desmatricular
 * * - GET /api/matriculas/matriculados?denominacion=... → Alumnos de una asignatura
 *
 * * Endpoints de la FASE 7:
 * * - GET /api/matriculas/consultar → Todas las matrículas
 * * - GET /api/matriculas/consultar/alumno/{nif} → Matrículas de un alumno
 * * - GET /api/matriculas/contar → Total de matrículas
 * * - GET /api/matriculas/contar/{nif} → Matrículas de un alumno (nº)
 * * - GET /api/matriculas/notamedia → Nota media global
 * * - GET /api/matriculas/notamedia/{nif} → Nota media de un alumno
 * * - GET /api/matriculas/recientes → Últimas 5 matrículas (DTO)
 * * - GET /api/matriculas/resumen-alumnos → Resumen por alumno (DTO)
 * * - GET /api/matriculas/resumen-asignaturas → Resumen por asignatura (DTO)
 * * - PUT /api/matriculas/nota/{id}?nota=8.5 → Poner/actualizar la nota
 */
@RestController
@RequestMapping("/api/matriculas")
public class MatriculaController {

    private final AlumnoRepository alumnoRepo;
    private final AsignaturaRepository asignaturaRepo;
    private final AlumnoAsignaturaRepository aluAsigRepo;

    public MatriculaController(AlumnoRepository alumnoRepo,
                               AsignaturaRepository asignaturaRepo,
                               AlumnoAsignaturaRepository aluAsigRepo) {
        this.alumnoRepo = alumnoRepo;
        this.asignaturaRepo = asignaturaRepo;
        this.aluAsigRepo = aluAsigRepo;
    }

    // + POST - Matricular un alumno en una asignatura
    // [NUEVO] Parámetros opcionales: fecha (ISO yyyy-MM-dd), nota y descuento
    // Ej: /api/matriculas/matricular/11A/2?fecha=2025-09-15&nota=8.5&descuento=DESEMPLEADO
    @PostMapping("/matricular/{nif}/{idAsignatura}")
    @Operation(summary = "Matricular un alumno en una asignatura")
    public ResponseEntity<?>

```

```

        matricular(@PathVariable String nif,
                    @PathVariable Integer idAsignatura,
                    @RequestParam(required = false)
                    @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate fecha,
                    @RequestParam(required = false) BigDecimal nota,
                    @RequestParam(required = false, defaultValue = "NINGUNO")
                    TipoDescuento descuento) {

    // 1) Validar existencia de Alumno y Asignatura
    final Alumno alumno = alumnoRepo.findById(nif).orElse(null);
    if (alumno == null)
        return ResponseEntity.notFound().build(); // 404 si el alumno no existe

    final Asignatura asig = asignaturaRepo.findById(idAsignatura).orElse(null);
    if (asig == null)
        return ResponseEntity.notFound().build(); // 404 si la asignatura no existe

    // 2) Evitar duplicados (además del unique constraint)
    if (aluAsigRepo.existeMatricula(nif, idAsignatura)) {
        return ResponseEntity.badRequest().body("El alumno ya está matriculado en esa asignatura");
    }

    // 3) Crear relación y devolverla
    // Si no llega fecha, usamos la de hoy
    AlumnoAsignatura rel = new AlumnoAsignatura(null, alumno, asig,
        (fecha != null) ? fecha : LocalDate.now(),
        nota,
        descuento);
    return ResponseEntity.ok(aluAsigRepo.save(rel)); // 200 con la matrícula creada
}

// ✗ DELETE - Desmatricular un alumno de una asignatura
@DeleteMapping("/desmatricular/{nif}/{idAsignatura}")
@Operation(summary = "Desmatricular un alumno de una asignatura")
public ResponseEntity<>
    desmatricular(@PathVariable String nif,
                  @PathVariable Integer idAsignatura) {

    // 1) Si no existe la matrícula - 404 (nada que borrar)
    if (!aluAsigRepo.existeMatricula(nif, idAsignatura)) {
        return ResponseEntity.notFound().build();
    }

    // 2) Borrar relación
    aluAsigRepo.borrarMatricula(nif, idAsignatura);
    return ResponseEntity.noContent().build(); // 204 OK sin contenido
}

// 🔍 GET - Alumnos matriculados en una asignatura (por su denominación)
@GetMapping("/matriculados")
@Operation(summary = "Listar los alumnos matriculados en una asignatura")
public List<Alumno> matriculados(@RequestParam String denominacion) {
    // Ej: /api/matriculas/matriculados?denominacion=Programación
    return aluAsigRepo.buscarAlumnosPorAsignatura(denominacion);
}

// -----
// [NUEVO] Endpoints de la FASE 7
// -----

// 🔍 GET - Todas las matrículas
@GetMapping("/consultar")
@Operation(summary = "Listar todas las matrículas")
public List<AlumnoAsignatura> verMatriculas() {
    return aluAsigRepo.findAll();
}

```



```

// 🔍 GET - Matriculas de un alumno (historial de la pantalla Matriculas)
@GetMapping("/consultar/alumno/{nif}")
@Operation(summary = "Listar las matriculas de un alumno")
public List<AlumnoAsignatura> verMatriculasDeAlumno(@PathVariable String nif) {
    return aluAsigRepo.buscarPorAlumno(nif);
}

// 📊 GET - Total de matriculas (tarjeta del Dashboard)
@GetMapping("/contar")
@Operation(summary = "Contar todas las matriculas")
public long contarMatriculas() {
    return aluAsigRepo.count();
}

// 📊 GET - Nº de matriculas de un alumno
@GetMapping("/contar/{nif}")
@Operation(summary = "Contar las matriculas de un alumno")
public long contarMatriculasDeAlumno(@PathVariable String nif) {
    return aluAsigRepo.contarPorAlumno(nif);
}

// 📊 GET - Nota media global (tarjeta del Dashboard)
@GetMapping("/notamedia")
@Operation(summary = "Nota media global de la academia")
public Double notaMediaGlobal() {
    return aluAsigRepo.notaMediaGlobal(); // null si no hay notas
}

// 📊 GET - Nota media de un alumno
@GetMapping("/notamedia/{nif}")
@Operation(summary = "Nota media de un alumno")
public Double notaMediaDeAlumno(@PathVariable String nif) {
    return aluAsigRepo.notaMediaPorAlumno(nif); // null si no tiene notas
}

// 🕒 GET - Últimas 5 matriculas (tabla "Matriculas recientes" del Dashboard)
@GetMapping("/recientes")
@Operation(summary = "Últimas 5 matriculas")
public List<MatriculaReciente> recientes() {
    // PageRequest.of(página, tamaño) → página 0 de 5 elementos
    return aluAsigRepo.ultimasMatriculas(PageRequest.of(0, 5));
}

// 📊 GET - Resumen por alumno (columnas MATRÍCULAS y NOTA MEDIA de /alumnos)
@GetMapping("/resumen-alumnos")
@Operation(summary = "Resumen por alumno: nº de matriculas y nota media")
public List<AlumnoResumen> resumenAlumnos() {
    return aluAsigRepo.resumenPorAlumno();
}

// 📊 GET - Resumen por asignatura (columnas ALUMNOS y NOTA MEDIA de /asignaturas)
@GetMapping("/resumen-asignaturas")
@Operation(summary = "Resumen por asignatura: nº de alumnos y nota media")
public List<AsignaturaResumen> resumenAsignaturas() {
    return aluAsigRepo.resumenPorAsignatura();
}

// 🖋 PUT - Poner o actualizar la nota de una matrícula
// Ej: /api/matriculas/nota/3?nota=8.50
@PutMapping("/nota/{id}")
@Operation(summary = "Poner/actualizar la nota final de una matrícula")
public ResponseEntity<AlumnoAsignatura> actualizarNota(@PathVariable Long id,
    @RequestParam BigDecimal nota) {
    return aluAsigRepo.findById(id)
        .map(m -> {
            m.setNotaFinal(nota);
            return ResponseEntity.ok(aluAsigRepo.save(m));
        });
}

```

```
    })  
    .orElse(ResponseEntity.notFound().build()); // 404 si no existe  
  }  
}
```

Y `AsignaturaController` con las **horas** en `crear` y `actualizar` (solo cambian esos dos métodos, pero lo mostramos completo para copiar-pegar):

- En `~/SpringBoot_Angular/crudalumnos/src/main/java/com/inserta/crudalumnos/controlador/AsignaturaController.java`

```

// src/main/java/com/inserta/crudalumnos/controlador/AsignaturaController.java
package com.inserta.crudalumnos.controlador;

import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

// Importamos el modelo y su repositorio
import com.inserta.crudalumnos.modelo.Asignatura;
import com.inserta.crudalumnos.repositorio.AsignaturaRepository;

// Y las anotaciones para Swagger
import io.swagger.v3.oas.annotations.Operation;

/**
 * Controlador REST para gestionar ASIGNATURAS.
 */
* Endpoints:
* - GET    /api/asignaturas/consultar           → Listar todas
* - GET    /api/asignaturas/consultar/{id}      → Buscar por ID
* - POST   /api/asignaturas/crear/{den}/{curso}/{horas} → Crear (sin JSON)
* - PUT    /api/asignaturas/actualizar/{id}/{den}/{curso}/{horas} → Actualizar (sin JSON)
* - DELETE /api/asignaturas/borrar/{id}         → Eliminar
* - PUT    /api/asignaturas/{id}/aulas?lista=1.1,2.2 → Definir aulas (CSV)
*/

@RestController
@RequestMapping("/api/asignaturas")
public class AsignaturaController {

    private final AsignaturaRepository asignaturaRepo;

    public AsignaturaController(AsignaturaRepository asignaturaRepo) {
        this.asignaturaRepo = asignaturaRepo;
    }

    // 🔍 GET - Listar todas las asignaturas
    @GetMapping("/consultar")
    @Operation(summary = "Listar todas las asignaturas")
    public List<Asignatura> verAsignaturas() {
        return asignaturaRepo.findAll();
    }

    // 🔍 GET - Buscar una asignatura por ID
    @GetMapping("/consultar/{id}")
    @Operation(summary = "Buscar una asignatura por ID")
    public ResponseEntity<Asignatura> verAsignaturaPorId(@PathVariable Integer id) {
        return asignaturaRepo.findById(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build()); // 404 si no existe
    }

    // ➕ POST - Crear asignatura (usando parámetros en ruta, sin JSON)
    // [NUEVO] Añadimos las horas del módulo
    @PostMapping("/crear/{denominacion}/{curso}/{horas}")
    @Operation(summary = "Crear una asignatura (sin JSON)")
    public Asignatura
        crearAsignatura(@PathVariable String denominacion,
            @PathVariable Integer curso,
            @PathVariable int horas) {
        Asignatura a = new Asignatura();
        a.setDenominacion(denominacion);
        a.setCurso(curso);
        a.setHoras(horas);
        // List<Float> aulas se deja vacio; se puede definir luego vía /{id}/aulas
    }

```

```

        return asignaturaRepo.save(a);
    }

    // 🚧 PUT - Actualizar asignatura (también sin JSON, para mantener la sencillez)
    // [NUEVO] Añadimos las horas del módulo
    @PutMapping("/actualizar/{id}/{denominacion}/{curso}/{horas}")
    @Operation(summary = "Actualizar una asignatura (sin JSON)")
    public ResponseEntity<Asignatura>
        actualizarAsignatura(
            @PathVariable Integer id,
            @PathVariable String denominacion,
            @PathVariable Integer curso,
            @PathVariable int horas) {

        return asignaturaRepo.findById(id)
            .map(a -> {
                a.setDenominacion(denominacion);
                a.setCurso(curso);
                a.setHoras(horas);
                return ResponseEntity.ok(asignaturaRepo.save(a));
            })
            .orElse(ResponseEntity.notFound().build()); // 404 si no existe
    }

    // ❌ DELETE - Eliminar asignatura por ID
    @DeleteMapping("/borrar/{id}")
    @Operation(summary = "Eliminar una asignatura por ID")
    public ResponseEntity<Void> borrarAsignatura(@PathVariable Integer id) {
        if (!asignaturaRepo.existsById(id)) {
            return ResponseEntity.notFound().build(); // 404 si no existe
        }
        // Si usas ON DELETE CASCADE, borrará matrículas automáticamente.
        // Si no, asegúrate de borrar primero relaciones en el servicio/repositorio.
        asignaturaRepo.deleteById(id);
        return ResponseEntity.noContent().build(); // 204 OK sin contenido
    }

    // 🏠 PUT - Definir/actualizar aulas con un CSV en ?lista=1.1,2.2,3.3
    @PutMapping("/{id}/aulas")
    @Operation(summary = "Definir las aulas donde se imparte (CSV en query param ?lista=1.1,2.2)")
    public ResponseEntity<?> definirAulas(@PathVariable Integer id,
                                           @RequestParam(name = "lista") String csv) {

        // Parseo robusto del CSV → List<Float>
        final List<Float> aulas;
        try {
            aulas = Arrays.stream(csv.split(","))
                .map(String::trim)
                .filter(s -> !s.isEmpty())
                .map(Float::parseFloat)
                .collect(Collectors.toList());
        } catch (NumberFormatException e) {
            return ResponseEntity.badRequest()
                .body("El formato de 'lista' es inválido. Usa comas y punto decimal. Ej: 1.1, 2.2");
        }

        return asignaturaRepo.findById(id)
            .map(a -> {
                a.getAulas().clear();
                a.getAulas().addAll(aulas);
                return ResponseEntity.ok(asignaturaRepo.save(a));
            })
            .orElse(ResponseEntity.notFound().build()); // 404 si no existe
    }
}

```

Compilamos y probamos en Swagger:

```
cd ~/SpringBoot_Angular/crudalumnos
# Paramos la compilación previa: CTRL + C!
./mvnw clean spring-boot:run
# En Swagger deben aparecer los endpoints nuevos de matriculas:
# http://localhost:8080/swagger-ui/index.html
```

Pruebas rápidas recomendadas antes de seguir:

Endpoint	Resultado esperado
GET /api/matriculas/recientes	5 matrículas con nombre de alumno, fecha y nota
GET /api/matriculas/resumen-alumnos	Un registro por alumno con totalMatriculas y notaMedia
GET /api/matriculas/resumen-asignaturas	Un registro por asignatura, incluidas las de 0 alumnos
GET /api/matriculas/notamedia	La media global (ej: 7.05)
PUT /api/matriculas/nota/3?nota=9.25	La matrícula 3 con la nota actualizada



7.2. Angular: modelos y servicios actualizados

Tabla de contenidos

El backend ya emite los campos y agregaciones nuevas: toca reflejarlo en el frontend. Actualizamos los modelos, creamos las interfaces para los DTOs de resumen y un servicio nuevo para las matrículas.

7.2.1. Modelos

- En ~/SpringBoot_Angular/frontendalumnos/src/app/modelos/asignatura.model.ts

```
// src/app/modelos/asignatura.model.ts
export interface Asignatura {
  id: number;
  denominacion: string;
  curso: number;
  horas: number;    // [NUEVO] horas del módulo
  // Aulas será un array de floats
  aulas: number[];  // List<Float> en backend → number[] en TS
}
```

- En ~/SpringBoot_Angular/frontendalumnos/src/app/modelos/alumno-asignatura.model.ts

```
// src/app/modelos/alumno-asignatura.model.ts
import { Asignatura } from './asignatura.model';

export interface AlumnoAsignatura {
  id: number;
  asignatura: Asignatura;
  // [NUEVO] Campos de la migración
  fechaMatricula: string | null;  // DATE viaja como texto ISO: "2025-09-15"
  notaFinal: number | null;       // DECIMAL(4,2) → number (null = sin evaluar)
  descuento: string | null;       // el enum viaja como texto: "DESEMPLEADO"
  // El campo alumno NO viaja (lleva @JsonIgnore en el backend)
}
```

Y un modelo nuevo con las **tres interfaces** de los DTOs de resumen (las agrupamos en un solo archivo porque siempre viajan juntas):

- En ~/SpringBoot_Angular/frontendalumnos/src/app/modelos/resumen.model.ts

```

// src/app/modelos/resumen.model.ts
// Interfaces espejo de los records DTO del backend (paquete dto).
// Los nombres de campos deben coincidir EXACTAMENTE con los del record.

// GET /api/matriculas/resumen-alumnos
export interface AlumnoResumen {
  nif: string;
  totalMatriculas: number;
  notaMedia: number | null; // null si el alumno no tiene notas
}

// GET /api/matriculas/resumen-asignaturas
export interface AsignaturaResumen {
  id: number;
  denominacion: string;
  totalAlumnos: number;
  notaMedia: number | null;
}

// GET /api/matriculas/recientes
export interface MatriculaReciente {
  nif: string;
  nombreAlumno: string;
  denominacion: string;
  fechaMatricula: string | null;
  notaFinal: number | null;
}

```

7.2.2. El servicio de matrículas

Creamos el servicio manualmente, siguiendo la nomenclatura de Angular 21 (archivo `matricula-service.ts`, sin `.service.` intermedio):

- En `~/SpringBoot_Angular/frontendalumnos/src/app/servicios/matricula-service.ts`

```

// src/app/servicios/matricula-service.ts
import { Injectable } from "@angular/core";
import { HttpClient } from "@angular/common/http";
import { Observable } from "rxjs";

// Modelos
import { AlumnoAsignatura } from "../../modelos/alumno-asignatura.model";
import { AlumnoResumen, AsignaturaResumen, MatriculaReciente } from "../../modelos/resumen.model";

@Injectable({ providedIn: 'root' })
export class MatriculaService {
  private baseUrl = "http://localhost:8080/api/matriculas";

  // Constructor por defecto
  constructor(private http: HttpClient) {}

  // 🔍 Todas las matrículas
  getMatriculas(): Observable<AlumnoAsignatura[]> {
    return this.http.get<AlumnoAsignatura[]>(`${this.baseUrl}/consultar`);
  }

  // 🔍 Historial de un alumno
  getMatriculasDeAlumno(nif: string): Observable<AlumnoAsignatura[]> {
    return this.http.get<AlumnoAsignatura[]>(`${this.baseUrl}/consultar/alumno/${encodeURIComponent(nif)}`);
  }

  // 📊 Total de matrículas (tarjeta del Dashboard)
  contar(): Observable<number> {
    return this.http.get<number>(`${this.baseUrl}/contar`);
  }

  // 📊 Nota media global (tarjeta del Dashboard)
  // El backend puede devolver null (sin notas) → body vacío
  notaMediaGlobal(): Observable<number | null> {
    return this.http.get<number | null>(`${this.baseUrl}/notamedia`);
  }

  // 🕒 Últimas 5 matrículas (tabla del Dashboard)
  recientes(): Observable<MatriculaReciente[]> {
    return this.http.get<MatriculaReciente[]>(`${this.baseUrl}/recientes`);
  }

  // 📊 Resumen por alumno (columnas MATRÍCULAS y NOTA MEDIA de /alumnos)
  resumenAlumnos(): Observable<AlumnoResumen[]> {
    return this.http.get<AlumnoResumen[]>(`${this.baseUrl}/resumen-alumnos`);
  }

  // 📊 Resumen por asignatura (columnas ALUMNOS y NOTA MEDIA de /asignaturas)
  resumenAsignaturas(): Observable<AsignaturaResumen[]> {
    return this.http.get<AsignaturaResumen[]>(`${this.baseUrl}/resumen-asignaturas`);
  }

  // ➕ Matricular. fecha y nota son opcionales → van como query params
  matricular(nif: string, idAsignatura: number,
    fecha?: string, nota?: number | null): Observable<AlumnoAsignatura> {
    // Construimos los query params solo con lo que llegue
    const params: string[] = [];
    if (fecha) params.push(`fecha=${fecha}`); // formato ISO: 2025-09-15
    if (nota !== null && nota !== undefined) params.push(`nota=${nota}`);
    const query = params.length > 0 ? `?${params.join('&')}` : '';

    return this.http.post<AlumnoAsignatura>(`${this.baseUrl}/matricular/${encodeURIComponent(nif)}/${idAsignatura}${query}`,
      null);
  }
}

```



```

    }

    // ❌ Desmatricular
    desmatricular(nif: string, idAsignatura: number): Observable<void> {
        return this.http.delete<void>(
            `${this.baseUrl}/desmatricular/${encodeURIComponent(nif)}/${idAsignatura}`
        );
    }

    // 🖋 Poner/actualizar la nota de una matricula
    actualizarNota(id: number, nota: number): Observable<AlumnoAsignatura> {
        return this.http.put<AlumnoAsignatura>(
            `${this.baseUrl}/nota/${id}?nota=${nota}`,
            null
        );
    }
}

```

7.2.3. Ampliación de los servicios existentes

Al servicio de alumnos le añadimos los contadores (para las pestañas *Todos / Hombres / Mujeres* y la tarjeta del Dashboard):

- En ~/SpringBoot_Angular/frontendalumnos/src/app/servicios/alumno-service.ts

```

// src/app/servicios/alumno-service.ts
// [NUEVO] Métodos a AÑADIR dentro de la clase AlumnoService,
// debajo de borrarAlumno(). El resto de la clase no cambia.

// 📊 Total de alumnos (tarjeta del Dashboard)
contarAlumnos(): Observable<number> {
    return this.http.get<number>(`${this.baseUrl}/contar`);
}

// 📊 Contadores por género (pestañas de la pantalla Alumnos)
contarMujeres(): Observable<number> {
    return this.http.get<number>(`${this.baseUrl}/consultar/contar/mujeres`);
}

contarHombres(): Observable<number> {
    return this.http.get<number>(`${this.baseUrl}/consultar/contar/hombres`);
}

```

Y en el de asignaturas, `crearAsignatura` y `actualizarAsignatura` pasan a enviar también las **horas** (el endpoint cambió en 7.1.6):

- En ~/SpringBoot_Angular/frontendalumnos/src/app/servicios/asignatura-service.ts

```

// src/app/servicios/asignatura-service.ts
// [NUEVO] Sustituir crearAsignatura y actualizarAsignatura por estas
// versiones con horas. El resto de la clase no cambia.

// + Crear (vía parámetros en URL, como en el backend)
crearAsignatura(a: Pick<Asignatura, 'denominacion' | 'curso' | 'horas'>): Observable<Asignatura> {
  return this.http.post<Asignatura>(
    `${this.baseUrl}/crear/${encodeURIComponent(a.denominacion)}/${a.curso}/${a.horas}`,
    null
  );
}

// 🖋 Actualizar (por ID en ruta)
actualizarAsignatura(id: number,
  a: Pick<Asignatura, 'denominacion' | 'curso' | 'horas'>): Observable<Asignatura> {
  return this.http.put<Asignatura>(
    `${this.baseUrl}/actualizar/${id}/${encodeURIComponent(a.denominacion)}/${a.curso}/${a.horas}`,
    null
  );
}

```



7.3. El layout general: topbar + sidebar

Tabla de contenidos

Los mockups comparten un armazón fijo: barra superior azul marino con el título **Academia Arelance · Gestión de Alumnos**, menú lateral con tres grupos (*PRINCIPAL*, *ANALÍTICA*, *CONFIGURACIÓN*) y una zona de contenido donde el router pinta cada pantalla. Ese armazón vive en el **componente raíz** (`app.ts` / `app.html`), que envuelve al `<router-outlet />`.

7.3.1. Tema de colores (aplicando el apartado 4.11)

Primero, el color corporativo. En `styles.css` (estilos **globales**, se cargan después del Bootstrap declarado en `angular.json`) aplicamos la técnica de variables CSS del apartado 4.11:

- En `~/SpringBoot_Angular/frontendalumnos/src/styles.css`

```
/* src/styles.css – estilos globales de la SPA */

/* 🎨 Tema Arelance (técnica del apartado 4.11: variables CSS) */
:root {
  --bs-primary: #123c69;           /* azul marino corporativo */
  --bs-primary-rgb: 18, 60, 105;
  --arelance-sidebar: #ffffff;
  --arelance-topbar: #0d2c4e;
}

/* Los botones no leen --bs-primary: sobrescribimos sus variables */
.btn-primary {
  --bs-btn-bg: #123c69;
  --bs-btn-border-color: #123c69;
  --bs-btn-hover-bg: #0d2c4e;
  --bs-btn-hover-border-color: #0d2c4e;
  --bs-btn-active-bg: #0a2340;
  --bs-btn-active-border-color: #0a2340;
}

body {
  background-color: #f4f6f9;       /* gris muy claro de fondo, como el mockup */
}
```

7.3.2. El componente raíz

Reescribimos los tres archivos del componente raíz. Primero la clase:

- En `~/SpringBoot_Angular/frontendalumnos/src/app/app.ts`

```
// src/app/app.ts
import { Component } from '@angular/core';
// RouterLink -> convierte cualquier elemento en un enlace de la SPA
// RouterLinkActive -> añade una clase CSS cuando su ruta está activa
import { RouterOutlet, RouterLink, RouterLinkActive } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, RouterLink, RouterLinkActive],
  templateUrl: './app.html',
  styleUrls: ['./app.css']
})
export class App {
  titulo = 'Academia Arelance · Gestión de Alumnos';
}
```

Ahora la plantilla con el armazón completo:

- En `~/SpringBoot_Angular/frontendalumnos/src/app/app.html`

```
<!-- src/app/app.html - armazón fijo de la SPA -->

<!-- 1 Barra superior -->
<nav class="navbar navbar-dark topbar px-3">
  <span class="navbar-brand mb-0 h1">🏠 {{ titulo }}</span>
</nav>

<!-- 2 Cuerpo: sidebar + contenido -->
<div class="d-flex layout">

  <!-- Menú lateral -->
  <aside class="sidebar bg-white border-end p-3">

    <h6 class="text-secondary text-uppercase small fw-bold mt-2">Principal</h6>
    <ul class="nav nav-pills flex-column mb-4">
      <!-- routerLinkActive="active" pinta el elemento del menú seleccionado -->
      <li class="nav-item">
        <a class="nav-link" routerLink="/dashboard" routerLinkActive="active">🏠 Dashboard</a>
      </li>
      <li class="nav-item">
        <a class="nav-link" routerLink="/alumnos" routerLinkActive="active">👥 Alumnos</a>
      </li>
      <li class="nav-item">
        <a class="nav-link" routerLink="/asignaturas" routerLinkActive="active">📚 Asignaturas</a>
      </li>
      <li class="nav-item">
        <a class="nav-link" routerLink="/matriculas" routerLinkActive="active">📝 Matriculas</a>
      </li>
    </ul>

    <h6 class="text-secondary text-uppercase small fw-bold">Analítica</h6>
    <ul class="nav nav-pills flex-column mb-4">
      <!-- Reservado para la fase de Python/FastAPI 🐍 -->
      <li class="nav-item"><a class="nav-link disabled">📊 Estadísticas 🐍</a></li>
      <li class="nav-item"><a class="nav-link disabled">📄 Carga CSV 🐍</a></li>
    </ul>

    <h6 class="text-secondary text-uppercase small fw-bold">Configuración</h6>
    <ul class="nav nav-pills flex-column">
      <!-- Propuestos como ejercicio -->
      <li class="nav-item"><a class="nav-link disabled">🎓 Cursos</a></li>
      <li class="nav-item"><a class="nav-link disabled">🏠 Aulas</a></li>
    </ul>
  </aside>

  <!-- Zona de contenido: aquí pinta el router cada pantalla -->
  <main class="flex-grow-1 p-4">
    <router-outlet />
  </main>
</div>
```

Y los estilos propios del armazón:

- En ~/SpringBoot_Angular/frontendalumnos/src/app/app.css

```

/* src/app/app.css – estilos del armazón */

.topbar {
  background-color: var(--areance-topbar);
}

/* El layout ocupa toda la altura restante de la ventana */
.layout {
  min-height: calc(100vh - 56px); /* 56px = altura de la navbar */
}

.sidebar {
  width: 230px;
  flex-shrink: 0; /* que no se encoja al estrechar */
}

/* Pastilla activa del menú con el color corporativo */
.sidebar .nav-link {
  color: #333;
}
.sidebar .nav-link.active {
  background-color: var(--bs-primary);
  color: #fff;
}

```

7.3.3. Las rutas definitivas

- En ~/SpringBoot_Angular/frontendalumnos/src/app/app.routes.ts

```

// src/app/app.routes.ts
import { Routes } from '@angular/router';

// Componentes de las pantallas (dashboard y matriculas los creamos
// en 7.4 y 7.7: hasta entonces, comenta sus dos líneas para compilar)
import { Dashboard } from './componentes/dashboard/dashboard';
import { AlumnoLista } from './componentes/alumno-lista/alumno-lista';
import { AsignaturaLista } from './componentes/asignatura-lista/asignatura-lista';
import { Matriculas } from './componentes/matriculas/matriculas';

export const routes: Routes = [
  // Ruta raíz → redirige al dashboard
  { path: '', redirectTo: 'dashboard', pathMatch: 'full' },
  { path: 'dashboard', component: Dashboard },
  { path: 'alumnos', component: AlumnoLista },
  { path: 'asignaturas', component: AsignaturaLista },
  { path: 'matriculas', component: Matriculas },
];

```



Note
Fíjate en la redirección: `path: ''` con `redirectTo: 'dashboard'` hace que `http://localhost:4200/` lleve directamente al Dashboard. El `pathMatch: 'full'` es obligatorio en redirecciones de ruta vacía: sin él, TODAS las rutas empezarían "coincidiendo" con `''`.



7.4. Pantalla Dashboard

Tabla de contenidos

La pantalla de inicio, según su mockup, tiene cuatro zonas: **tarjetas de resumen** (alumnos, asignaturas, matrículas, nota media global), la tabla de **matrículas recientes**, las barras de **alumnos por asignatura** y las **alertas**. Todo sale de los endpoints de 7.1.

Generamos el componente:

```
cd ~/SpringBoot_Angular/frontendalumnos/src/app/  
ng g c componentes/dashboard  
# Se generan dashboard.ts, dashboard.html, dashboard.css  
# y dashboard.spec.ts (SIN el sufijo .component)
```

- En ~/SpringBoot_Angular/frontendalumnos/src/app/componentes/dashboard/dashboard.ts

```

// src/app/componentes/dashboard/dashboard.ts
import { Component, OnInit, signal, computed } from '@angular/core';
// DatePipe -> para formatear fechas en la plantilla: {{ f | date:'dd/MM/yyyy' }}
import { DatePipe, DecimalPipe } from '@angular/common';
import { RouterLink } from '@angular/router';

// Modelos y servicios
import { AlumnoAsignatura } from '../../../modelos/alumno-asignatura.model';
import { AsignaturaResumen, MatriculaReciente } from '../../../modelos/resumen.model';
import { AlumnoService } from '../../../servicios/alumno-service';
import { AsignaturaService } from '../../../servicios/asignatura-service';
import { MatriculaService } from '../../../servicios/matricula-service';

@Component({
  selector: 'app-dashboard',
  imports: [DatePipe, DecimalPipe, RouterLink],
  templateUrl: './dashboard.html',
  styleUrls: ['./dashboard.css']
})
export class Dashboard implements OnInit {
  // — Señales con los datos de cada zona —————
  totalAlumnos = signal<number>(0);
  totalAsignaturas = signal<number>(0);
  totalMatriculas = signal<number>(0);
  notaMediaGlobal = signal<number | null>(null);
  recientes = signal<MatriculaReciente[]>([]);
  resumenAsignaturas = signal<AsignaturaResumen[]>([]);
  matriculas = signal<AlumnoAsignatura[]>([]); // para calcular alertas

  constructor(
    private alumnoService: AlumnoService,
    private asignaturaService: AsignaturaService,
    private matriculaService: MatriculaService
  ) {}

  ngOnInit(): void {
    // Cada zona se alimenta de su endpoint (peticiones en paralelo)
    this.alumnoService.contarAlumnos().subscribe(n => this.totalAlumnos.set(n));
    this.asignaturaService.getAsignaturas().subscribe(as => this.totalAsignaturas.set(as.length));
    this.matriculaService.contar().subscribe(n => this.totalMatriculas.set(n));
    this.matriculaService.notaMediaGlobal().subscribe(m => this.notaMediaGlobal.set(m));
    this.matriculaService.recientes().subscribe(r => this.recientes.set(r));
    this.matriculaService.resumenAsignaturas().subscribe(r => this.resumenAsignaturas.set(r));
    this.matriculaService.getMatriculas().subscribe(m => this.matriculas.set(m));
  }

  // — Señales COMPUTADAS: se recalculan solas al cambiar las de arriba —

  // Media de matrículas por alumno (texto de la 3ª tarjeta)
  mediaPorAlumno = computed(() =>
    this.totalAlumnos() > 0 ? this.totalMatriculas() / this.totalAlumnos() : 0
  );

  // Barras "Alumnos por asignatura": las 5 con más alumnos, de mayor a menor
  topAsignaturas = computed(() =>
    [...this.resumenAsignaturas()]
      .sort((a, b) => b.totalAlumnos - a.totalAlumnos)
      .slice(0, 5)
  );

  // Máximo de alumnos (para escalar el ancho de las barras al 100%)
  maxAlumnos = computed(() =>
    Math.max(1, ...this.topAsignaturas().map(r => r.totalAlumnos))
  );

  // 🚨 Alerta 1: matrículas sin nota con más de 30 días, agrupadas por asignatura
  sinNota = computed(() => {

```

```

const hace30dias = new Date();
hace30dias.setDate(hace30dias.getDate() - 30);

// Agrupamos con un Map: denominación → nº de matrículas sin nota
const grupos = new Map<string, number>();
for (const m of this.matriculas()) {
  if (m.notaFinal === null && m.fechaMatricula
    && new Date(m.fechaMatricula) < hace30dias) {
    const den = m.asignatura.denominacion;
    grupos.set(den, (grupos.get(den) ?? 0) + 1);
  }
}
// Map → array de objetos para iterarlo con @for
return Array.from(grupos, ([denominacion, total]) => ({ denominacion, total }));
});

// 🚨 Alerta 2: asignaturas sin ningún alumno matriculado
sinAlumnos = computed(() =>
  this.resumenAsignaturas().filter(r => r.totalAlumnos === 0)
);

// Color de fondo del badge según la nota (reutilizado en varias pantallas)
claseNota(nota: number | null): string {
  if (nota === null) return 'bg-secondary-subtle text-secondary-emphasis';
  if (nota >= 7) return 'bg-success-subtle text-success-emphasis';
  if (nota >= 5) return 'bg-warning-subtle text-warning-emphasis';
  return 'bg-danger-subtle text-danger-emphasis';
}
}

```



Tip

Las señales `computed()` son la joya de esta pantalla: `sinNota` o `topAsignaturas` **no se llaman nunca a mano**. Angular detecta de qué señales dependen y las recalcula automáticamente cuando llegan los datos. Es programación **reactiva declarativa**: describimos QUÉ es cada dato, no CUÁNDO recalcularlo.

- En `~/SpringBoot_Angular/frontendalumnos/src/app/componentes/dashboard/dashboard.html`


```

<!-- src/app/componentes/dashboard/dashboard.html -->

<!-- Cabecera de la pantalla -->
<div class="d-flex justify-content-between align-items-center mb-4">
  <div>
    <h3 class="mb-0">Dashboard</h3>
    <span class="text-secondary">Resumen general de la academia</span>
  </div>
  <!-- Acceso directo a la pantalla de matrículas -->
  <a class="btn btn-primary" routerLink="/matriculas">+ Nueva matrícula</a>
</div>

<!-- 1 Tarjetas de resumen -->
<div class="row g-3 mb-4">
  <div class="col-md-3">
    <div class="card shadow-sm borde-azul">
      <div class="card-body">
        <div class="text-secondary small">ALUMNOS ACTIVOS</div>
        <div class="fs-3 fw-bold">{{ totalAlumnos() }}</div>
      </div>
    </div>
  </div>
  <div class="col-md-3">
    <div class="card shadow-sm borde-verde">
      <div class="card-body">
        <div class="text-secondary small">ASIGNATURAS</div>
        <div class="fs-3 fw-bold">{{ totalAsignaturas() }}</div>
      </div>
    </div>
  </div>
  <div class="col-md-3">
    <div class="card shadow-sm borde-naranja">
      <div class="card-body">
        <div class="text-secondary small">MATRÍCULAS ACTIVAS</div>
        <div class="fs-3 fw-bold">{{ totalMatriculas() }}</div>
        <!-- number: '1.1-1' -> mínimo 1 entero, entre 1 y 1 decimales -->
        <div class="small text-secondary">{{ mediaPorAlumno() | number: '1.1-1' }} por alumno</div>
      </div>
    </div>
  </div>
  <div class="col-md-3">
    <div class="card shadow-sm borde-morado">
      <div class="card-body">
        <div class="text-secondary small">NOTA MEDIA GLOBAL</div>
        <div class="fs-3 fw-bold">
          @if (notaMediaGlobal() !== null) {
            {{ notaMediaGlobal() | number: '1.2-2' }}
          } @else {
            -
          }
        </div>
      </div>
    </div>
  </div>
</div>

<div class="row g-3">
  <!-- 2 Matrículas recientes -->
  <div class="col-lg-7">
    <div class="card shadow-sm">
      <div class="card-header bg-white fw-bold">Matrículas recientes</div>
      <table class="table table-hover align-middle mb-0">
        <thead class="table-light">
          <tr><th>Alumno</th><th>Asignatura</th><th>Fecha</th><th>Nota</th></tr>
        </thead>
        <tbody>
          @for (m of recientes()); track m.nif + m.denominacion) {

```

```

        <tr>
          <td>{{ m.nombreAlumno }} <span class="text-secondary small">({{ m.nif }})</span></td>
          <td>{{ m.denominacion }}</td>
          <td>{{ m.fechaMatricula | date:'dd/MM/yyyy' }}</td>
          <td>
            <span class="badge" [class]="claseNota(m.notaFinal)">
              {{ m.notaFinal !== null ? (m.notaFinal | number:'1.2-2') : 'Sin nota' }}
            </span>
          </td>
        </tr>
      </tbody>
    </table>
  </div>
</div>

<!-- 3 Alumnos por asignatura + 4 Alertas -->
<div class="col-lg-5">
  <div class="card shadow-sm mb-3">
    <div class="card-header bg-white fw-bold">Alumnos por asignatura</div>
    <div class="card-body">
      @for (r of topAsignaturas(); track r.id) {
        <div class="d-flex justify-content-between small">
          <span>{{ r.denominacion }}</span><span>{{ r.totalAlumnos }}</span>
        </div>
        <div class="progress mb-3" style="height: 8px;">
          <!-- Ancho proporcional al máximo -->
          <div class="progress-bar"
            [style.width.%]="r.totalAlumnos / maxAlumnos() * 100"></div>
        </div>
      }
    </div>
  </div>

  <div class="card shadow-sm">
    <div class="card-header bg-white fw-bold">Alertas</div>
    <div class="card-body">
      @for (a of sinNota(); track a.denominacion) {
        <div class="alert alert-warning py-2 mb-2">
          ⚠ {{ a.total }} matrícula(s) sin nota en <b>{{ a.denominacion }}</b> (+30 días)
        </div>
      }
      @for (r of sinAlumnos(); track r.id) {
        <div class="alert alert-info py-2 mb-2">
          ⓘ <b>{{ r.denominacion }}</b> no tiene alumnos matriculados
        </div>
      }
      @if (sinNota().length === 0 && sinAlumnos().length === 0) {
        <span class="text-secondary">Sin alertas 🍀</span>
      }
    </div>
  </div>
</div>
</div>

```

- En ~/SpringBoot_Angular/frontendalumnos/src/app/componentes/dashboard/dashboard.css

```

/* src/app/componentes/dashboard/dashboard.css */
/* Borde superior de color de cada tarjeta, como en el mockup */
.borde-azul { border-top: 3px solid #0d6efd; }
.borde-verde { border-top: 3px solid #198754; }
.borde-naranja { border-top: 3px solid #fd7e14; }
.borde-morado { border-top: 3px solid #6f42c1; }

```

**Note**

Aparecen dos *pipes* de Angular: `date: 'dd/MM/yyyy'` formatea el texto ISO `2025-09-15` como `15/09/2025`, y `number: '1.2-2'` fuerza dos decimales. Se importan (`DatePipe`, `DecimalPipe`) en el array `imports` del componente, como cualquier otra dependencia standalone.



7.5. Pantalla Alumnos

Tabla de contenidos

Sustituimos la vista de alumnos de las fases 5 por la del mockup: **buscador**, pestañas *Todos / Hombres / Mujeres* con contadores, **paginación** de 7 filas, columnas de **matrículas** y **nota media**, y el alta/edición en un **modal** de Bootstrap. Es una **reescritura completa** de los tres archivos de `alumno-lista` (haz copia si quieres conservar la versión anterior).

Dos decisiones de diseño que conviene entender antes del código:

1. **Filtrado en el cliente:** en la FASE 4 buscábamos contra el endpoint `/consultar/nombre?nombre=`. Aquí, como la tabla completa ya está cargada, filtramos **en memoria** con señales `computed` (instantáneo, sin peticiones). Para miles de alumnos volveríamos al filtrado en servidor con paginación real (`/consultar/pagina`, apartado 3.4).
 2. **Los resúmenes se fusionan por NIF:** `resumen-alumnos` llega como lista aparte; la convertimos en un `Map` `NIF → resumen` para leerla en $O(1)$ desde cada fila.
- En `~/SpringBoot_Angular/frontendalumnos/src/app/componentes/alumno-lista/alumno-lista.ts`

```

// src/app/componentes/alumno-lista/alumno-lista.ts
import { Component, OnInit, signal, computed, inject, ViewChild, ElementRef } from '@angular/core';
import { DecimalPipe } from '@angular/common';
import { ReactiveFormsModule, FormBuilder, Validators } from '@angular/forms';
import { finalize } from 'rxjs';

// Modelos y servicios
import { Alumno } from '../../../modelos/alumno.model';
import { AlumnoResumen } from '../../../modelos/resumen.model';
import { AlumnoService } from '../../../servicios/alumno-service';
import { MatriculaService } from '../../../servicios/matricula-service';

@Component({
  selector: 'app-alumno-lista',
  imports: [ReactiveFormsModule, DecimalPipe],
  templateUrl: './alumno-lista.html',
  styleUrls: ['./alumno-lista.css']
})
export class AlumnoLista implements OnInit {
  // — Datos base —————
  alumnos = signal<Alumno[]>([]);
  // Map NIF → resumen (matriculas y nota media), para leerlo en O(1)
  resumen = signal<Map<string, AlumnoResumen>>(new Map());
  error = signal<string | null>(null);
  cargando = signal<boolean>(false);
  guardando = signal<boolean>(false);

  // — Filtros y paginación (estado de la pantalla) —————
  busqueda = signal<string>(''); // texto del buscador
  filtroGenero = signal<'todos' | 'hombres' | 'mujeres'>('todos');
  pagina = signal<number>(1);
  readonly POR_PAGINA = 7; // como en el mockup

  // — Formulario del modal (crear/editar) —————
  private fb = inject(FormBuilder);
  modoEdicion = signal<boolean>(false);
  nifOriginal: string | null = null; // NIF del registro en edición
  nifABorrar = signal<Alumno | null>(null); // alumno pendiente de confirmación

  form = this.fb.group({
    nif: ['', [Validators.required, Validators.maxLength(9)]],
    nombre: ['', [Validators.required, Validators.maxLength(50)]],
    edad: [null as number | null, [Validators.min(0), Validators.max(120)]],
    genero: [false, Validators.required] // false = hombre, true = mujer
  });

  // Botones ocultos para cerrar los modales de Bootstrap desde TS
  @ViewChild('cerrarModalForm') cerrarModalForm!: ElementRef<HTMLButtonElement>;
  @ViewChild('cerrarModalBorrar') cerrarModalBorrar!: ElementRef<HTMLButtonElement>;

  constructor(private alumnoService: AlumnoService,
    private matriculaService: MatriculaService) {}

  ngOnInit(): void {
    this.recargar();
  }

  recargar(): void {
    this.cargando.set(true);
    this.alumnoService.getAlumnos()
      .pipe(finalize(() => this.cargando.set(false)))
      .subscribe({
        next: datos => this.alumnos.set(datos),
        error: () => this.error.set('No se pudo conectar con el backend')
      });
  }

  // El resumen llega como lista → lo convertimos en Map por NIF
  this.matriculaService.resumenAlumnos().subscribe(lista =>

```

```

        this.resumen.set(new Map(lista.map(r => [r.nif, r])))
    );
}

// — Cadena de computed: alumnos → filtrados → página visible —

// 1) Aplica género + texto de búsqueda (por nombre o por NIF)
filtrados = computed(() => {
    const texto = this.búsqueda().toLowerCase().trim();
    return this.alumnos().filter(a => {
        const pasaGenero =
            this.filtroGenero() === 'todos' ||
            (this.filtroGenero() === 'mujeres' && a.genero === true) ||
            (this.filtroGenero() === 'hombres' && a.genero === false);
        const pasaTexto = texto === '' ||
            a.nombre.toLowerCase().includes(texto) ||
            a.nif.toLowerCase().includes(texto);
        return pasaGenero && pasaTexto;
    });
});

// 2) Recorta la página actual
totalPaginas = computed(() =>
    Math.max(1, Math.ceil(this.filtrados().length / this.POR_PAGINA))
);
paginados = computed(() => {
    const inicio = (this.pagina() - 1) * this.POR_PAGINA;
    return this.filtrados().slice(inicio, inicio + this.POR_PAGINA);
});

// 3) Contadores de las pestañas
nHombres = computed(() => this.alumnos().filter(a => a.genero === false).length);
nMujeres = computed(() => this.alumnos().filter(a => a.genero === true).length);

// — Acciones de la pantalla —

buscar(texto: string): void {
    this.búsqueda.set(texto);
    this.pagina.set(1); // toda búsqueda vuelve a la página 1
}

cambiarFiltro(f: 'todos' | 'hombres' | 'mujeres'): void {
    this.filtroGenero.set(f);
    this.pagina.set(1);
}

irAPagina(p: number): void {
    if (p >= 1 && p <= this.totalPaginas()) this.pagina.set(p);
}

// Lecturas cómodas del Map de resumen (desde la plantilla)
matriculasDe(nif: string): number {
    return this.resumen().get(nif)?.totalMatriculas ?? 0;
}
notaMediaDe(nif: string): number | null {
    return this.resumen().get(nif)?.notaMedia ?? null;
}

claseNota(nota: number | null): string {
    if (nota === null) return 'bg-secondary-subtle text-secondary-emphasis';
    if (nota >= 7) return 'bg-success-subtle text-success-emphasis';
    if (nota >= 5) return 'bg-warning-subtle text-warning-emphasis';
    return 'bg-danger-subtle text-danger-emphasis';
}

// — Modal de crear/editar —

```

```

nuevoAlumno(): void {
  this.modosEdicion.set(false);
  this.nifOriginal = null;
  this.form.reset({ nif: '', nombre: '', edad: null, genero: false });
  this.form.get('nif').enable(); // el NIF se edita solo al crear
}

editarAlumno(a: Alumno): void {
  this.modosEdicion.set(true);
  this.nifOriginal = a.nif;
  this.form.patchValue({ nif: a.nif, nombre: a.nombre, edad: a.edad, genero: a.genero });
  this.form.get('nif').disable(); // el NIF (PK) no se cambia al editar
}

guardar(): void {
  if (this.form.invalid) { this.form.markAllAsTouched(); return; }
  this.guardando.set(true);

  // getRawValue() incluye también los controles disabled (el NIF en edición)
  const v = this.form.getRawValue();
  const datos = {
    nif: v.nif!, nombre: v.nombre!,
    edad: v.edad ?? 0, genero: v.genero!
  };

  // Según el modo, POST /crear o PUT /actualizar
  const peticion = this.modosEdicion()
    ? this.alumnoService.actualizarAlumno(this.nifOriginal!, datos)
    : this.alumnoService.crearAlumno(datos);

  peticion.pipe(finally(() => this.guardando.set(false))).subscribe({
    next: () => {
      this.cerrarModalForm.nativeElement.click(); // cierra el modal
      this.recargar();
    },
    error: () => this.error.set('No se pudo guardar el alumno')
  });
}

// — Modal de confirmación de borrado —————

pedirBorrado(a: Alumno): void {
  this.nifABorrar.set(a);
}

confirmarBorrado(): void {
  const a = this.nifABorrar();
  if (!a) return;
  this.alumnoService.borrarAlumno(a.nif).subscribe({
    next: () => {
      this.cerrarModalBorrar.nativeElement.click();
      this.recargar();
    },
    error: () => this.error.set('No se pudo borrar (¿tiene matrículas?)')
  });
}
}

```

- En ~/SpringBoot_Angular/frontendalumnos/src/app/componentes/alumno-lista/alumno-lista.html

```

<!-- src/app/componentes/alumno-lista/alumno-lista.html -->

<!-- Cabecera -->
<div class="d-flex justify-content-between align-items-center mb-3">
  <div>
    <h3 class="mb-0">Alumnos</h3>
    <span class="text-secondary">{{ filtrados().length }} alumnos encontrados</span>
  </div>
  <!-- Abre el modal del formulario en modo CREAR -->
  <button class="btn btn-primary" (click)="nuevoAlumno()"
    data-bs-toggle="modal" data-bs-target="#modalAlumno">
    + Nuevo alumno
  </button>
</div>

@if (error()) {
  <div class="alert alert-danger">{{ error() }}</div>
}

<!-- Buscador -->
<div class="input-group mb-3" style="max-width: 420px;">
  <span class="input-group-text"><img alt="search icon" data-bbox="350 350 365 365"/></span>
  <input type="text" class="form-control"
    placeholder="Buscar por nombre o NIF.."
    [value]="busqueda()"
    (input)="buscar(campoBusqueda.value)" #campoBusqueda>
</div>

<!-- Pestañas por género, con contadores -->
<ul class="nav nav-pills mb-3">
  <li class="nav-item">
    <button class="nav-link" [class.active]="filtroGenero() === 'todos'"
      (click)="cambiarFiltro('todos')">Todos ({{ alumnos().length }})</button>
  </li>
  <li class="nav-item">
    <button class="nav-link" [class.active]="filtroGenero() === 'hombres'"
      (click)="cambiarFiltro('hombres')">Hombres ({{ nHombres() }})</button>
  </li>
  <li class="nav-item">
    <button class="nav-link" [class.active]="filtroGenero() === 'mujeres'"
      (click)="cambiarFiltro('mujeres')">Mujeres ({{ nMujeres() }})</button>
  </li>
</ul>

<!-- Tabla -->
<div class="card shadow-sm">
  <table class="table table-hover align-middle mb-0">
    <thead class="table-light">
      <tr>
        <th>NIF</th><th>NOMBRE</th><th>EDAD</th><th>GÉNERO</th>
        <th>MATRÍCULAS</th><th>NOTA MEDIA</th><th>ACCIONES</th>
      </tr>
    </thead>
    <tbody>
      @if (cargando()) {
        <tr><td colspan="7" class="text-center p-4">
          <div class="spinner-border text-primary"></div>
        </td></tr>
      }
      @for (a of paginados(); track a.nif) {
        <tr>
          <td class="text-secondary">{{ a.nif }}</td>
          <td class="fw-semibold">{{ a.nombre }}</td>
          <td>{{ a.edad }}</td>
          <td>
            @if (a.genero) {
              <span class="badge bg-info-subtle text-info-emphasis">♀ Mujer</span>
            }
          </td>
          <td>
            <div class="float-right">
              <button class="btn btn-sm btn-primary">Editar</button>
              <button class="btn btn-sm btn-danger">Eliminar</button>
            </div>
          </td>
        </tr>
      }
    </tbody>
  </table>

```



```

    } @else {
      <span class="badge bg-warning-subtle text-warning-emphasis">♂ Hombre</span>
    }
  </td>
</td>{{ matriculasDe(a.nif) }}</td>
<td>
  <span class="badge" [class]="claseNota(notaMediaDe(a.nif))">
    @if (notaMediaDe(a.nif) !== null) {
      {{ notaMediaDe(a.nif) | number:'1.2-2' }}
    } @else { - }
  </span>
</td>
<td>
  <!-- Abre el MISMO modal del formulario, en modo EDITAR -->
  <button class="btn btn-sm btn-outline-primary me-1" (click)="editarAlumno(a)"
    data-bs-toggle="modal" data-bs-target="#modalAlumno">✎</button>
  <button class="btn btn-sm btn-outline-danger" (click)="pedirBorrado(a)"
    data-bs-toggle="modal" data-bs-target="#modalBorrar">🗑</button>
</td>
</tr>
} @empty {
  @if (!cargando()) {
    <tr><td colspan="7" class="text-center text-secondary p-4">Sin resultados</td></tr>
  }
}
</tbody>
</table>

<!-- Paginación -->
<div class="d-flex justify-content-between align-items-center p-2">
  <span class="text-secondary small ms-2">
    Página {{ pagina() }} de {{ totalPaginas() }}
  </span>
  <nav>
    <ul class="pagination pagination-sm mb-0 me-2">
      <li class="page-item" [class.disabled]="pagina() === 1">
        <button class="page-link" (click)="irAPagina(pagina() - 1)"><</button>
      </li>
      <li class="page-item" [class.disabled]="pagina() === totalPaginas()">
        <button class="page-link" (click)="irAPagina(pagina() + 1)">>></button>
      </li>
    </ul>
  </nav>
</div>
</div>

<!-- 🗑 Modal del formulario (crear/editar) -->
<div class="modal fade" id="modalAlumno" tabindex="-1">
  <div class="modal-dialog">
    <div class="modal-content">
      <form [formGroup]="form" (ngSubmit)="guardar()">
        <div class="modal-header">
          <h5 class="modal-title">
            {{ modoEdicion() ? '✎ Editar alumno' : '+ Nuevo alumno' }}
          </h5>
          <button type="button" class="btn-close" data-bs-dismiss="modal"></button>
        </div>
        <div class="modal-body">
          <div class="mb-3">
            <label class="form-label">NIF</label>
            <input class="form-control" formControlName="nif" maxLength="9"
              [class.is-invalid]="form.get('nif')?.invalid && form.get('nif')?.touched">
            <div class="invalid-feedback">El NIF es obligatorio (máx. 9 caracteres)</div>
          </div>
          <div class="mb-3">
            <label class="form-label">Nombre</label>
            <input class="form-control" formControlName="nombre" maxLength="50"

```

```

        [class.is-invalid]="form.get('nombre')?.invalid && form.get('nombre')?.touched">
        <div class="invalid-feedback">El nombre es obligatorio</div>
      </div>
      <div class="row">
        <div class="col-6 mb-3">
          <label class="form-label">Edad</label>
          <input type="number" class="form-control" formControlName="edad">
        </div>
        <div class="col-6 mb-3">
          <label class="form-label">Género</label>
          <select class="form-select" formControlName="genero">
            <!-- ngValue conserva el tipo boolean (value lo pasaría a texto) -->
            <option [ngValue]="false">Hombre</option>
            <option [ngValue]="true">Mujer</option>
          </select>
        </div>
      </div>
      <div class="modal-footer">
        <button type="button" class="btn btn-secondary" data-bs-dismiss="modal"
          #cerrarModalForm>Cancelar</button>
        <button type="submit" class="btn btn-primary" [disabled]="guardando()"
          {{ guardando() ? 'Guardando...' : 'Guardar' }}>
        </button>
      </div>
    </form>
  </div>
</div>
</div>

<!-- 🗑 Modal de confirmación de borrado -->
<div class="modal fade" id="modalBorrar" tabindex="-1">
  <div class="modal-dialog modal-sm">
    <div class="modal-content">
      <div class="modal-header">
        <h5 class="modal-title">Confirmar borrado</h5>
        <button type="button" class="btn-close" data-bs-dismiss="modal"></button>
      </div>
      <div class="modal-body">
        ¿Borrar al alumno <b>{{ nifABorrar()?.nombre }}</b> ({{ nifABorrar()?.nif }})?
      </div>
      <div class="modal-footer">
        <button type="button" class="btn btn-secondary" data-bs-dismiss="modal"
          #cerrarModalBorrar>Cancelar</button>
        <button type="button" class="btn btn-danger" (click)="confirmarBorrado()">Borrar</button>
      </div>
    </div>
  </div>
</div>
</div>

```

El CSS del componente queda vacío (todo lo hace Bootstrap):

```

/* src/app/componentes/alumno-lista/alumno-lista.css */
/* Vacío intencionadamente: los estilos son 100% Bootstrap */

```

🔴 **Observa el doble uso del mismo modal** #modalAlumno : el botón + *Nuevo alumno* llama a `nuevoAlumno()` (resetea el formulario) y el lápiz de cada fila a `editarAlumno(a)` (lo rellena con `patchValue` y bloquea el NIF con `disable()`). Al leer los valores usamos `getRawValue()` en vez de `.value` porque este último **excluye los controles deshabilitados**.



7.6. Pantalla Asignaturas

[Tabla de contenidos](#)

Misma receta que en Alumnos (reescritura completa de `asignatura-lista`), pero más corta: buscador + pestañas por **curso**, columnas de **horas**, **aulas** (badges), **alumnos** y **nota media** (del endpoint `resumen-asignaturas`), y modal de alta/edición que incluye las aulas como CSV (reutilizando el endpoint `/id/aulas` de 6.1).

- En `~/SpringBoot_Angular/frontendalumnos/src/app/componentes/asignatura-lista/asignatura-lista.ts`

```

// src/app/componentes/asnatura-lista/asnatura-lista.ts
import { Component, OnInit, signal, computed, inject, ViewChild, ElementRef } from '@angular/core';
import { DecimalPipe } from '@angular/common';
import { ReactiveFormsModule, FormBuilder, Validators } from '@angular/forms';
import { finalize, switchMap } from 'rxjs';

// Modelos y servicios
import { Asignatura } from '../../../modelos/asignatura.model';
import { AsignaturaResumen } from '../../../modelos/resumen.model';
import { AsignaturaService } from '../../../servicios/asignatura-service';
import { MatriculaService } from '../../../servicios/matricula-service';

@Component({
  selector: 'app-asignatura-lista',
  imports: [ReactiveFormsModule, DecimalPipe],
  templateUrl: './asnatura-lista.html',
  styleUrls: ['./asnatura-lista.css']
})
export class AsignaturaLista implements OnInit {
  // — Datos base —
  asignaturas = signal<Asignatura[]>([]);
  resumen = signal<Map<number, AsignaturaResumen>>(new Map()); // id → resumen
  error = signal<string | null>(null);
  guardando = signal<boolean>(false);

  // — Filtros —
  busqueda = signal<string>('');
  filtroCurso = signal<number | 'todos'>('todos');

  // — Formulario del modal —
  private fb = inject(FormBuilder);
  modoEdicion = signal<boolean>(false);
  idOriginal: number | null = null;
  aBorrar = signal<Asignatura | null>(null);

  form = this.fb.group({
    denominacion: ['', [Validators.required, Validators.maxLength(50)]],
    curso: [1, Validators.required],
    horas: [0, [Validators.required, Validators.min(0)]],
    // Las aulas se editan como texto CSV: "1.1, 2.2"
    aulasCSV: ['']
  });

  @ViewChild('cerrarModalForm') cerrarModalForm!: ElementRef<HTMLButtonElement>;
  @ViewChild('cerrarModalBorrar') cerrarModalBorrar!: ElementRef<HTMLButtonElement>;

  constructor(private asignaturaService: AsignaturaService,
    private matriculaService: MatriculaService) {}

  ngOnInit(): void {
    this.recargar();
  }

  recargar(): void {
    this.asignaturaService.getAsignaturas().subscribe({
      next: datos => this.asignaturas.set(datos),
      error: () => this.error.set('No se pudo conectar con el backend')
    });
    this.matriculaService.resumenAsignaturas().subscribe(lista =>
      this.resumen.set(new Map(lista.map(r => [r.id, r])))
    );
  }

  // — Computed: filtros y pestañas —
  filtradas = computed(() => {
    const texto = this.busqueda().toLowerCase().trim();

```

```

    return this.asignaturas().filter(a => {
      const pasaCurso = this.filtroCurso() === 'todos' || a.curso === this.filtroCurso();
      const pasaTexto = texto === '' || a.denominacion.toLowerCase().includes(texto);
      return pasaCurso && pasaTexto;
    });
  });

  // Lista de cursos distintos, para pintar una pestaña por curso
  cursos = computed(() =>
    [...new Set(this.asignaturas().map(a => a.curso))]
      .filter((c): c is number => c !== null)
      .sort()
  );

  alumnosDe(id: number): number {
    return this.resumen().get(id)?.totalAlumnos ?? 0;
  }

  notaMediaDe(id: number): number | null {
    return this.resumen().get(id)?.notaMedia ?? null;
  }

  claseNota(nota: number | null): string {
    if (nota === null) return 'bg-secondary-subtle text-secondary-emphasis';
    if (nota >= 7) return 'bg-success-subtle text-success-emphasis';
    if (nota >= 5) return 'bg-warning-subtle text-warning-emphasis';
    return 'bg-danger-subtle text-danger-emphasis';
  }

  // — Modal crear/editar —————

  nuevaAsignatura(): void {
    this.modaEdicion.set(false);
    this.idOriginal = null;
    this.form.reset({ denominacion: '', curso: 1, horas: 0, aulasCSV: '' });
  }

  editarAsignatura(a: Asignatura): void {
    this.modaEdicion.set(true);
    this.idOriginal = a.id;
    this.form.patchValue({
      denominacion: a.denominacion,
      curso: a.curso ?? 1,
      horas: a.horas,
      aulasCSV: a.aulas.join(', ') // number[] -> "1.1, 2.2"
    });
  }

  guardar(): void {
    if (this.form.invalid) { this.form.markAllAsTouched(); return; }
    this.guardando.set(true);

    const v = this.form.getRawValue();
    const datos = { denominacion: v.denominacion!, curso: v.curso!, horas: v.horas! };
    const csv = (v.aulasCSV ?? '').trim();

    // 1º guardamos la asignatura; 2º (si hay CSV) definimos sus aulas.
    // switchMap encadena la 2ª petición usando la respuesta de la 1ª (su id)
    const peticion = this.modaEdicion()
      ? this.asignaturaService.actualizarAsignatura(this.idOriginal!, datos)
      : this.asignaturaService.crearAsignatura(datos);

    peticion.pipe(
      switchMap(asig => csv !== ''
        ? this.asignaturaService.definirAulasAsignaturas(asig.id, csv)
        : [asig]), // sin CSV: seguimos con la asignatura tal cual
      finalize(() => this.guardando.set(false))
    ).subscribe({

```

```

        next: () => {
            this.cerrarModalForm.nativeElement.click();
            this.recargar();
        },
        error: () => this.error.set('No se pudo guardar la asignatura')
    });
}

// — Modal de borrado —————

pedirBorrado(a: Asignatura): void {
    this.aBorrar.set(a);
}

confirmarBorrado(): void {
    const a = this.aBorrar();
    if (!a) return;
    this.asignaturaService.borrarAsignatura(a.id).subscribe({
        next: () => {
            this.cerrarModalBorrar.nativeElement.click();
            this.recargar();
        },
        error: () => this.error.set('No se pudo borrar (¿tiene alumnos matriculados?)')
    });
}
}

```

- En ~/SpringBoot_Angular/frontendalumnos/src/app/componentes/asignatura-lista/asignatura-lista.html

```

<!-- src/app/componentes/asnignatura-lista/asnignatura-lista.html -->

<!-- Cabecera -->
<div class="d-flex justify-content-between align-items-center mb-3">
  <div>
    <h3 class="mb-0">Asignaturas</h3>
    <span class="text-secondary">{{ filtradas().length }} asignaturas</span>
  </div>
  <button class="btn btn-primary" (click)="nuevaAsignatura()"
    data-bs-toggle="modal" data-bs-target="#modalAsignatura">
    + Nueva asignatura
  </button>
</div>

@if (error()) {
  <div class="alert alert-danger">{{ error() }}</div>
}

<!-- Buscador -->
<div class="input-group mb-3" style="max-width: 420px;">
  <span class="input-group-text"><img alt="search icon" data-bbox="350 338 365 350"/></span>
  <input type="text" class="form-control" placeholder="Buscar asignatura..."
    [value]="busqueda()" (input)="busqueda.set(campo.value)" #campo>
</div>

<!-- Pestañas por curso (se generan según los cursos existentes) -->
<ul class="nav nav-pills mb-3">
  <li class="nav-item">
    <button class="nav-link" [class.active]="filtroCurso() === 'todos'"
      (click)="filtroCurso.set('todos')>Todos</button>
  </li>
  @for (c of cursos(); track c) {
    <li class="nav-item">
      <button class="nav-link" [class.active]="filtroCurso() === c"
        (click)="filtroCurso.set(c)">Curso {{ c }}</button>
    </li>
  }
</ul>

<!-- Tabla -->
<div class="card shadow-sm">
  <table class="table table-hover align-middle mb-0">
    <thead class="table-light">
      <tr>
        <th>ID</th><th>DENOMINACIÓN</th><th>CURSO</th><th>HORAS</th>
        <th>AULAS</th><th>ALUMNOS</th><th>NOTA MEDIA</th><th>ACCIONES</th>
      </tr>
    </thead>
    <tbody>
      @for (a of filtradas(); track a.id) {
        <tr>
          <td class="text-secondary">{{ a.id }}</td>
          <td class="fw-semibold">{{ a.denominacion }}</td>
          <td><span class="badge bg-primary-subtle text-primary-emphasis">{{ a.curso ?? '-' }}</span></td>
          <td>{{ a.horas }} h</td>
          <td>
            @for (aula of a.aulas; track $index) {
              <span class="badge bg-light text-dark border me-1">{{ aula }}</span>
            } @empty {
              <span class="text-secondary small">Sin aulas</span>
            }
          </td>
          <td>{{ alumnosDe(a.id) }}</td>
          <td>
            <span class="badge" [class]="claseNota(notaMediaDe(a.id))">
              @if (notaMediaDe(a.id) !== null) {
                {{ notaMediaDe(a.id) | number:'1.2-2' }}
              }
            </span>
          </td>
        </tr>
      }
    </tbody>
  </table>

```

```

    } @else { - }
  </span>
</td>
<td>
  <button class="btn btn-sm btn-outline-primary me-1" (click)="editarAsignatura(a)"
    data-bs-toggle="modal" data-bs-target="#modalAsignatura">✎</button>
  <button class="btn btn-sm btn-outline-danger" (click)="pedirBorrado(a)"
    data-bs-toggle="modal" data-bs-target="#modalBorrarAsig">🗑️</button>
</td>
</tr>
} @empty {
  <tr><td colspan="8" class="text-center text-secondary p-4">Sin resultados</td></tr>
}
</tbody>
</table>
</div>

<!-- 📄 Modal crear/editar -->
<div class="modal fade" id="modalAsignatura" tabindex="-1">
  <div class="modal-dialog">
    <div class="modal-content">
      <form [formGroup]="form" (ngSubmit)="guardar()">
        <div class="modal-header">
          <h5 class="modal-title">
            {{ modoEdicion() ? '✎ Editar asignatura' : '+ Nueva asignatura' }}
          </h5>
          <button type="button" class="btn-close" data-bs-dismiss="modal"></button>
        </div>
        <div class="modal-body">
          <div class="mb-3">
            <label class="form-label">Denominación</label>
            <input class="form-control" formControlName="denominacion" maxlength="50"
              [class.is-invalid]="form.get('denominacion')?.invalid && form.get('denominacion')?.touched">
            <div class="invalid-feedback">La denominación es obligatoria</div>
          </div>
          <div class="row">
            <div class="col-6 mb-3">
              <label class="form-label">Curso</label>
              <input type="number" class="form-control" formControlName="curso" min="1" max="9">
            </div>
            <div class="col-6 mb-3">
              <label class="form-label">Horas</label>
              <input type="number" class="form-control" formControlName="horas" min="0">
            </div>
          </div>
          <div class="mb-3">
            <label class="form-label">Aulas (CSV)</label>
            <input class="form-control" formControlName="aulasCSV" placeholder="1.1, 2.2">
            <div class="form-text">Separadas por comas, con punto decimal</div>
          </div>
        </div>
        <div class="modal-footer">
          <button type="button" class="btn btn-secondary" data-bs-dismiss="modal"
            #cerrarModalForm>Cancelar</button>
          <button type="submit" class="btn btn-primary" [disabled]="guardando()">
            {{ guardando() ? 'Guardando...' : 'Guardar' }}
          </button>
        </div>
      </form>
    </div>
  </div>
</div>

<!-- 📄 Modal de confirmación de borrado -->
<div class="modal fade" id="modalBorrarAsig" tabindex="-1">
  <div class="modal-dialog modal-sm">
    <div class="modal-content">

```



```

<div class="modal-header">
  <h5 class="modal-title">Confirmar borrado</h5>
  <button type="button" class="btn-close" data-bs-dismiss="modal"></button>
</div>
<div class="modal-body">
  ¿Borrar la asignatura <b>{{ aBorrar()?.denominacion }}</b>?
</div>
<div class="modal-footer">
  <button type="button" class="btn btn-secondary" data-bs-dismiss="modal"
    #cerrarModalBorrar>Cancelar</button>
  <button type="button" class="btn btn-danger" (click)="confirmarBorrado()">Borrar</button>
</div>
</div>
</div>

```

```

/* src/app/componentes/asignatura-lista/asignatura-lista.css */
/* Vacío intencionadamente: los estilos son 100% Bootstrap */

```

**Tip**

El operador `switchMap` de RxJS encadena dos peticiones **dependientes**: primero el POST/PUT de la asignatura, y con el `id` de la respuesta, el PUT de sus aulas. Sin él acabaríamos en el temido *callback hell* de `subscribe` dentro de `subscribe`.



7.7. Pantalla Matrículas

[Tabla de contenidos](#)

La pantalla más interactiva: a la izquierda el formulario de **nueva matrícula** (busco un alumno, elijo asignatura, fecha y nota opcional) y a la derecha el **historial** del alumno seleccionado, con edición de nota *inline* y botón de desmatricular.

Generamos el componente:

```
cd ~/SpringBoot_Angular/frontendalumnos/src/app/  
ng g c componentes/matriculas
```

- En ~/SpringBoot_Angular/frontendalumnos/src/app/componentes/matriculas/matriculas.ts

```

// src/app/componentes/matriculas/matriculas.ts
import { Component, OnInit, signal, computed, inject } from '@angular/core';
import { DatePipe, DecimalPipe } from '@angular/common';
import { ReactiveFormsModule, FormBuilder, Validators } from '@angular/forms';
import { finalize } from 'rxjs';

// Modelos y servicios
import { Alumno } from '../../../modelos/alumno.model';
import { Asignatura } from '../../../modelos/asignatura.model';
import { AlumnoAsignatura } from '../../../modelos/alumno-asignatura.model';
import { AsignaturaResumen } from '../../../modelos/resumen.model';
import { AlumnoService } from '../../../servicios/alumno-service';
import { AsignaturaService } from '../../../servicios/asignatura-service';
import { MatriculaService } from '../../../servicios/matricula-service';

@Component({
  selector: 'app-matriculas',
  imports: [ReactiveFormsModule, DatePipe, DecimalPipe],
  templateUrl: './matriculas.html',
  styleUrls: ['./matriculas.css']
})
export class Matriculas implements OnInit {
  // — Datos base —————
  alumnos = signal<Alumno[]>([]);
  asignaturas = signal<Asignatura[]>([]);
  resumen = signal<Map<number, AsignaturaResumen>>(new Map());

  // — Estado de la pantalla —————
  busquedaAlumno = signal<string>('');
  alumnoSel = signal<Alumno | null>(null); // alumno seleccionado
  historial = signal<AlumnoAsignatura[]>([]); // sus matrículas
  editandoId = signal<number | null>(null); // matrícula con nota en edición
  mensaje = signal<string | null>(null); // feedback de éxito/error
  guardando = signal<boolean>(false);

  // — Formulario de nueva matrícula —————
  private fb = inject(FormBuilder);
  form = this.fb.group({
    idAsignatura: [null as number | null, Validators.required],
    // La fecha por defecto es HOY en formato ISO (lo que espera el input date)
    fecha: [new Date().toISOString().substring(0, 10), Validators.required],
    nota: [null as number | null, [Validators.min(0), Validators.max(9.99)]]
  });

  constructor(private alumnoService: AlumnoService,
    private asignaturaService: AsignaturaService,
    private matriculaService: MatriculaService) {}

  ngOnInit(): void {
    this.alumnoService.getAlumnos().subscribe(a => this.alumnos.set(a));
    this.asignaturaService.getAsignaturas().subscribe(a => this.asignaturas.set(a));
    this.recargarResumen();
  }

  recargarResumen(): void {
    this.matriculaService.resumenAsignaturas().subscribe(lista =>
      this.resumen.set(new Map(lista.map(r => [r.id, r])))
    );
  }

  // — Búsqueda y selección de alumno —————

  // Solo se muestran resultados a partir de 2 caracteres (máx. 5)
  candidatos = computed(() => {
    const texto = this.busquedaAlumno().toLowerCase().trim();
    if (texto.length < 2) return [];
    return this.alumnos()

```

```

        .filter(a => a.nombre.toLowerCase().includes(texto)
            || a.nif.toLowerCase().includes(texto))
        .slice(0, 5);
    });

    seleccionar(a: Alumno): void {
        this.alumnoSel.set(a);
        this.búsquedaAlumno.set(''); // limpia el buscador (oculta la lista)
        this.cargarHistorial();
    }

    cargarHistorial(): void {
        const a = this.alumnoSel();
        if (!a) return;
        this.matriculaService.getMatriculasDeAlumno(a.nif)
            .subscribe(h => this.historial.set(h));
    }

    // — Resumen del historial (parte baja de la tarjeta derecha) —

    notasDelHistorial = computed(() =>
        this.historial()
            .map(m => m.notaFinal)
            .filter((n): n is number => n !== null)
    );
    mediaHistorial = computed(() => {
        const notas = this.notasDelHistorial();
        if (notas.length === 0) return null;
        return notas.reduce((suma, n) => suma + n, 0) / notas.length;
    });
    mejorNota = computed(() => {
        const notas = this.notasDelHistorial();
        return notas.length > 0 ? Math.max(...notas) : null;
    });

    // Info de la asignatura elegida en el formulario (caja informativa)
    asigSeleccionada = computed(() => {
        // Convertimos a Number: el <select> devuelve el value como texto
        const id = Number(this.form.get('idAsignatura')?.value);
        return this.asignaturas().find(a => a.id === id) ?? null;
    });
    infoAsigSeleccionada = computed(() => {
        const a = this.asigSeleccionada();
        return a ? this.resumen().get(a.id) ?? null : null;
    });

    // — Acciones —

    matricular(): void {
        const alumno = this.alumnoSel();
        if (!alumno) { this.mensaje.set('⚠ Selecciona primero un alumno'); return; }
        if (this.form.invalid) { this.form.markAllAsTouched(); return; }

        const v = this.form.getRawValue();
        this.guardando.set(true);
        this.matriculaService
            .matricular(alumno.nif, Number(v.idAsignatura), v.fecha!, v.nota)
            .pipe(finalize(() => this.guardando.set(false)))
            .subscribe({
                next: () => {
                    this.mensaje.set('✅ Matricula creada');
                    this.form.patchValue({ idAsignatura: null, nota: null });
                    this.cargarHistorial();
                    this.recargarResumen();
                },
                // El backend responde 400 si ya estaba matriculado
                error: () => this.mensaje.set('❌ No se pudo matricular (¿ya estaba matriculado?)')
            });
    }

```

```

    });
}

// Edición de nota inline: guardamos el id en edición y leemos el input
guardarNota(m: AlumnoAsignatura, valor: string): void {
    const nota = Number(valor);
    if (isNaN(nota) || nota < 0 || nota > 9.99) return;
    this.matriculaService.actualizarNota(m.id, nota).subscribe(() => {
        this.editandoId.set(null);
        this.cargarHistorial();
        this.recargarResumen();
    });
}

desmatricular(m: AlumnoAsignatura): void {
    const alumno = this.alumnoSel();
    if (!alumno) return;
    this.matriculaService.desmatricular(alumno.nif, m.asignatura.id).subscribe(() => {
        this.cargarHistorial();
        this.recargarResumen();
    });
}

claseNota(nota: number | null): string {
    if (nota === null) return 'bg-secondary-subtle text-secondary-emphasis';
    if (nota >= 7) return 'bg-success-subtle text-success-emphasis';
    if (nota >= 5) return 'bg-warning-subtle text-warning-emphasis';
    return 'bg-danger-subtle text-danger-emphasis';
}
}

```

- En ~/SpringBoot_Angular/frontendalumnos/src/app/componentes/matriculas/matriculas.html

```

<!-- src/app/componentes/matriculas/matriculas.html -->

<div class="mb-3">
  <h3 class="mb-0">Matriculas</h3>
  <span class="text-secondary">Matricular alumnos y consultar su historial</span>
</div>

@if (mensaje()) {
  <div class="alert alert-info py-2">{{ mensaje() }}</div>
}

<div class="row g-3">

  <!-- 📄 Tarjeta izquierda: nueva matrícula -->
  <div class="col-lg-5">
    <div class="card shadow-sm">
      <div class="card-header bg-white fw-bold">+ Nueva matrícula</div>
      <div class="card-body">

        <!-- Buscador de alumno -->
        <label class="form-label">Alumno</label>
        <input type="text" class="form-control"
          placeholder="Buscar por nombre o NIF (mín. 2 letras)..."
          [value]="busquedaAlumno()"
          (input)="busquedaAlumno.set(campo.value)" #campo>

        <!-- Lista de candidatos (aparece al escribir) -->
        <div class="list-group mt-1">
          @for (a of candidatos(); track a.nif) {
            <button type="button" class="list-group-item list-group-item-action"
              (click)="seleccionar(a)">
              {{ a.nombre }} <span class="text-secondary small">({{ a.nif }})</span>
            </button>
          }
        </div>

        <!-- Alumno seleccionado -->
        @if (alumnoSel(); as alumno) {
          <div class="alert alert-primary py-2 mt-2 mb-3">
            👤 <b>{{ alumno.nombre }}</b> ({{ alumno.nif }})
          </div>
        } @else {
          <div class="text-secondary small mt-2 mb-3">Ningún alumno seleccionado</div>
        }

        <!-- Formulario -->
        <form [formGroup]="form" (ngSubmit)="matricular()">
          <div class="mb-3">
            <label class="form-label">Asignatura</label>
            <select class="form-select" formControlName="idAsignatura">
              <option [ngValue]="null" disabled>– Elegir asignatura –</option>
              @for (a of asignaturas(); track a.id) {
                <option [ngValue]="a.id">{{ a.denominacion }} ({{ a.curso }})</option>
              }
            </select>
          </div>

          <!-- Caja informativa de la asignatura elegida -->
          @if (asigSeleccionada(); as asig) {
            <div class="border rounded p-2 mb-3 bg-light small">
              📖 <b>{{ asig.denominacion }}</b> · {{ asig.horas }} h ·
              aulas: {{ asig.aulas.length > 0 ? asig.aulas.join(', ') : '-' }}
              @if (infoAsigSeleccionada(); as info) {
                <br>👥 {{ info.totalAlumnos }} alumnos matriculados
                @if (info.notaMedia !== null) {
                  · nota media {{ info.notaMedia | number:'1.2-2' }}
                }
              }
            </div>
          }
        </form>
      </div>
    </div>
  </div>

```

```

    }
  </div>
}

<div class="row">
  <div class="col-7 mb-3">
    <label class="form-label">Fecha</label>
    <input type="date" class="form-control" formControlName="fecha">
  </div>
  <div class="col-5 mb-3">
    <label class="form-label">Nota <span class="text-secondary">(opc.)</span></label>
    <input type="number" step="0.01" min="0" max="9.99"
      class="form-control" formControlName="nota">
  </div>
</div>

<button type="submit" class="btn btn-primary w-100" [disabled]="guardando()">
  {{ guardando() ? 'Guardando...' : 'Matricular' }}
</button>
</form>
</div>
</div>
</div>

<!-- 2 Tarjeta derecha: historial del alumno -->
<div class="col-lg-7">
  <div class="card shadow-sm">
    <div class="card-header bg-white fw-bold">
      Historial
      @if (alumnoSel(); as alumno) { - {{ alumno.nombre }} }
    </div>

    @if (!alumnoSel()) {
      <div class="card-body text-secondary">
        Selecciona un alumno para ver sus matrículas.
      </div>
    } @else {
      <table class="table table-hover align-middle mb-0">
        <thead class="table-light">
          <tr><th>Asignatura</th><th>Fecha</th><th>Nota</th><th>Descuento</th><th></th></tr>
        </thead>
        <tbody>
          @for (m of historial(); track m.id) {
            <tr>
              <td>
                {{ m.asignatura.denominacion }}
                <span class="badge bg-primary-subtle text-primary-emphasis">{{ m.asignatura.curso }}</span>
              </td>
              <td>{{ m.fechaMatricula | date:'dd/MM/yyyy' }}</td>
              <td>
                <!-- Nota: badge o input inline si está en edición -->
                @if (editandoId() === m.id) {
                  <input type="number" step="0.01" min="0" max="9.99"
                    class="form-control form-control-sm" style="width: 90px;"
                    [value]="m.notaFinal" #campoNota
                    (keyup.enter)="guardarNota(m, campoNota.value)">
                } @else {
                  <span class="badge" [class]="claseNota(m.notaFinal)">
                    {{ m.notaFinal !== null ? (m.notaFinal | number:'1.2-2') : 'Sin nota' }}
                  </span>
                }
              </td>
              <td class="small text-secondary">{{ m.descuento ?? 'NINGUNO' }}</td>
              <td class="text-end">
                @if (editandoId() === m.id) {
                  <button class="btn btn-sm btn-outline-secondary"
                    (click)="editandoId.set(null)">✖</button>
                }
              </td>
            </tr>
          }
        </tbody>
      </table>
    }
  </div>

```

```

        } @else {
            <button class="btn btn-sm btn-outline-primary me-1" title="Editar nota (Enter para guardar)"
                (click)="editandoId.set(m.id)"></button>
            <button class="btn btn-sm btn-outline-danger" title="Desmatricular"
                (click)="desmatricular(m)"></button>
        }
    </td>
</tr>
} @empty {
    <tr><td colspan="5" class="text-center text-secondary p-3">Sin matriculas</td></tr>
}
</tbody>
</table>

<!-- Resumen del alumno -->
<div class="card-footer bg-white d-flex gap-4 small">
    <span><b>{{ historial().length }}</b> matriculas</span>
    <span><b>Media:</b>
        <b>{{ mediaHistorial() != null ? (mediaHistorial() | number:'1.2-2') : '-' }}</b>
    </span>
    <span><b>Mejor nota:</b>
        <b>{{ mejorNota() != null ? (mejorNota() | number:'1.2-2') : '-' }}</b>
    </span>
</div>
}
</div>
</div>
</div>

```

```

/* src/app/componentes/matriculas/matriculas.css */
/* Vacío intencionadamente: los estilos son 100% Bootstrap */

```



Note

Dos detalles finos de la plantilla: `@if (alumnoSel()); as alumno` guarda el valor de la señal en la variable local `alumno` (así no hay que llamarla varias veces), y `(keyup.enter)` en el input de nota permite guardar pulsando Enter, leyendo el valor con la referencia local `#campoNota`.



7.8. Prueba final

Tabla de contenidos

Arrancamos las dos mitades y recorremos la SPA completa:

```
# Terminal 1 - Backend
export JAVA_HOME=/usr/lib/jvm/jdk-25.0.2-oracle-x64
cd ~/SpringBoot_Angular/crudalumnos
./mvnw clean spring-boot:run

# Terminal 2 - Frontend
cd ~/SpringBoot_Angular/frontendalumnos
ng serve -o
```

Checklist de comprobación (en `http://localhost:4200`):

#	Prueba	Resultado esperado
1	Entrar en <code>/</code>	Redirige a <code>/dashboard</code> con las 4 tarjetas rellenas
2	Dashboard → Alertas	Aparecen las matrículas sin nota de <code>DatosIniciales</code>
3	<code>/alumnos</code> → buscar "an"	Filtra a Angela (y cualquier NIF/nombre que contenga "an")
4	<code>/alumnos</code> → pestaña Mujeres	Solo Angela, contador (1)
5	<code>/alumnos</code> → + Nuevo alumno	El modal crea el alumno y la tabla se recarga
6	<code>/asignaturas</code> → editar	Cambiar horas/aulas y comprobar en MySQL
7	<code>/matriculas</code> → matricular	Seleccionar alumno, asignatura y guardar; aparece en el historial
8	<code>/matriculas</code> → repetir la misma	Mensaje de error (ya estaba matriculado)
9	Historial → nota + Enter	La nota se guarda y el badge cambia de color
10	Dashboard de nuevo	Los contadores y medias reflejan los cambios



Tip

Si una pantalla se queda vacía, abre las herramientas del navegador (F12) → pestaña *Network*. Ahí se ve cada petición al backend con su código de estado: un 404 delata una URL mal escrita en el servicio; un error CORS, que falta la configuración del apartado 5.4; y un *connection refused*, que el backend no está arrancado.

FASE 8: El Documento de Análisis (minicurso 25h)

Tabla de contenidos

Ya sabéis **construir** una aplicación FullStack completa. La última fase del curso trata de lo que ocurre **antes** de escribir la primera línea de código: el análisis. En septiembre cada grupo de 4 desarrollará su propio proyecto, y este minicurso de 25 horas os guía para entregar antes del **31 de julio de 2026** el `ANALISIS.md` que servirá de plano de ese desarrollo. El proyecto `crudalumnos` que acabamos de terminar es el ejemplo resuelto que usaremos en todas las sesiones.



Introducción al minicurso

Tabla de contenidos

Este minicurso de **25 horas** tiene un único objetivo: que cada grupo de 4 alumnos entregue antes del **31 de julio de 2026** un **Documento de Análisis** completo para su proyecto FullStack, listo para arrancar el desarrollo en septiembre.

El proyecto de referencia que usaremos como ejemplo en todas las sesiones es `crudalumnos`, la aplicación que hemos construido durante el curso principal. Conocéis cada decisión que se tomó en él: por qué hay una entidad intermedia en vez de `@ManyToMany`, por qué el PK de `Alumno` es un `VARCHAR` y no un `BIGINT` autogenerado, qué endpoints tiene cada controlador. Ese conocimiento es vuestro punto de partida.

El stack tecnológico está **bloqueado**: vuestro proyecto usará exactamente el mismo que `crudalumnos`.

Stack fijo del proyecto

Capa	Tecnología
Backend	Java 25 · Spring Boot 3.5 · Maven
Persistencia	Spring Data JPA · Hibernate · MySQL 8
Documentación API	Springdoc OpenAPI 2.8 (Swagger)
Frontend	Angular 21 (standalone)
Estilos	Bootstrap 5.3.8
IDE	Visual Studio Code



Important

El Documento de Análisis **no contiene código**. Contiene decisiones: qué construir, cómo estructurarlo y quién hace qué. El código llegará en septiembre.



El Documento de Análisis: estructura final

Tabla de contenidos

Antes de empezar a trabajar, conviene saber adónde vamos. Vuestro documento final tendrá estas secciones:

Sección	Bloque donde se trabaja
1. Descripción del sistema	Bloque 2
2. Requisitos funcionales	Bloque 2
3. Requisitos no funcionales	Bloque 2

Sección	Bloque donde se trabaja
4. Casos de uso	Bloque 2
5. Modelo de datos (E-R)	Bloque 3
6. API REST	Bloque 3
7. Diseño del Frontend (mockups + rutas)	Bloque 4
8. Gestión y planificación (GitHub + sprints)	Bloque 5

Al final del minicurso cada grupo hará una **defensa de 15 minutos** presentando su documento.



BLOQUE 1 — ¿Qué es un Documento de Análisis?

[Tabla de contenidos](#)

2 horas · 1 sesión



B1.S1 — Para qué sirve y cuándo se hace

[Tabla de contenidos](#)

¿Por qué se analiza antes de programar?

Programar sin analizar equivale a construir una casa sin planos. Puedes empezar a poner ladrillos, pero cuando llegues a la segunda planta descubrirás que no pusiste una viga donde la necesitabas.

En el desarrollo de software esto se traduce en:

- Reescribir entidades porque el modelo de datos estaba mal pensado
- Cambiar el nombre de los endpoints a mitad del desarrollo porque el frontend y el backend no se pusieron de acuerdo
- Conflictos en Git porque dos personas modificaron el mismo archivo sin saberlo
- No saber quién tiene que hacer qué en cada momento

El análisis resuelve todos esos problemas **antes** de abrir VSCode.

¿Cuándo se hace el análisis?

En el ciclo de vida de un proyecto software, el análisis ocupa la primera fase, antes del diseño técnico detallado y mucho antes del desarrollo.

```
Análisis → Diseño → Desarrollo → Pruebas → Despliegue
      ↑
Aquí estamos
```

El documento que vais a crear en este minicurso corresponde a la fase de **análisis** y a la parte inicial del **diseño** (modelo de datos, bocetos de UI, estructura de la API).

crudalumnos como ejemplo resuelto

Vamos a repasar las decisiones de análisis que se tomaron en `crudalumnos` y que ya conocéis:

Decisiones de modelo de datos:

Decisión	Alternativa descartada	Motivo
PK de <code>Alumno</code> es <code>VARCHAR nif</code>	<code>BIGINT</code> autogenerado	El NIF ya es un identificador natural único
Entidad intermedia <code>AlumnoAsignatura</code> explícita	<code>@ManyToMany</code> de JPA	Evita problemas de serialización JSON y permite añadir campos extra (nota, fecha)
<code>Boolean genero</code> → <code>TINYINT(1)</code>	<code>VARCHAR</code> con valores "H"/"M"	Ocupa menos espacio, es más eficiente en filtros

Decisión	Alternativa descartada	Motivo
<code>List<Float> aulas</code> con <code>@ElementCollection</code>	Tabla separada manual	JPA gestiona la tabla auxiliar <code>asignaturas_aulas</code> automáticamente

Decisiones de API:

Decisión	Motivo
Prefijo <code>/api/alumnos</code> para todos los endpoints de alumnos	Convención REST estándar, facilita el CORS y el proxy de Angular
GET <code>/consultar/nombre?q=...</code> con <code>@RequestParam</code>	Patrón barra de búsqueda: parámetro opcional en la URL
POST <code>/matricular/{nif}/{idAsignatura}</code>	La matrícula relaciona dos recursos ya existentes, no crea un recurso nuevo complejo

Decisiones de frontend:

Decisión	Motivo
Arquitectura standalone de Angular 21	Sin <code>NgModule</code> , menos archivos, arranque más rápido
<code>app.routes.ts</code> como fichero de rutas	Convención de proyectos standalone (no <code>app-routing.module.ts</code>)
Servicio por entidad (<code>AlumnoService</code> , <code>AsignaturaService</code>)	Separación de responsabilidades: el componente no llama directamente a la API

**Note**

Ninguna de estas decisiones es "la única correcta". Son las que **el equipo acordó** durante el análisis. Lo importante es que queden documentadas y que todo el equipo las conozca antes de empezar a programar.

Estructura del documento que vais a crear

Revisad la sección [El Documento de Análisis: estructura final](#) al inicio de este documento. Ese es vuestro mapa de ruta para las próximas semanas.

Tarea de esta sesión

Cada grupo debe acordar hoy:

1. El **nombre del proyecto** (ejemplo: `gestionbiblioteca`, `clinicaveterinaria`, `tiendaonline`)
2. Un **nombre corto** que se usará como prefijo en los paquetes Java y en el repositorio GitHub (ejemplo: `biblioteca`, `clinica`, `tienda`)
3. Quién asume cada **rol inicial** en el grupo:

Rol	Responsabilidad principal
Arquitecto/a de datos	Lidera el modelo E-R y los tipos de datos
Líder de API	Lidera el diseño de endpoints REST
Líder de Frontend	Lidera los mockups y la estructura Angular
Gestor/a de proyecto	Lidera el GitHub, el Kanban y las actas

**Tip**

Los roles son para organizar el liderazgo de cada sección del análisis. En el desarrollo de septiembre todos tocarán todas las capas.

BLOQUE 2 – Definición del proyecto y requisitos

[Tabla de contenidos](#)

 5 horas · 2 sesiones

B2.S1 – Descripción del sistema y requisitos funcionales

[Tabla de contenidos](#)

¿Qué es un requisito?

Un requisito es una capacidad que el sistema **debe tener** para satisfacer al usuario. Se dividen en:

- **Funcionales:** qué hace el sistema (registrar un alumno, buscar por nombre, matricular en una asignatura)
- **No funcionales:** cómo lo hace (tecnología, rendimiento, seguridad, usabilidad)

Descripción del sistema — ejemplo con crudalumnos

La descripción del sistema responde a tres preguntas: **qué es, para quién es y qué problema resuelve**.

Ejemplo de descripción — `crudalumnos` :

crudalumnos es una aplicación web de gestión académica para la Academia Arelance. Permite al personal administrativo gestionar el registro de alumnos, el catálogo de asignaturas y las matrículas entre ambos. El sistema sustituye a una hoja de cálculo Excel que resultaba difícil de mantener y no permitía múltiples usuarios simultáneos. La aplicación ofrece una interfaz web accesible desde cualquier navegador sin necesidad de instalación en el equipo del usuario.

Una buena descripción del sistema cabe en un párrafo. Si necesitáis más de tres párrafos, es que estáis describiendo la implementación, no el sistema.

Requisitos funcionales

Los requisitos funcionales se expresan como **el sistema debe permitir + acción + sobre qué**.

Organizadlos por entidad principal. Aquí el ejemplo de `crudalumnos` :

RF – Gestión de Alumnos

Código	Requisito
RF-A01	El sistema debe permitir registrar un alumno con NIF, nombre, edad y género
RF-A02	El sistema debe permitir consultar la lista completa de alumnos
RF-A03	El sistema debe permitir buscar alumnos por nombre (búsqueda parcial)
RF-A04	El sistema debe permitir filtrar alumnos por género
RF-A05	El sistema debe permitir modificar los datos de un alumno existente
RF-A06	El sistema debe permitir eliminar un alumno del sistema
RF-A07	El sistema debe mostrar el número de matrículas y la nota media de cada alumno

RF – Gestión de Asignaturas

Código	Requisito
RF-S01	El sistema debe permitir registrar una asignatura con denominación, curso y aulas
RF-S02	El sistema debe permitir consultar la lista de asignaturas con filtros por curso y aula

Código	Requisito
RF-S03	El sistema debe permitir modificar los datos de una asignatura
RF-S04	El sistema debe permitir eliminar una asignatura
RF-S05	El sistema debe mostrar el número de alumnos matriculados y la nota media por asignatura

RF – Gestión de Matriculas

Código	Requisito
RF-M01	El sistema debe permitir matricular un alumno en una asignatura
RF-M02	El sistema debe impedir matricular dos veces al mismo alumno en la misma asignatura
RF-M03	El sistema debe permitir registrar la fecha de matrícula y la nota final
RF-M04	El sistema debe permitir aplicar un descuento según el perfil del alumno (NINGUNO / DESEMPLEADO / FAMILIA_NUMEROSA / VULNERABLE)
RF-M05	El sistema debe permitir desmatricular a un alumno de una asignatura



Tip

Cada requisito funcional se convertirá en uno o varios endpoints de la API (Bloque 3) y en uno o varios elementos de la interfaz (Bloque 4). Cuanto más claro sea el requisito, más fácil será ese mapeo.



Tarea de esta sesión

Redactad la **Descripción del sistema** y la tabla de **Requisitos funcionales** de vuestro proyecto. Usad el formato de los ejemplos anteriores. Mínimo 15 requisitos funcionales distribuidos entre las entidades principales.



B2.S2 – Requisitos no funcionales y casos de uso

Tabla de contenidos

Requisitos no funcionales

Los requisitos no funcionales describen las restricciones y la calidad del sistema. En vuestro caso muchos están ya fijados por el stack tecnológico del curso.

Ejemplo de tabla para `crudalumnos`:

Código	Categoría	Requisito
RNF-01	Tecnología backend	Java 25 · Spring Boot 3.5 · Maven
RNF-02	Tecnología persistencia	MySQL 8 · Spring Data JPA · Hibernate
RNF-03	Tecnología frontend	Angular 21 standalone · Bootstrap 5.3.8
RNF-04	Documentación API	Swagger (Springdoc OpenAPI 2.8)
RNF-05	Entorno de desarrollo	Visual Studio Code
RNF-06	Control de versiones	Git · GitHub (repositorio único con ramas <code>main</code> / <code>develop</code>)
RNF-07	Compatibilidad	Navegadores modernos (Chrome, Firefox, Edge); sin instalación en cliente

Código	Categoría	Requisito
RNF-08	Usabilidad	Interfaz en español; navegación desde menú lateral siempre visible
RNF-09	Integridad de datos	No se puede matricular el mismo alumno dos veces en la misma asignatura
RNF-10	Arquitectura API	REST · JSON · HTTP status codes estándar (200, 201, 400, 404, 409)

**Note**

Los RNF-01 a RNF-06 son iguales para todos los proyectos del curso porque el stack está bloqueado. Los RNF-07 en adelante son específicos de cada proyecto.

Casos de uso

Un caso de uso describe cómo **un actor** interactúa con el sistema para conseguir **un objetivo**. En el nivel de análisis de este curso usamos una tabla simplificada, no diagramas UML completos.

Actores del sistema `crudaLumnos` :

- **Administrativo:** usuario principal, gestiona alumnos, asignaturas y matrículas
- **Sistema:** respuestas automáticas (validaciones, cálculos de nota media)

Ejemplo de tabla de casos de uso:

Código	Actor	Acción	Resultado esperado	RF relacionado
CU-01	Administrativo	Registra un nuevo alumno con NIF, nombre, edad y género	El alumno queda guardado; aparece en el listado	RF-A01
CU-02	Administrativo	Busca alumnos escribiendo "García" en la barra de búsqueda	Se filtra la lista mostrando solo alumnos cuyo nombre contiene "García"	RF-A03
CU-03	Administrativo	Intenta matricular a un alumno ya matriculado en la misma asignatura	El sistema rechaza la operación con un mensaje de error (409 Conflict)	RF-M02
CU-04	Administrativo	Registra una nota final de 8.50 en una matrícula	La nota queda guardada; la nota media del alumno y de la asignatura se actualiza	RF-M03
CU-05	Administrativo	Elimina un alumno que tiene matrículas activas	El sistema elimina también sus matrículas (cascade) o impide la eliminación, según la decisión del equipo	RF-A06
CU-06	Sistema	Al cargar la vista de Alumnos	Se muestra para cada alumno su número de matrículas y su nota media calculada desde la BBDD	RF-A07

**Important**

El CU-05 requiere una decisión de diseño: ¿cascade delete o error controlado? Documentad qué habéis decidido y por qué. Ese tipo de decisiones son exactamente para lo que sirve el análisis.

**Tarea de esta sesión**

Redactad la tabla de **Requisitos no funcionales** y la tabla de **Casos de uso** de vuestro proyecto. Mínimo 8 casos de uso, cubriendo al menos un caso de validación/error (como el CU-03 o el CU-05).

**BLOQUE 3 — Modelo de datos y API REST**

[Tabla de contenidos](#)



8 horas · 3 sesiones



B3.S1 — Entidades, atributos y tipos de datos

Tabla de contenidos

¿Qué es una entidad?

Una entidad es un concepto del dominio del problema que tiene identidad propia y que necesitamos almacenar. En `crudalumnos` las entidades son `Alumno` , `Asignatura` y `AlumnoAsignatura` (la matrícula).

Para identificar las entidades de vuestro proyecto, buscad los **sustantivos principales** en los requisitos funcionales. Son candidatas a entidad todas las cosas sobre las que el sistema "guarda datos".

Los 8 tipos de datos del curso

Durante el desarrollo de `crudalumnos` hemos trabajado con estos tipos. Vuestro proyecto debe usar **al menos 5** de ellos:

Tipo Java	Tipo SQL	Ejemplo en crudalumnos	Cuándo usarlo
<code>String</code>	<code>VARCHAR</code>	<code>Alumno.nif</code> , <code>Alumno.nombre</code>	Texto de longitud variable
<code>int</code> / <code>Integer</code>	<code>INT</code>	<code>Alumno.edad</code> , <code>Asignatura.curso</code>	Números enteros sin decimales
<code>Boolean</code> / <code>boolean</code>	<code>TINYINT(1)</code>	<code>Alumno.genero</code>	Valores sí/no, verdadero/falso
<code>Long</code> / <code>long</code>	<code>BIGINT</code>	<code>AlumnoAsignatura.id</code> (PK autogenerada)	IDs autogenerados, números grandes
<code>Float</code> / <code>float</code>	<code>FLOAT</code>	<code>Asignatura.aulas</code> (en <code>@ElementCollection</code>)	Números con decimales, coordenadas, ratios
<code>LocalDate</code>	<code>DATE</code>	<code>AlumnoAsignatura.fechaMatricula</code>	Fechas sin hora
<code>BigDecimal</code>	<code>DECIMAL(p,s)</code>	<code>AlumnoAsignatura.notaFinal</code> con <code>DECIMAL(4,2)</code>	Valores monetarios o notas que requieren precisión exacta
<code>enum</code>	<code>VARCHAR</code> (con <code>@Enumerated(STRING)</code>)	<code>AlumnoAsignatura.descuento</code> → <code>NINGUNO / DESEMPLEADO / FAMILIA_NUMEROSA / VULNERABLE</code>	Conjuntos cerrados de valores posibles

Caution

Si usáis un `enum` en Java, **siempre** anotadlo con `@Enumerated(EnumType.STRING)` . El valor por defecto (`ORDINAL`) guarda el índice numérico del enum en la base de datos: si algún día añadís o reordenáis los valores del enum, todos los datos históricos quedan corruptos.

Cómo documentar una entidad en el análisis

Para cada entidad documentad: nombre, tabla SQL, PK, atributos con tipo Java y tipo SQL, y restricciones relevantes.

Ejemplo completo — entidad `AlumnoAsignatura` de `crudalumnos` :

Entidad: `AlumnoAsignatura`

Atributo	Tipo Java	Tipo SQL	Restricciones	Notas
<code>id</code>	<code>Long</code>	<code>BIGINT</code>	PK · <code>AUTO_INCREMENT</code>	Generado automáticamente por la BBDD
<code>alumno</code>	<code>Alumno</code> (FK)	<code>VARCHAR(9)</code> → <code>alumno_nif</code>	NOT NULL · FK → <code>alumnos.nif</code>	<code>@ManyToOne</code> · <code>@JsonIgnore</code> para evitar recursión

Atributo	Tipo Java	Tipo SQL	Restricciones	Notas
asignatura	Asignatura (FK)	INT → asignatura_id	NOT NULL · FK → asignaturas.id	@ManyToOne
fechaMatricula	LocalDate	DATE	NOT NULL	Fecha en que se formalizó la matrícula
notaFinal	BigDecimal	DECIMAL(4,2)	Nullable · 0.00–10.00	@Column(precision=4, scale=2)
descuento	TipoDescuento (enum)	VARCHAR	NOT NULL · default NINGUNO	@Enumerated(EnumType.STRING)
(unicidad)	—	—	UNIQUE(alumno_nif, asignatura_id)	Un alumno no puede estar matriculado dos veces en la misma asignatura

**Note**

La restricción `UNIQUE` compuesta se añade en JPA con

`@Table(uniqueConstraints = @UniqueConstraint(columnNames = {"alumno_nif", "asignatura_id"}))` en la clase entidad. Esta restricción actúa como segunda línea de defensa: la primera es la validación en el controlador (`existeMatricula()`), pero si por algún motivo se saltan esa validación, la BBDD rechaza el duplicado igualmente.

Tarea de esta sesión

Documentad todas las entidades de vuestro proyecto usando la tabla del ejemplo. Para cada entidad:

- Nombre de la entidad y nombre de la tabla SQL
- Todos los atributos con tipo Java, tipo SQL y restricciones
- Justificación breve del tipo de PK elegido (natural como el NIF, o autogenerada)
- Si hay relaciones N:M, describid la entidad intermedia y qué campos extra añade

**B3.S2 — Diagrama E-R y relaciones****Tabla de contenidos****Tipos de relaciones en JPA**

Relación	Anotación JPA	Ejemplo en crudalumnos
Un alumno tiene muchas matrículas	@OneToMany en Alumno	List<AlumnoAsignatura> matriculas
Cada matrícula pertenece a un alumno	@ManyToOne en AlumnoAsignatura	Alumno alumno
Una asignatura tiene muchas matrículas	@OneToMany en Asignatura	(implícito)
Cada matrícula pertenece a una asignatura	@ManyToOne en AlumnoAsignatura	Asignatura asignatura
Una asignatura tiene múltiples aulas	@ElementCollection en Asignatura	List<Float> aulas

**Important**

En `crudalumnos` **no se usa** `@ManyToMany` directamente aunque la relación entre `Alumno` y `Asignatura` es N:M. Se define explícitamente la entidad intermedia `AlumnoAsignatura`. Haced lo mismo en vuestro proyecto: si tenéis una relación N:M, siempre cread la entidad intermedia a mano. Os permitirá añadir campos extra en esa relación (como `fechaMatricula` o `notaFinal`) y evitaréis los problemas de serialización JSON que genera `@ManyToMany`.

El diagrama E-R en Mermaid

El diagrama E-R de vuestro documento se escribirá en Mermaid, exactamente como en los apuntes del curso principal. La sintaxis que usaréis:

```

```mermaid
%%{init: {'theme': 'default', 'themeVariables': {'fontSize': '10px', 'fontFamily': 'Roboto'}}}%
erDiagram
 ENTIDAD_A {
 tipo campo PK
 tipo campo
 }
 ENTIDAD_B {
 tipo campo PK
 tipo campo FK
 }
 ENTIDAD_A ||--o{ ENTIDAD_B : "descripción"

```

Notación de cardinalidades en Mermaid:

Símbolo	Significado
	Exactamente uno
o	Cero o uno
	Uno o muchos
o {	Cero o muchos

Ejemplo – diagrama E-R completo de `crudalumnos` (versión de análisis, con todos los campos incluyendo pendientes):

```

```mermaid
%%{init: {'theme': 'default', 'themeVariables': {'fontSize': '10px', 'fontFamily': 'Roboto'}}}%
erDiagram
    alumnos {
        varchar nif PK
        varchar nombre
        int edad
        tinyint genero
    }
    asignaturas {
        int id PK
        varchar denominacion
        int curso
        int horas
    }
    asignaturas_aulas {
        int asignatura_id FK
        float aulas
    }
    alumnos_asignaturas {
        bigint id PK
        varchar alumno_nif FK
        int asignatura_id FK
        date fechaMatricula
        decimal notaFinal
        varchar descuento
    }
    docentes {
        bigint id PK
        varchar nombre
        varchar especialidad
        varchar email
    }

    alumnos ||--o{ alumnos_asignaturas : "tiene"
    asignaturas ||--o{ alumnos_asignaturas : "tiene"
    asignaturas ||--o{ asignaturas_aulas : "contiene"
    docentes ||--o{ asignaturas : "imparte"

```

 Tarea de esta sesión

Cread el **diagrama E-R en Mermaid** de vuestro proyecto. Debe incluir:

- Todas las entidades con sus atributos y tipos SQL
- Todas las relaciones con su cardinalidad y etiqueta descriptiva
- Si tenéis relaciones N:M, la entidad intermedia con sus campos extra

Pegad el diagrama directamente en vuestro Documento de Análisis.



B3.S3 — Diseño de la API REST

[Tabla de contenidos](#)

Principios REST que aplicamos en el curso

Principio	Ejemplo en crudalumnos
URL identifica el recurso, verbo HTTP identifica la acción	GET /api/alumnos/consultar VS DELETE /api/alumnos/borrar/{nif}
Prefijo común por controlador	/api/alumnos/... , /api/asignaturas/... , /api/matriculas/...
Código de respuesta HTTP significativo	200 OK , 404 Not Found , 400 Bad Request , 409 Conflict
Respuesta en JSON	Spring Boot con @RestController lo hace automáticamente
Sin estado en el servidor	Cada petición lleva toda la información necesaria en la URL o el body

Cómo documentar un endpoint en el análisis

Para cada endpoint documentad: verbo HTTP, ruta completa, descripción, parámetros de entrada y respuesta esperada.

Plantilla de tabla de endpoints:

Verbo	Ruta	Descripción	Entrada	Respuesta OK	Error
GET	/api/alumnos/consultar	Listar todos los alumnos	—	200 · List<Alumno> JSON	—
GET	/api/alumnos/consultar/{nif}	Buscar alumno por NIF	nif en ruta	200 · Alumno JSON	404 si no existe
GET	/api/alumnos/consultar/nombre	Buscar por nombre	?q=texto (@RequestParam)	200 · List<Alumno> JSON	—
POST	/api/alumnos/crear	Crear alumno	Body JSON con Alumno	201 · Alumno creado	400 si NIF ya existe
PUT	/api/alumnos/actualizar/{nif}	Actualizar alumno	nif en ruta + Body JSON	200 · Alumno actualizado	404 si no existe
DELETE	/api/alumnos/borrar/{nif}	Eliminar alumno	nif en ruta	200 · mensaje texto	404 si no existe
POST	/api/matriculas/matricular/{nif}/{idAsignatura}	Matricular alumno	nif + idAsignatura en ruta	200 · AlumnoAsignatura JSON	404 si no existe alguno · 400 si ya matriculado

Verbo	Ruta	Descripción	Entrada	Respuesta OK	Error
DELETE	/api/matriculas/desmatricular/{nif}/{idAsignatura}	Desmatricular alumno	nif + idAsignatura en ruta	200 · mensaje texto	404 si la matrícula no existe

**Note**

En el análisis no escribimos el código del controlador, pero sí decidimos **qué parámetros** recibe cada endpoint y **qué devuelve**. Cuando lleguéis al desarrollo en septiembre, estas tablas son vuestra especificación: el que hace el backend sabe exactamente qué tiene que implementar, y el que hace el frontend sabe exactamente qué puede llamar.

GET con @RequestParam vs datos en la ruta

Esta es una de las decisiones de diseño más frecuentes en la API:

Situación	Técnica	Ejemplo
Buscar por un campo opcional (barra de búsqueda)	@RequestParam en GET	GET /api/alumnos/consultar/nombre?q=García
Acceder a un recurso por su identificador	@PathVariable en la ruta	GET /api/alumnos/consultar/12345678A
Filtrar con múltiples criterios complejos	@RequestBody en POST	POST /api/alumnos/buscar con JSON { "nombre": "García", "genero": false }

En vuestro proyecto, para cada endpoint de búsqueda/filtrado, documentad cuál de los tres enfoques elegís y por qué.

Swagger como contrato de la API

En el desarrollo usaréis `@Operation(summary = "...")` en cada endpoint para que Swagger lo documente automáticamente. En el análisis, la tabla de endpoints que redactéis **es el borrador** de esa documentación Swagger.

**Tip**

Cuando tengáis dudas sobre si un endpoint tiene sentido, haceos esta pregunta: ¿Hay alguna pantalla del frontend que necesite llamar a este endpoint? Si la respuesta es no, probablemente el endpoint sobra. Si la respuesta es sí, aseguraos de que el endpoint está en la tabla.

**Tarea de esta sesión**

Documentad **todos los endpoints** de vuestra API usando la plantilla de tabla. Organizadlos en secciones por controlador. Mínimo:

- Un controlador por entidad principal
- CRUD completo (GET todos, GET por ID, POST, PUT, DELETE) para al menos 2 entidades
- Al menos un endpoint de búsqueda con `@RequestParam`
- Los endpoints de la entidad intermedia (si aplica)

**BLOQUE 4 — Diseño del Frontend**

[Tabla de contenidos](#)



4 horas · 1 sesión

**B4.S1 — Estructura Angular y mockups Bootstrap**

[Tabla de contenidos](#)

La estructura de carpetas Angular que usaréis

Igual que en `crudaLumnos`, vuestro proyecto frontend seguirá esta estructura:

```
frontendalumnos/    - nombre de nuestro proyecto (sin guiones)
src/
└─ app/
   │ │ modelos/           - interfaces TypeScript (una por entidad)
   │ │ │ └─ alumno.model.ts
   │ │ │ └─ asignatura.model.ts
   │ │ servicios/        - llamadas HTTP a la API (uno por entidad)
   │ │ │ └─ alumno-service.ts
   │ │ │ └─ asignatura-service.ts
   │ componentes/       - vistas (uno o varios por entidad)
   │ │ └─ alumno-lista/
   │ │ └─ alumno-form/
   │ │ └─ asignatura-lista/
   │ app.routes.ts      - tabla de rutas SPA
   │ app.config.ts     - providers y configuración
   └─ app.ts            - componente raíz (topbar + sidebar + router-outlet)
```

En el análisis, documentad esta estructura con los nombres reales de vuestro proyecto. Cada carpeta en `componentes/` corresponde a una ruta de la aplicación.

Tabla de rutas SPA

La tabla de rutas documenta qué URL muestra qué componente. En el análisis esta tabla es la especificación de `app.routes.ts`.

Ejemplo – `cruda1umnos` :

Ruta	Componente	Descripción
/	—	Redirige a <code>/alumnos</code>
/alumnos	AlumnoLista	Listado de alumnos con búsqueda y filtros
/alumnos/nuevo	AlumnoForm	Formulario de alta de alumno
/alumnos/:nif	AlumnoForm	Formulario de edición de alumno existente
/asignaturas	AsignaturaLista	Listado de asignaturas con filtros por curso y aula
/asignaturas/nuevo	AsignaturaForm	Formulario de alta de asignatura
/asignaturas/:id	AsignaturaForm	Formulario de edición de asignatura existente
/matriculas	MatriculaLista	Gestión de matrículas: selección de alumno y asignatura

Los mockups de referencia – crudalumnos

Los mockups que se han diseñado para `crudalumnos` son vuestra referencia visual. Estudiándolos podéis ver el patrón que se repite en cada vista:

Patrón de vista de listado (Alumnos / Asignaturas / Matrículas):

- Navbar lateral fijo con secciones: PRINCIPAL / ANALÍTICA / CONFIG
- Cabecera de sección: título + contador + botón "+ Nuevo"
- Barra de búsqueda + selectores de filtro (Bootstrap `form-control` + `form-select`)
- Pestañas de filtro rápido (Bootstrap `nav-tabs`)
- Tabla Bootstrap con columnas: datos principales + columnas calculadas (nº matrículas, nota media) + columna ACCIONES (🔗 editar · ✕ eliminar)
- Paginación al pie de la tabla

Patrón de formulario (alta / edición):

- Modal Bootstrap o panel lateral
- Campos del formulario con `form-control`
- Botones Guardar / Cancelar
- Mensaje de error en `alert alert-danger` si la validación falla

Patrón de panel de detalle (historial de un alumno en Matrículas):

- Aparece al seleccionar un alumno
- Muestra las matrículas del alumno en una tabla secundaria
- Resumen calculado al pie: asignaturas cursadas, nota media, mejor nota

Mockups de vuestro proyecto

Para cada vista principal de vuestro proyecto, incluid en el documento un mockup en texto estructurado. No hace falta diseño gráfico: una tabla ASCII o una descripción estructurada es suficiente para el análisis.

Ejemplo de mockup en texto para la vista de Listado de Alumnos de `crudalumnos` :

Academia Arelance · Gestión de Alumnos					
PRINCIPAL	Alumnos				
Dashboard	124 alumnos registrados		[+ Nuevo alumno]		
Alumnos ←					
Asignaturas	 Buscar por nombre o NIF... [Género ▼]				
Matrículas					
	[Todos (124)] [Hombres (67)] [Mujeres (57)]				
ANALÍTICA					
Estadísticas	NIF	NOMBRE	EDAD	GÉNERO	
Carga CSV	MATRÍCULAS	NOTA MEDIA	ACCIONES		
CONFIG	12345678A	García López, Ana	22	♀	3 8.20
Cursos					[x]
Aulas	87654321B	Martínez Ruiz, Pedro	25	♂	2 6.50
					[x]
	Mostrando 1-7 de 124 < 1 2 ... >				

Interfaz de los modelos TypeScript

En el análisis documentad también las **interfaces TypeScript** que necesitaréis, ya que son el "contrato" entre el servicio (que llama a la API) y el componente (que muestra los datos).

Ejemplo — interfaces de `crudalumnos` :

```
// modelos/alumno.model.ts
export interface Alumno {
  nif: string;
  nombre: string;
  edad: number;
  genero: boolean;
  // Campos calculados que devuelve la API (no están en la tabla directamente)
  totalMatriculas?: number;
  notaMedia?: number;
}

// modelos/asignatura.model.ts
export interface Asignatura {
  id: number;
  denominacion: string;
  curso: number;
  aulas: number[]; // List<Float> en backend → number[] en TypeScript
}

// modelos/alumno-asignatura.model.ts
export interface AlumnoAsignatura {
  id: number;
  asignatura: Asignatura; // objeto completo anidado (el alumno va con @JsonIgnore)
  fechaMatricula: string; // LocalDate en backend → string ISO en TypeScript
  notaFinal: number | null;
  descuento: 'NINGUNO' | 'DESEMPLEADO' | 'FAMILIA_NUMEROSA' | 'VULNERABLE';
}
```

**Tip**

Fijaos en el campo `descuento`: en TypeScript podemos expresar el enum como un tipo unión de strings literales. Esto nos da autocompletado en el IDE y detección de errores en tiempo de compilación de TypeScript, exactamente igual que el enum de Java en el backend.

Tarea de esta sesión

Documentad en vuestro Documento de Análisis:

1. La **estructura de carpetas** de vuestro proyecto Angular con los nombres reales
2. La **tabla de rutas** (`app.routes.ts` en borrador)
3. Un **mockup en texto** para cada vista principal (mínimo 3 vistas)
4. Las **interfaces TypeScript** de todos vuestros modelos

**BLOQUE 5 — Gestión del proyecto y trabajo en equipo**

[Tabla de contenidos](#)

6 horas · 2 sesiones

**B5.S1 — GitHub: repositorio, ramas y Kanban**

[Tabla de contenidos](#)

Configuración del repositorio

Cada grupo creará un **único repositorio GitHub** para todo el proyecto. Estructura recomendada:

```
nombre-proyecto/      ← repositorio en GitHub
├─ backend/           ← proyecto Spring Boot (aquí va crudalumnos equivalente)
│   └─ (estructura Maven estándar)
├─ frontend/          ← proyecto Angular
│   └─ (estructura Angular estándar)
├─ docs/              ← documentación del proyecto
│   ├── ANALISIS.md    ← vuestro Documento de Análisis (este archivo)
│   ├── actas/          ← actas de reuniones
│   │   └─ acta-001-YYYY-MM-DD.md
│   └─ decisiones/     ← registro de decisiones de diseño (ADR simplificados)
│       └─ ADR-001-pk-entidad-principal.md
└─ README.md           ← descripción del proyecto y cómo arrancarlo
```



Note

El Documento de Análisis (`ANALISIS.md`) vive en la carpeta `docs/` del repositorio. No es un archivo adjunto en un correo: es parte del repositorio y evoluciona con el proyecto a través de commits.

Estrategia de ramas

Usaréis una estrategia de ramas simplificada basada en **Git Flow**:

Rama	Propósito	Quién hace merge
<code>main</code>	Código estable y revisado. Solo se actualiza desde <code>develop</code>	El/la Gestor/a de proyecto (con PR aprobada)
<code>develop</code>	Integración continua. Aquí se fusionan las features terminadas	Cualquier miembro (con PR aprobada por otro)
<code>feature/nombre-feature</code>	Una funcionalidad en desarrollo. Sale de <code>develop</code> y vuelve a <code>develop</code>	El miembro que la desarrolla
<code>fix/nombre-bug</code>	Corrección de un error. Mismo ciclo que <code>feature/</code>	El miembro que lo corrige

Flujo de trabajo para desarrollar una funcionalidad:

```
# 1. Siempre partir de develop actualizado
git checkout develop
git pull origin develop

# 2. Crear rama de feature
git checkout -b feature/crud-alumnos-backend

# 3. Trabajar, hacer commits frecuentes con mensajes descriptivos
git add .
git commit -m "feat: añade entidad Alumno con JPA"
git commit -m "feat: añade AlumnoRepository con métodos de búsqueda"
git commit -m "feat: añade AlumnoController con CRUD básico"

# 4. Cuando esté terminado, subir y abrir Pull Request hacia develop
git push origin feature/crud-alumnos-backend
# → Abrir PR en GitHub → Pedir revisión a un compañero → Merge
```

Convención de mensajes de commit

Usad prefijos para que el historial sea legible:

Prefijo	Cuándo usarlo
<code>feat:</code>	Nueva funcionalidad
<code>fix:</code>	Corrección de un error

Prefijo	Cuándo usarlo
docs:	Cambios en documentación
refactor:	Cambio de código que no añade funcionalidad ni corrige error
test:	Añadir o corregir tests
chore:	Tareas de mantenimiento (dependencias, configuración)

Ejemplo: feat: añade endpoint GET /api/alumnos/consultar/nombre

GitHub Projects como tablero Kanban

Cread un **GitHub Project** (tipo Board) vinculado a vuestro repositorio con estas columnas:

Columna	Qué contiene
 Backlog	Todas las tareas identificadas, sin asignar aún
 En progreso	Tareas que alguien del equipo tiene asignadas y está trabajando
 En revisión	PR abierta, esperando revisión de un compañero
 Hecho	Tarea completada y mergeada en develop

Cada tarjeta del Kanban debe tener:

- Título descriptivo: [BACK] Entidad Alumno + repositorio
- Asignada a una persona
- Etiqueta de capa: backend , frontend , docs , bbdd
- Sprint al que pertenece



Important
El tablero Kanban no es un adorno. Es la herramienta con la que el Gestor/a de proyecto controla el avance y detecta bloqueos. Si una tarea lleva más de 3 días en "En progreso" sin avanzar, hay que hablar.

Actas de reunión

Cada reunión de equipo debe generar un acta en docs/actas/. Formato mínimo:

```
# Acta de reunión · Sesión 001
**Fecha:** 2026-09-15
**Asistentes:** Ana García, Pedro Martínez, Laura Sánchez, Iván López
**Duración:** 45 minutos

## Temas tratados
1. Revisión del sprint anterior
2. Planificación del sprint actual
3. Resolución de bloqueos

## Decisiones tomadas
- Se decide usar cascade DELETE en la relación Alumno → AlumnoAsignatura
- El frontend de Matriculas lo desarrolla Pedro; el backend lo desarrolla Ana

## Próxima reunión
2026-09-22 · misma hora
```

Tarea de esta sesión

1. Cread el **repositorio GitHub** con la estructura de carpetas descrita
2. Cread las ramas main y develop
3. Cread el **GitHub Project** con las 4 columnas del Kanban
4. Añadid al repositorio el borrador de vuestro ANALISIS.md como primer commit en develop

5. Redactad el `README.md` con: qué es el proyecto, el stack tecnológico y cómo arrancarlo en local (aunque el código aún no exista, ya sabéis los comandos del curso)



B5.S2 — Reparto de tareas y calendario de sprints

Tabla de contenidos

Reparto de tareas por capa

El desarrollo en septiembre se organizará por **capas y entidades**. El reparto que hagáis en el análisis no tiene que ser definitivo, pero sí tiene que ser explícito.

Plantilla de reparto para un grupo de 4:

Miembro	Rol principal	Área de desarrollo septiembre
Persona A	Arquitecto/a de datos	Backend: entidades + repositorios + BBDD
Persona B	Líder de API	Backend: controladores + Swagger
Persona C	Líder de Frontend	Frontend: modelos + servicios + componentes listado
Persona D	Gestor/a de proyecto	Frontend: componentes formulario + GitHub/Kanban



Note

En un proyecto real los roles son más fluidos y todos los miembros tocan todas las capas. En este curso el reparto inicial sirve para que no haya ambigüedad sobre quién empieza cada cosa. Después de los primeros sprints es normal que las personas ayuden fuera de su área.

Calendario de sprints (septiembre – entrega)

Planificad los sprints de desarrollo partiendo de la fecha de inicio (septiembre) hasta la entrega. Cada sprint dura **2 semanas**.

Plantilla de calendario — adaptadla a las fechas reales que os dé el instructor:

Sprint	Fechas	Objetivo	Entregable
Sprint 0	01–12 sep	Setup del entorno. Proyecto Spring Boot + conexión BBDD. Proyecto Angular + Bootstrap.	Aplicación arranca sin errores. <code>main</code> y <code>develop</code> en GitHub.
Sprint 1	15–26 sep	Backend completo de la entidad principal (entidad + repositorio + controlador CRUD).	Endpoints probados en Swagger.
Sprint 2	29 sep – 10 oct	Frontend de la entidad principal (modelo + servicio + componente lista + componente form).	Vista funcionando contra el backend real.
Sprint 3	13–24 oct	Backend de la entidad secundaria y la entidad intermedia.	Matrículas/relaciones funcionando en Swagger.
Sprint 4	27 oct – 07 nov	Frontend de la entidad secundaria y la gestión de la relación.	Todas las vistas principales funcionando.
Sprint 5	10–21 nov	Pulido: validaciones, mensajes de error, paginación, búsquedas.	Aplicación sin errores conocidos.
Sprint 6	24 nov – 05 dic	Extras opcionales: estadísticas, exportación, mejoras de UI. Demo final.	Presentación del proyecto.



Tip

El Sprint 0 es el más crítico: si el entorno no funciona al final de la primera semana de septiembre, todo lo demás se retrasa. Prestad especial atención a la instalación de Java 25, MySQL 8 y la versión correcta de Angular. Todos los comandos los tenéis en los apuntes

del curso principal.

Definición de "Hecho" (Definition of Done)

Antes de cerrar una tarea en el Kanban, verificad que cumple estos criterios:

- ☐ El código compila sin errores
- ☐ La funcionalidad está probada (Swagger para backend, navegador para frontend)
- ☐ El código está en una rama `feature/` y la PR ha sido revisada por al menos un compañero
- ☐ La PR está mergeada en `develop`
- ☐ La tarjeta del Kanban está en la columna "Hecho"
- ☐ Si es un endpoint nuevo: está documentado en Swagger con `@operation`
- ☐ Si es un componente nuevo: la ruta está añadida en `app.routes.ts`

Gestión de bloqueos

Un bloqueo es cualquier situación que impide a un miembro del equipo avanzar con su tarea. Protocolo:

1. El miembro bloqueado lo comunica en la siguiente reunión diaria (o antes si es urgente)
2. El equipo busca solución conjunta (máximo 15 minutos)
3. Si no se resuelve en 15 minutos, se escala al instructor
4. El bloqueo queda registrado en el acta de la reunión

Tarea de esta sesión

1. Completad la **tabla de reparto de tareas** con los nombres reales de vuestro grupo
2. Completad el **calendario de sprints** con las fechas reales de vuestro curso
3. Cread en el Kanban de GitHub las tarjetas correspondientes al **Sprint 0** y el **Sprint 1**, asignadas a cada miembro
4. Añadid en `ANALISIS.md` la sección de "Gestión y Planificación" con el reparto y el calendario



Sesión final — Defensa del Documento de Análisis

Tabla de contenidos

dentro de las 25 horas totales

Cada grupo dispondrá de **15 minutos** para presentar su Documento de Análisis:

- **10 minutos de presentación:** el grupo expone las secciones del documento. No hace falta presentación de diapositivas; se puede navegar directamente por el `ANALISIS.md` en Notable o VSCode
- **5 minutos de preguntas:** el instructor y el resto de grupos pueden preguntar sobre decisiones de diseño

Criterios de evaluación de la defensa:

Criterio	Descripción
Compleitud	El documento tiene todas las secciones requeridas
Coherencia	Las decisiones del modelo de datos, la API y el frontend son consistentes entre sí
Justificación	El equipo puede explicar por qué tomó cada decisión de diseño importante
GitHub listo	El repositorio existe, tiene la estructura correcta y el Kanban tiene tarjetas
Reparto claro	Queda claro quién hará qué en septiembre



Important

El documento no tiene que ser perfecto. Puede (y debe) evolucionar durante el desarrollo. Lo que se evalúa es que el equipo ha pensado en profundidad sobre su proyecto y que tiene un plan claro para empezar en septiembre.



Plantilla del Documento de Análisis

[Tabla de contenidos](#)

Esta es la estructura que debe tener vuestro `ANALISIS.md` al entregarlo. Copiadla como punto de partida y rellenad cada sección.

 **Estructura del ANALISIS.md**
Plantilla para el proyecto · entrega 31 julio 2026

Documento de Análisis · [Nombre del Proyecto]

Equipo (4) · Fecha · Inicio desarrollo: sept. 2026 · Repositorio GitHub

1

Descripción del sistema
Qué es · para quién · qué problema resuelve

2

Requisitos funcionales
RF-A / RF-B / RF-C por entidad (tabla código-requisito)

3

Requisitos no funcionales
RNF: stack, persistencia, frontend, API, Git...

4

Casos de uso
CU-nn · actor · acción · resultado · RF relacionado

5

Modelo de datos
5.1 Entidades y atributos · 5.2 Diagrama E-R (Mermaid)

mermaid erDiagram tabla de atributos

6

API REST
Verbo · ruta · entrada · respuesta OK · error, por controlador

7

Diseño del Frontend
7.1 Carpetas · 7.2 Rutas · 7.3 Interfaces · 7.4 Mockups

modelos/ servicios/ componentes/ mockups

8

Gestión y planificación
Repo · reparto · sprints · Definition of Done

Cada grupo copia esta estructura y rellena cada sección · crudalumnos es el ejemplo resuelto

Anexo A: Los Ciclos de Vida en un trámite administrativo

Tabla de contenidos

Cuando hicimos el CRUD de `crudaLumnos`, una matrícula era un registro **plano**: existía o no existía, y sus campos podían cambiar. Pero en el mundo real —y muy especialmente en la **Administración Pública**— casi nada es plano. Una solicitud de beca, una matrícula, un expediente de la Renta Mínima de Inserción... **no son un dato, son un proceso**: nacen, avanzan por una serie de **estados**, sufren idas y venidas (subsanciones, requerimientos, silencios) y finalmente se **cierran**. A eso lo llamamos el **ciclo de vida** de un trámite.

Este anexo es una guía visual de cómo se **analiza y representa** ese ciclo de vida, usando como caso guía un trámite administrativo real:

"Retomar Estudios" de la Universidad Pablo de Olavide (UPO). Es el hueco que muchos análisis olvidan: se modela la entidad (`Solicitud`, con sus campos) pero no su **comportamiento en el tiempo**. Un buen Documento de Análisis (FASE 8) debería incluir *siempre* el ciclo de vida de sus entidades principales.



Note

Este anexo nace de la experiencia real. El autor de estos apuntes fue responsable del proyecto de implementación de la **Renta Mínima de Inserción (RMI)** de la Junta de Andalucía, donde el ciclo de vida del expediente lo era **todo**: sin él, es imposible saber qué le puede pasar a una solicitud, quién puede actuar en cada momento ni qué plazos corren. El [simulador de la RMI](#) y el trámite de [Retomar Estudios de la UPO](#) son dos ejemplos vivos del mismo patrón.



A.1. ¿Qué es un ciclo de vida?

Tabla de contenidos

Un **ciclo de vida** es el conjunto de **estados** por los que pasa una entidad desde que se crea hasta que se archiva, junto con las **transiciones** permitidas entre ellos. Tres ideas clave:

1. **Estados**: situaciones estables en las que puede encontrarse la entidad (`Borrador`, `Registrada`, `En subsanación`, `Resuelta favorable`, `Cerrada` ...). En cada estado, unas acciones están permitidas y otras no.
2. **Transiciones**: los saltos de un estado a otro. No todas son posibles: de `Cerrada` no se vuelve a `Registrada`. Cada transición la **dispara** un evento (el ciudadano presenta algo, la Administración resuelve, vence un plazo...).
3. **Actores**: quién puede provocar cada transición. En la Administración suele haber al menos tres: el **ciudadano/a**, el **personal gestor** (quien tramita) y el **sistema** (que aplica plazos automáticos: silencio administrativo, caducidad...).

Esto es, ni más ni menos, una **máquina de estados finita** (FSM): el mismo concepto que en informática usamos para un semáforo, el carrito de una tienda o el reproductor de música. La Administración lleva **siglos** trabajando así (el procedimiento administrativo es una máquina de estados regulada por ley); nosotros solo la traducimos a software.



Tip

Regla práctica para detectar un ciclo de vida en un análisis: si una entidad tiene un campo que suele llamarse `estado`, `situación` o `fase`, y ese campo **cambia con el tiempo siguiendo reglas**, no tienes un dato: tienes una máquina de estados. Merece su propio diagrama.



A.2. El trámite de la UPO paso a paso

Tabla de contenidos

El caso guía tiene siete pasos. Los describimos primero en lenguaje natural (como los contaría la persona que hace el trámite) y luego los formalizamos:

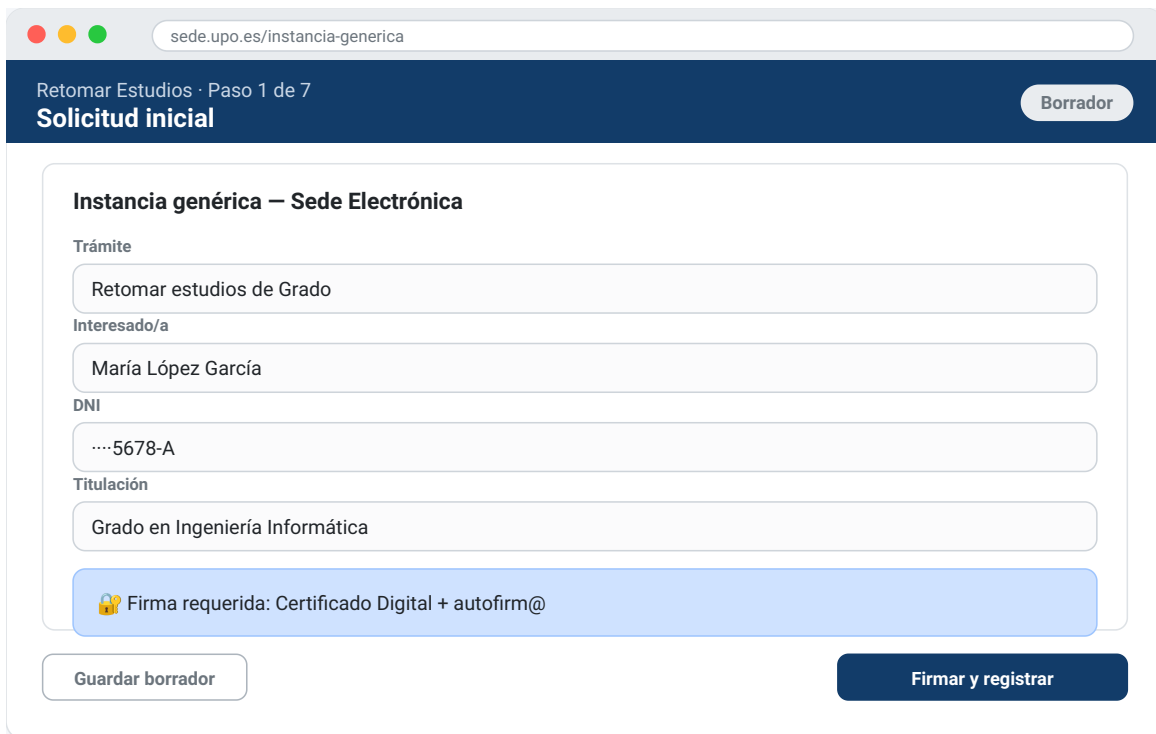
#	Paso	Actor principal	Qué ocurre
1	Solicitud inicial	Ciudadano/a	Presenta instancia genérica en la Sede Electrónica con certificado digital y autofirm@

#	Paso	Actor principal	Qué ocurre
2	Alta y seguimiento	Sistema / Gestor	La solicitud se registra, obtiene número de registro y entra en la cola de tramitación
3	Subsanación de errores	Gestor → Ciudadano/a	Si falta documentación, se requiere al interesado; se le informa y abre un plazo para corregir
4	Activación de usuario previo	Gestor / TIC	Se reactiva la cuenta institucional del estudiante (que llevaba años inactiva y no podía acceder)
5	Comunicación del plazo de matrícula	Sistema / Gestor	Concedido el retomar, se comunica al interesado su plazo personal de matrícula del curso 26/27
6	Matriculación	Ciudadano/a	Automatricula: un subproceso con su propio ciclo de vida (elección de asignaturas, pago...)
7	Cierre del procedimiento	Sistema	El expediente se archiva. Fin del ciclo

 Fíjate en el **paso 4**, el más fácil de olvidar en un análisis y el que más problemas causa en la vida real: la persona **no puede ni entrar** a su cuenta de la UPO porque estuvo años sin matricularse. Es un ejemplo perfecto de por qué el ciclo de vida importa: hay un **precondición técnica** (cuenta activa) que no forma parte del "trámite administrativo" en sentido estricto, pero sin la cual el ciudadano queda bloqueado. Un buen análisis lo detecta *antes* de que ocurra.

Veamos cada paso como lo percibe el usuario. Cada pantalla lleva, arriba, el número de paso y el **estado** del expediente en la máquina de estados que formalizaremos en A.3:

Paso 1 · Solicitud inicial — presenta la instancia genérica firmada (estado *Borrador* → *Registrada*):



sede.upo.es/instancia-generica

Retomar Estudios · Paso 1 de 7

Solicitud inicial Borrador

Instancia genérica — Sede Electrónica

Trámite

Retomar estudios de Grado

Interesado/a

María López García

DNI

....5678-A

Titulación

Grado en Ingeniería Informática

 Firma requerida: Certificado Digital + autofirm@

Guardar borrador Firmar y registrar

Paso 2 · Alta en el sistema y seguimiento — la solicitud obtiene número de registro y entra en tramitación (estado *Registrada*):

sede.upo.es/mi-expediente

Retomar Estudios · Paso 2 de 7

Alta en el sistema y seguimiento

Registrada

 **Solicitud registrada correctamente**

Nº de registro: 2026/12345 · Fecha: 03/07/2026 09:14

Ya puede consultar el estado en «Mi expediente».

SEGUIMIENTO

 Solicitud presentada

 Alta y registro

 En revisión por el gestor

 Resolución

Paso 3 · Subsanación de errores — se informa al usuario de lo que falta y del plazo (estado *En subsanación*):

sede.upo.es/expediente/subsanacion

Retomar Estudios · Paso 3 de 7

Subsanación de errores

En subsanación

 **Información al usuario: falta documentación**

Debe aportar para continuar:

- Justificante del traslado de expediente (si procede)

 Arrastra aquí el archivo o pulsa para seleccionar...

Plazo: hasta 17/07/2026. Si no responde → DESISTIDA (art. 68).

Enviar documentación

Paso 4 · Activación de la cuenta previa — la precondition técnica: reactivar la cuenta inactiva (estado *Concedida*):

tic.upo.es/cuentas/reactivar

Retomar Estudios · Paso 4 de 7

Activación de la cuenta previa

Concedida

 **Problema detectado**

La cuenta institucional de la estudiante lleva 2 cursos inactiva: no puede acceder a los servicios de la UPO.

Reactivación de cuenta — Servicios TIC

Usuario

maria.lopez@alu.upo.es

Estado

Inactiva → Activar

Solicitado por

Gestión Académica

✓ Reactivar y notificar a la interesada

Paso 5 · Comunicación del plazo de matrícula — se notifica al interesado su plazo personal (estado *Plazo comunicado*):

sede.upo.es/notificaciones

Retomar Estudios · Paso 5 de 7

Comunicación del plazo de matrícula

Plazo comunicado

 **Nueva notificación**

Estimada María López:

Le comunicamos que su solicitud de retomar estudios ha sido CONCEDIDA. Su plazo personal de matrícula para el curso 2026-2027 es:

Del 1 al 5 de septiembre de 2026

Automatricula en <https://matricula.upo.es>

Acceder a la automatricula

Paso 6 · Matriculación — la automatricula, un subproceso con su propio ciclo de vida (estado *Matriculada*):

matricula.upo.es/automatricula

Retomar Estudios · Paso 6 de 7

Matriculación (subproceso propio)

Matriculada

Automatricula · Curso 2026-2027

subproceso con su ciclo

✓

Asignaturas

✓

Grupos/horarios

✓

Descuentos

4

Pago

5

Confirmación

Asignaturas seleccionadas (24 ECTS)

• Bases de Datos

6 ECTS

• Programación

9 ECTS

• Fund. Orient. Laboral

3 ECTS

• Sistemas Informáticos

6 ECTS

Confirmar y pagar

Paso 7 · Cierre del procedimiento — el expediente se archiva (estado *Cerrada*):

sede.upo.es/mi-expediente

Retomar Estudios · Paso 7 de 7

Cierre del procedimiento

Cerrada

✓ **Procedimiento finalizado**

El expediente RE-2026-000842 ha sido archivado.
Matrícula del curso 2026-2027 formalizada correctamente.

RESUMEN DEL EXPEDIENTE

Solicitud presentada

02/07/2026

Subsanación resuelta

12/07/2026

Resolución

Concedida

Matriculación

01/09/2026

Cierre y archivo

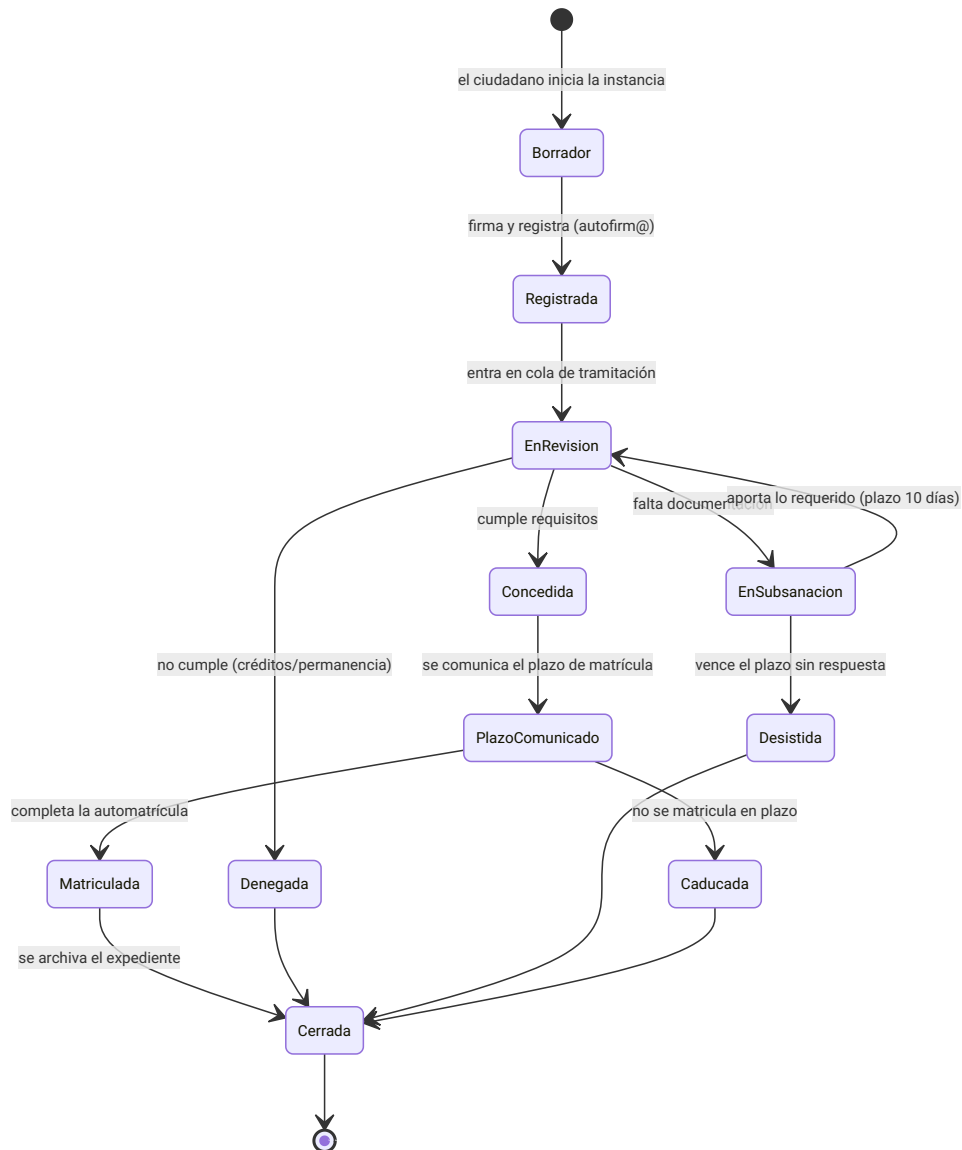
02/09/2026



A.3. Representación 1: la máquina de estados

Tabla de contenidos

La forma más compacta de ver un ciclo de vida es el **diagrama de estados**. Cada caja es un estado; cada flecha, una transición etiquetada con el evento que la dispara. Es la vista que responde a la pregunta: "estando aquí, ¿a dónde puedo ir y por qué?".



Observa tres cosas que este diagrama deja claras de un vistazo y que la lista de 7 pasos escondía:

- **La subsanación es un bucle:** EnRevisión → EnSubsanación → EnRevisión . No es un paso lineal; se puede entrar y salir.
- **Hay estados terminales de "fracaso":** Denegada , Desistida , Caducada . El trámite no siempre acaba en Matriculada . Un análisis que solo modele el "camino feliz" está incompleto.
- **Los plazos generan transiciones automáticas** (Desistida , Caducada): las dispara el **sistema**, no una persona. Son el equivalente informático del silencio administrativo.



Note

En `crudalumnos` esto se traduciría en un campo `estado` en la entidad `Solicitud` (un `enum`, como hicimos con `TipoDescuento` en la FASE 7) y en una tabla de transiciones permitidas. El paso 4 de la UPO (activar la cuenta) sería una

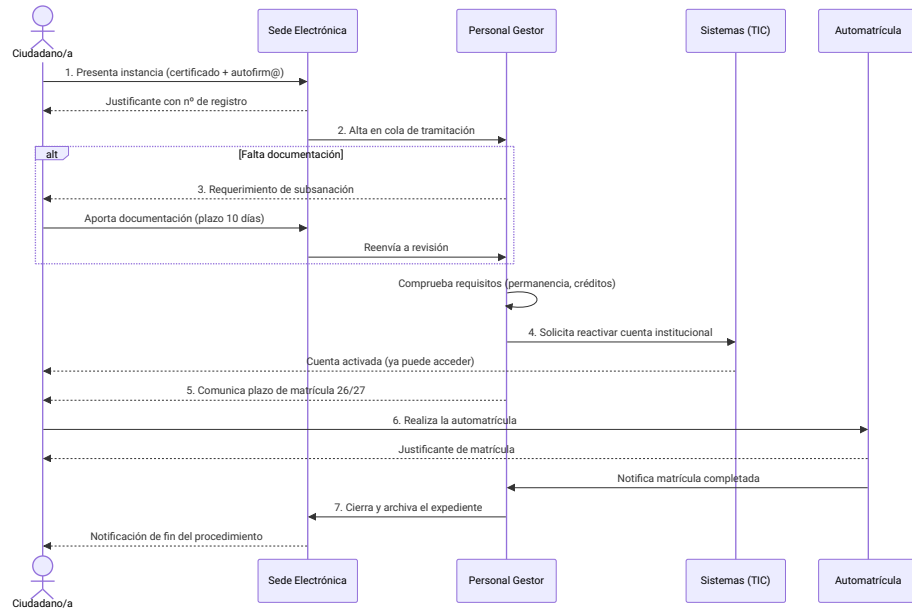
precondición de la transición Concedida → PlazoComunicado .



A.4. Representación 2: el diagrama de secuencia (actores ↔ sistema)

Tabla de contenidos

La máquina de estados dice *qué* estados hay, pero no *quién habla con quién* ni *en qué orden*. Para eso está el **diagrama de secuencia**: muestra el diálogo, en el tiempo, entre los actores y los sistemas. Es la vista que responde: "*¿quién le manda qué mensaje a quién, y en qué orden?*".



Este diagrama aporta lo que la máquina de estados no puede: el **paso 4 (TIC reactiva la cuenta)** aparece aquí como un mensaje a un actor **distinto** (los servicios informáticos), dejando claro que es una dependencia externa al circuito administrativo puro. También se ve que la Sede Electrónica devuelve **acuses** (el justificante de registro) que el ciudadano necesita como prueba.



Tip

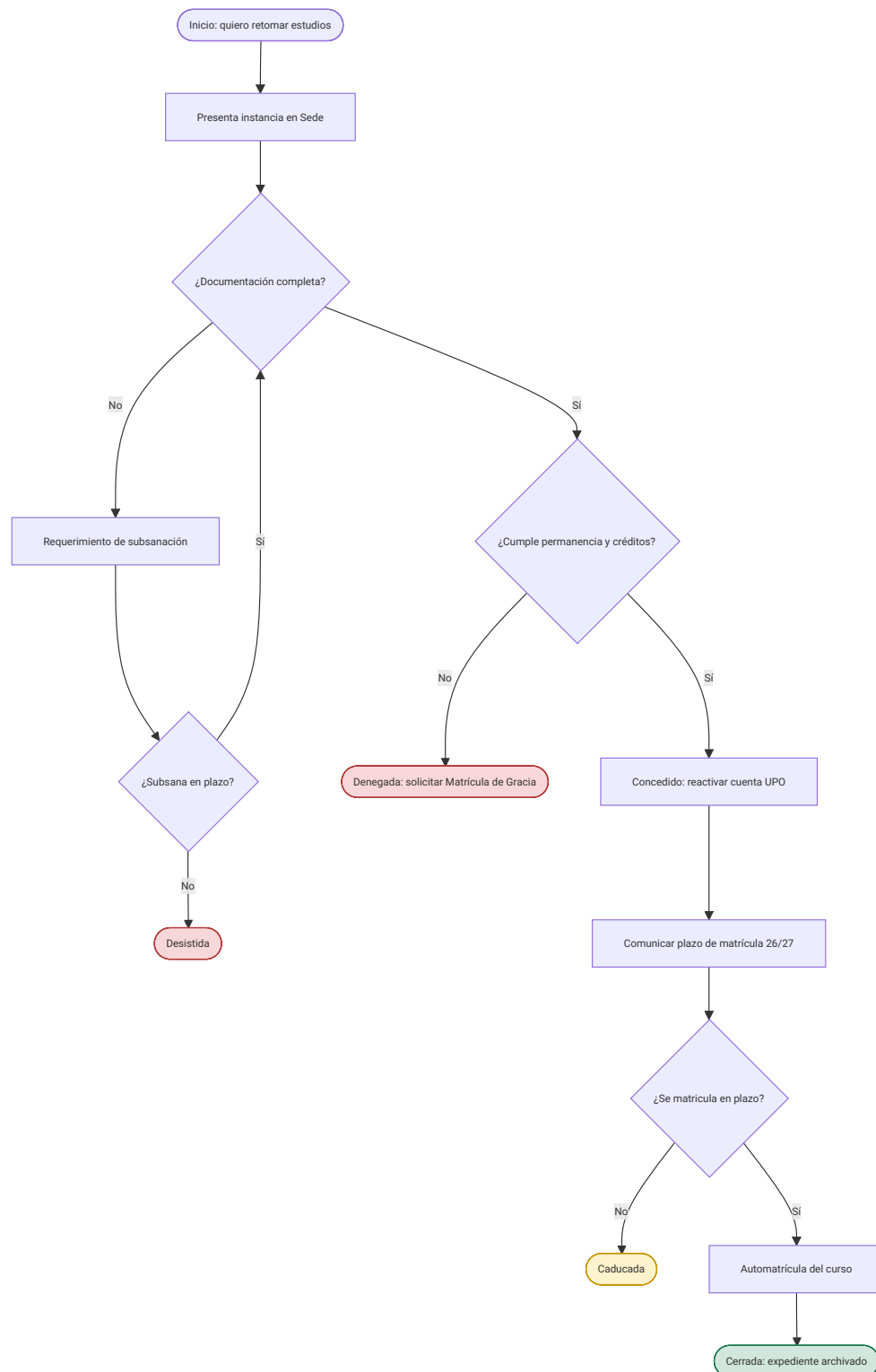
Regla de oro: la **máquina de estados** y el **diagrama de secuencia** son **complementarios, no alternativos**. El primero modela el **estado** de la entidad; el segundo, la **interacción** entre actores. En un análisis serio conviene tener los dos: describen la misma realidad desde dos ángulos.



A.5. Representación 3: el flujo con bifurcaciones (subsanación)

[Tabla de contenidos](#)

El **diagrama de flujo** (flowchart) pone el foco en las **decisiones**: cada rombo es un "¿sí o no?" que bifurca el camino. Es la vista ideal para explicar la **subsanción de errores** y las salidas alternativas, porque las decisiones son su protagonista.



El código de color no es decorativo, es semántico: **rojo** para las salidas por incumplimiento (`Desistida` , `Denegada`), **ámbar** para la salida por plazo (`Caducada`) y **verde** para el final satisfactorio (`Cerrada`). Un mismo trámite tiene **cuatro finales posibles**, y solo uno es el "camino feliz". Enseñar esto a los alumnos les vacuna contra el error más típico del principiante: programar solo el caso en que todo va bien.



Warning

No confundas los tres diagramas. El **flowchart** describe el *proceso* (los pasos y decisiones); la **máquina de estados**, la *entidad* (en qué situación está). Un flowchart puede tener dos cajas "Comprobar documentación" en sitios distintos; un estado `EnRevisión` es **uno solo**, se llegue a él por donde se llegue. Si tu diagrama de estados repite estados, probablemente estás dibujando un flujo disfrazado.



A.6. Representación 4: los mockups de las pantallas

Tabla de contenidos

Los diagramas anteriores son para el **equipo de desarrollo**. Pero el ciudadano no ve estados ni flechas: ve **pantallas**. La última representación traduce el ciclo de vida a lo que el usuario percibe. Igual que en la FASE 4 dibujábamos mockups con Bootstrap, aquí esbozamos las tres pantallas clave del trámite. Son **maquetas estáticas** (HTML + Bootstrap, sin lógica) que sirven para validar el diseño antes de programar.

A.6.1. Pantalla de seguimiento del expediente

El corazón de la experiencia: una línea de tiempo (*timeline*) donde el ciudadano ve **en qué estado** está su solicitud. Cada hito se corresponde con un estado de la máquina de A.3.

Mockup of the "Mi expediente" page on the "Sede Electrónica UPO" website. The page displays the current status of the application and a timeline of events.

Sede Electrónica UPO · Mi expediente

Solicitud de Retomar Estudios · Curso 2026-2027

Expediente RE-2026-000842 · Registro nº 2026/12345

Estado actual: En subsanación Acción requerida

- ✓ Solicitud presentada 02/07/2026
- ✓ Alta y registro en tramitación 03/07/2026
- ⌚ Subsanación requerida — ver requerimiento plazo hasta 17/07/2026
 - Resolución pendiente
 - Comunicación del plazo de matrícula pendiente
 - Matriculación pendiente

A.6.2. Aviso de subsanación (información al usuario)

Cuando el expediente entra en **EnSubsanación**, hay que **informar con claridad**: qué falta, hasta cuándo y qué pasa si no se corrige. Un buen aviso evita que la solicitud acabe **Desistida** por desconocimiento.

sede.upo.es/expediente/subsanacion

⚠ Se requiere subsanación

Para continuar con su solicitud, debe aportar la siguiente documentación:

- Justificante del pago del traslado de expediente (si procede).

Plazo: 10 días hábiles desde esta notificación (hasta el 17/07/2026).

Si no aporta la documentación en plazo, se considerará que **DESISTE** de su solicitud (art. 68 Ley 39/2015).

[📎 Aportar documentación](#) [Ver requerimiento completo \(PDF\)](#)

A.6.3. Panel del personal gestor (transiciones de estado)

La otra cara: el personal que **tramita** necesita ver el expediente y **disparar transiciones**. Los botones de esta pantalla se corresponden uno a uno con las flechas de la máquina de estados.

tramita.upo.es/expediente/RE-2026-000842

Expediente RE-2026-000842 · María López En subsanación

Grado en Ingeniería Informática · 2 cursos sin matrícula

TRANSICIONES DISPONIBLES

[✓ Conceder \(cumple requisitos\)](#)
[✗ Denegar \(permanencia\)](#)
[↩ Volver a revisión](#)

PRECONDICIÓN TÉCNICA

🔑 Cuenta institucional de la estudiante: inactiva
[Solicitar reactivación a TIC](#)

🔗 Observa la coherencia entre las cuatro representaciones: los **botones del panel del gestor** (A.6.3) son exactamente las **transiciones salientes** del estado **EnRevisión** en la máquina de estados (A.3), que a su vez son las **flechas** que salen del gestor en el diagrama de secuencia (A.4) y los **rombos de decisión** del flowchart (A.5). Cuando las cuatro vistas cuadran, el análisis es sólido. Cuando no cuadran, has encontrado un error **antes** de escribir código: justo el objetivo de la FASE 8.



A.7. Lo que los alumnos deben llevarse

Tabla de contenidos

Un resumen para el proyecto de septiembre:

1. **Toda entidad que "avanza" tiene un ciclo de vida.** Si en vuestro análisis aparece un campo `estado` , `situación` o `fase` , dibujad su máquina de estados. No os quedéis en la lista de campos.
2. **Modelad los finales tristes.** `Denegada` , `Desistida` , `Caducada` : el camino feliz es solo uno de los posibles. El software que solo contempla el éxito falla en producción el primer día.
3. **Identificad los actores y las precondiciones.** ¿Quién dispara cada transición? ¿Hay dependencias externas (como la cuenta de la UPO) que bloquean al usuario? Detectarlas en el análisis vale oro.
4. **Usad varias vistas.** Estados para la entidad, secuencia para la interacción, flujo para las decisiones, mockups para el usuario. Ninguna sobra; cada una revela un tipo de error distinto.
5. **La Administración es una máquina de estados con siglos de antigüedad.** El procedimiento administrativo (Ley 39/2015) ya define estados, plazos y silencios. Programar la Administración es, en buena medida, **traducir esa máquina de estados a software**.



Tip

Ejercicio propuesto para el proyecto: elegid **un** trámite real que hayáis vivido (una beca, un carné, una cita médica, un alta de autónomo) y modeladlo con las cuatro representaciones de este anexo. Descubriréis que un trámite "sencillo" esconde, casi siempre, más de tres finales posibles y al menos un bucle de subsanación.

Citas Coloreadas

Tabla de contenidos



Note

Highlights information that users should take into account, even when skimming.



Tip

Optional information to help a user be more successful.



Crucial information necessary for users to succeed.



Warning

Critical content demanding immediate user attention due to potential risks.



Caution

Negative potential consequences of an action.